

CHAPTER

Syntax

6.1	Lexical Structure of Programming Languages	204
6.2	Context-Free Grammars and BNFs	208
6.3	Parse Trees and Abstract Syntax Trees	213
6.4	Ambiguity, Associativity, and Precedence	216
6.5	EBNFs and Syntax Diagrams	220
6.6	Parsing Techniques and Tools	224
6.7	Lexics vs. Syntax vs. Semantics	235
6.8	Case Study: Building a Syntax Analyzer for TinyAda	237

Syntax is the structure of a language. In Chapter 1, we noted the difference between the syntax and semantics of a programming language. In the early days of programming languages, both the syntax and semantics of a language were described by lengthy English explanations and many examples. While the semantics of a language are still usually described in English, one of the great advances in programming languages has been the development of a formal system for describing syntax that is now almost universally in use. In the 1950s, Noam Chomsky developed the idea of context-free grammars, and John Backus, with contributions by Peter Naur, developed a notational system for describing these grammars that was used for the first time to describe the syntax of Algol60. These **Backus-Naur forms—BNFs** for short—have subsequently been used in the definition of many programming languages, including C, Java, and Ada. Indeed, every modern programmer and computer scientist needs to know how to read, interpret, and apply BNF descriptions of language syntax. BNFs occur with minor textual variations in three basic forms: original BNF; extended BNF (EBNF), popularized by Niklaus Wirth (and very briefly used to describe the syntax of Scheme in Chapter 3); and syntax diagrams.

In Section 6.1, we briefly look at lexical structures, which are the building blocks on which syntax is often based. We also explore the process of scanning, or recognizing, lexical structure. In Section 6.2, we introduce context-free grammars and their description in BNF. In Section 6.3, we describe the representation of syntactic structure using trees. In Section 6.4, we consider a few of the issues that arise in constructing BNFs for a programming language. EBNFs and syntax diagrams are introduced in Section 6.5. In Section 6.6, we discuss the basic technique of recursive-descent parsing and its close relationship to EBNF and syntax diagrams and briefly look at YACC/Bison, a standard tool for analyzing grammars and constructing parsers. In Section 6.7, we examine the sometimes hazy boundaries among the lexical, syntactic, and semantic structures of a programming language. Finally, Section 6.8 puts some of these ideas to work by developing a syntax analyzer for a small programming language.

6.1 Lexical Structure of Programming Languages

The **lexical structure** of a programming language is the structure of its **tokens**, or words. A language's lexical structure and its syntactic structure are two different things, but they are closely related. Typically, during its **scanning** phase, a translator collects sequences of characters from the input program and forms them into tokens. During its **parsing** phase, the translator then processes these tokens, thereby determining the program's syntactic structure.

Tokens generally fall into several distinct categories. Typical token categories include the following:

- **reserved words**, sometimes called **keywords**, such as `if` and `while`
- **literals** or **constants**, such as `42` (a numeric literal) or `"hello"` (a string literal)
- **special symbols**, such as `“;”`, `“<=”`, or `“+”`
- **identifiers**, such as `x24`, `monthly_balance`, or `putchar`

Reserved words are so named because an identifier cannot have the same character string as a reserved word. Thus, in C, the following variable declaration is illegal because `if` is a reserved word:

```
double if;
```

Sometimes programmers are confused about the difference between reserved words and **predefined identifiers**, or identifiers that have been given an initial meaning for all programs in the language but that are capable of redefinition. (To add to the confusion, sometimes these identifiers are also called keywords.) Examples of predefined identifiers are all standard data types in Ada, such as `integer` and `boolean`, and in Python, such as `round` and `print`. See Section 6.7 for more on this phenomenon.

In some older languages identifiers had a fixed maximum size, while in most contemporary languages identifiers can have arbitrary length. Occasionally, even when arbitrary-length identifiers are allowed, only the first six or eight characters are guaranteed to be significant. (You can see how this can be a source of confusion for programmers.)

A problem arises in determining the end of a variable-length token such as an identifier and in distinguishing identifiers from reserved words. As an example, the sequence of characters:

```
doif
```

in a program could be either the two reserved words `do` and `if` or the identifier `doif`. Similarly, the string `x12` could be a single identifier or the identifier `x` and the numeric constant `12`. To eliminate this ambiguity, it is a standard convention to use the **principle of longest substring** in determining tokens (sometimes called the principle of “maximum munch”). The principle works like this: At each point, the longest possible string of nonblank characters is collected into a single token. This means that `doif` and `x12` are always identifiers. It also means that intervening characters, even blanks, can make a difference. Thus, in most languages:

```
do if
```

is not an identifier; instead, it is the two reserved words `do` and `if`.

The format of a program can affect the way tokens are recognized. For example, as you just saw, the principle of longest substring requires that certain tokens be separated by **token delimiters** or **white space**. The end of a line of text can be doubly meaningful because it can be white space, but it can also mean the end of a structural entity. Indentation can also be used in a programming language to determine structure. In fact, Python’s simple syntax rests on the significance of indentation.

A **free-format** language is one in which format has no effect on the program structure (other than to satisfy the principle of longest substring). Most modern languages are free format, but a few have significant format restrictions. Rarely is a language **fixed format**, in which all tokens must occur in prespecified locations on the page.

Tokens in a programming language are often described in English, but they can also be described formally by **regular expressions**, which are descriptions of patterns of characters. Regular expressions have three basic operations: concatenation, repetition, and choice or selection. Repetition is indicated by the use of an asterisk after the item to be repeated, choice is indicated by a vertical bar between the items from which the selection is to be made, and concatenation is given simply by sequencing the items without an explicit operation. Parentheses are also often included to allow for the grouping of operations. For example, `(a|b)*c` is a regular expression indicating 0 or more repetitions of either the characters `a` or `b` (choice), followed by a single character `c` (concatenation); strings that match this regular expression include `ababaaac`, `aac`, `babbc`, and just `c` itself, but not `aaaab` or `bca`.

Regular expression notation is often extended by additional operations and special characters to make them easier to write. For example, square brackets with a hyphen indicate a range of characters, `+` indicates one or more repetitions, `?` indicates an optional item, and a period indicates any character. With these notations, we can write compact regular expressions for even fairly complex tokens. For example,

`[0-9]+`

is a regular expression for simple integer constants consisting of one or more digits (characters between 0 and 9). Also, the regular expression:

`[0-9]+(\\. [0-9]+)?`

describes simple (unsigned) floating-point literals consisting of one or more digits followed by an optional fractional part consisting of a decimal point (with a backslash, or escape, character in front of it to remove the special meaning of a period as matching any character), followed again by one or more digits.

Many utilities use regular expressions in text searches. This is true of most modern text editors, as well as various file search utilities, such as UNIX `grep` (“global regular expression print”). The UNIX `lex` utility can also be used to automatically turn a regular expression description of the tokens of a language into a scanner. (A successor to `lex`, called `flex`, or fast `lex`, is freely available and runs on most operating systems.)

While a study of `lex/flex` and the use of regular expressions to construct scanners is beyond the scope of this text, in simple cases, scanners can be constructed by hand relatively easily. To give you a sense of such a scanner construction, we present C code for a simple scanner in Figure 6.1. There are six tokens in this language (represented as constants of an enumeration type, lines 4–5): integer constants consisting of one or more digits; the special characters `+`, `*`, `(`, and `)`; and the end of line token (represented in C or Java as the special character `'\n'`, or newline). Such a language may be used to specify a simple integer arithmetic expression with addition and multiplication on a single line of input, such as

`(2 + 34) * 42`

Blanks are to be skipped, and any character (such as `a` or `#` or `-`) that is not specified is to be considered an error (thus, the need for the additional `ERROR` token on line 5).

Before closing this section and moving on to the study of syntax and parsing, we note the following additional features of the code in Figure 6.1. First, the code contains a `main` function (lines 31–47) to test drive the scanner (which is represented by the `getToken` function, lines 8–30). Given a line of input, the `main` function simply calls `getToken` repeatedly and prints each token as it is recognized; a number is also printed with its numeric value, and an error is printed with the offending character. Second, we note

that the scanner communicates the additional numeric and character information to the main program via the global variables `numVal` and `currChar` (lines 6 and 7). A more sophisticated design would eliminate these globals and package this information into a data structure containing all token information; see Exercise 6.7.

```
(1) #include <ctype.h>
(2) #include <stdio.h>

(3) /* the tokens as an enumerated type */
(4) typedef enum {TT_PLUS, TT_TIMES, TT_PAREN, TT_RPAREN,
(5)               TT_EOL, TT_NUMBER, TT_ERROR} TokenType;

(6) int numVal; /* computed numeric value of a NUMBER token */
(7) int currChar; /* current character */

(8) TokenType getToken(){
(9)     while (currChar == ' ') /* skipblanks */
(10)         currChar = getchar();
(11)     if (isdigit(currChar)){ /* recognize a NUMBER token */
(12)         numval = 0;
(13)         while (isdigit(currChar)){
(14)             /* compute numeric value */
(15)             numval = 10 * numval + currChar - '0';
(16)             currChar = getchar();
(17)         }
(18)         return TT_NUMBER;
(19)     }
(20)     else{ /* recognize a special symbol */
(21)         switch (currChar){
(22)             case '(': return TT_LPAREN; break;
(23)             case ')': return TT_RPAREN; break;
(24)             case '+': return TT_PLUS; break;
(25)             case '*': return TT_TIMES; break;
(26)             case '\n': return TT_EOL; break;
(27)             default: return TT_ERROR; break;
(28)         }
(29)     }
(30) }
(31) main(){
(32)     TokenType token;
(33)     currChar = getchar();
(34)     do{
(35)         token = getToken();
(36)         switch (token){
(37)             case TT_PLUS: printf("TT_PLUS\n"); break;
```

Figure 6.1 A scanner for simple integer arithmetic expressions (*continues*)

(continued)

```

(38)         case TT_TIMES: printf("TT_TIMES\n"); break;
(39)         case TT_LPAREN: printf("TT_LPAREN\n"); break;
(40)         case TT_RPAREN: printf("TT_RPAREN\n"); break;
(41)         case TT_EOL:   printf("TT_EOL\n"); break;
(42)         case TT_NUMBER: printf("TT_NUMBER: %d\n", numval); break;
(43)         case TT_ERROR:  printf("TT_ERROR: %c\n", curr_char); break;
(44)     }
(45) } while (token != TT_EOL);
(46) return 0;
(47) }

```

Figure 6.1 A scanner for simple integer arithmetic expressions

Given the input line:

* + () 42 # 345

the program of Figure 6.1 produces the following output:

```

TT_TIMES
TT_PLUS
TT_LPAREN
TT_RPAREN
TT_NUMBER: 42
TT_ERROR: #
TT_NUMBER: 345
TT_EOL

```

Note that no order for the tokens is specified by the scanner. Defining and recognizing an appropriate order is the subject of syntax and parsing, studied in the remainder of this chapter.

6.2 Context-Free Grammars and BNFs

We begin our description of grammars and BNFs with the example in Figure 6.2 (the numbers are for reference purposes).

- (1) *sentence* $\rightarrow$ *noun-phrase verb-phrase* .
- (2) *noun-phrase* $\rightarrow$ *article noun*
- (3) *article* $\rightarrow$ a | the
- (4) *noun* $\rightarrow$ girl | dog
- (5) *verb-phrase* $\rightarrow$ *verb noun-phrase*
- (6) *verb* $\rightarrow$ sees | pets

Figure 6.2 A grammar for simple english sentences

In English, simple sentences consist of a noun phrase and a verb phrase followed by a period. We express this as the grammar rule 1 in Figure 6.2. Rules 2 through 6 describe in turn the structure of noun phrases, verb phrases, and other substructures that appear in previous rules. Each of these grammar rules consists of a name in italics (the name of the phrase being described), followed by an arrow (which can be read as “consists of” or “is defined as,”), followed by a sequence of other names and symbols. The italics serve to distinguish the names of the phrases from the actual words, or tokens, that may appear in the language (which, in our examples, are also indicated in a different typeface). For instance, we would not wish to confuse the keyword *program* in Pascal with the program structure itself:

$$\textit{program} \rightarrow \text{program} \dots$$

Typically, of course, we would prefer also to use different names to distinguish phrases from tokens.

The symbol “ $\rightarrow$ ” is a **metasymbol** that serves to separate the left-hand side from the right-hand side of a rule. The vertical bar “|” is also a metasymbol and means “or” or choice (similar to its use in regular expressions). Thus, rule 6 above states that a verb is either the word “sees” or the word “pets.” Sometimes a metasymbol is also an actual symbol in a language. In that case the symbol can be surrounded by quotation marks to distinguish it from the metasymbol, or the metasymbol can be written in a different typeface, or both. Often it is a good idea to do this for special symbols like punctuation marks, even when they are not also metasymbols. For example, rule 1 has a period in it. While a period is not part of any metasymbol described, it can easily be mistaken for one. Hence, it might be better to write the rule as follows:

$$\textit{sentence} \rightarrow \textit{noun-phrase} \textit{ verb-phrase} \text{ '}'$$

In this case, then, the quotation marks also become metasymbols.

Some notations also rely on metasymbols (such as angle brackets) rather than italics or fonts to distinguish phrases from tokens, which can also be useful in situations where formatting is not available (such as in text files or handwritten text). In that case, the arrow is also often replaced by a metasymbol that is also pure text (such as two colons and an equal sign). For example, we might see rule 1 of Figure 6.2 written as follows:

$$\langle \textit{sentence} \rangle ::= \langle \textit{noun-phrase} \rangle \langle \textit{verb-phrase} \rangle \text{ "}'$$

Here the angle brackets also become metasymbols, and double quotation marks have been used instead of single quotation marks to mark the period as a token.

Indeed, there are many different, often personal, preferences in the notational conventions used in writing grammar rules. There is also an ISO standard format for BNF notation (ISO 14977 [1996]), which is almost universally ignored, perhaps because, by the time of its adoption (1996), other conventions such as the above were already entrenched. For example, the grammar rule for a sentence would appear in standard ISO form as:

$$\textit{sentence} = \textit{noun phrase} , \textit{ verb phrase} \text{ "}' ;$$

Any legal sentence according to the foregoing grammar can be constructed as follows: We start with the symbol *sentence* (the **start symbol** for the grammar) and proceed to replace left-hand sides by choices of right-hand sides in the foregoing rules. This process is called a **derivation** in the language. Thus, we could construct, or derive, the sentence “the girl sees a dog.” as in Figure 6.3, where each step is annotated by the number of the rule from Figure 6.2 that is used in that step.

```

sentence  $\Rightarrow$  noun-phrase verb-phrase . (rule 1)
            $\Rightarrow$  article noun verb-phrase . (rule 2)
            $\Rightarrow$  the noun verb-phrase . (rule 3)
            $\Rightarrow$  the girl verb-phrase . (rule 4)
            $\Rightarrow$  the girl verb noun-phrase . (rule 5)
            $\Rightarrow$  the girl sees noun-phrase . (rule 6)
            $\Rightarrow$  the girl sees article noun . (rule 2)
            $\Rightarrow$  the girl sees a noun . (rule 3)
            $\Rightarrow$  the girl sees a dog . (rule 4)

```

Figure 6.3 A derivation using the grammar of Figure 6.2

Conversely, we could start with the sentence “the girl sees a dog.” and work backward through the derivation of Figure 6.3 to arrive at *sentence* and so have shown that the sentence is legal in the language.

The simple grammar of Figure 6.2 already exhibits most of the properties of programming language grammars. Note that you can’t assume that just because sentence is legal that it actually makes sense in a real-life context. For example, “the dog pets the girl.” is a nonsensical, yet legal sentence. Note that this example also contains a subtle error: Articles that appear at the beginning of sentences should be capitalized. This type of property, known as a positional property, is hard to represent using context-free grammars. Also, we have written the tokens in the foregoing sentence with spaces between the tokens, but the grammar does not specify whether such spaces are necessary (we might have easily written “thegirlseesadog.”). This question is left to a scanner, which consumes white-space characters and recognizes the individual tokens or words, as noted in the previous section. (See also Exercise 6.9.) Finally, the grammar also does not specify additional requirements on input format, such as the fact that a sentence should be followed by some kind of termination symbol (such as a newline or end of file marker). Occasionally, we need to make this latter condition explicit in a grammar, writing a rule such as:

$$\text{input} \rightarrow \text{sentence } \$$$

to indicate that a sentence is to be followed by some kind of end marker. The marker is indicated by the dollar sign, and may or may not need to be generated explicitly.

Let’s take a moment to formulate some informal definitions for the topics discussed so far.

A **context-free grammar** consists of a series of grammar rules. These rules, in turn, consist of a left-hand side that is a single phrase structure name, followed by the metasymbol “ $\rightarrow$ ”, followed by a right-hand side consisting of a sequence of items that can be symbols or other phrase structure names. The names for phrase structures (like *sentence*) are called **nonterminals**, since they are broken down into further phrase structures. The words or token symbols are also called **terminals**, since they cannot be broken down. Grammar rules are also called **productions**, since they “produce” the strings of the language using derivations. Productions are in **Backus-Naur form** if they are as given using only the metasymbols “ $\rightarrow$ ” and “[]”. (Sometimes parentheses are also allowed to group things together.)

A context-free grammar also has a special nonterminal called the **start symbol**, as noted above. This nonterminal stands for the entire, top-level phrase being defined (such as a sentence or a program),

and is the symbol that all derivations begin with. A context-free grammar defines a language, called the **language of the grammar**. This language is the set of all strings of terminals for which there exists a derivation beginning with the start symbol and ending with the string of terminals.

Figure 6.2 includes seven terminals (“girl,” “dog,” “sees,” “pets,” “the,” “a,” and “.”), six non-terminals, and six productions. The language defined by this grammar is also finite, meaning that there are only finitely many sentences that can be derived in the language (see Exercise 6.8), but in general languages defined by grammars are not finite.

Typically there are as many productions in a context-free grammar as there are nonterminals, although one could eliminate the “|” metasymbol by writing each choice separately, such as:

$$\begin{aligned} \text{noun} &\rightarrow \text{girl} \\ \text{noun} &\rightarrow \text{dog} \end{aligned}$$

in which case each nonterminal would correspond to as many productions as there are choices.

Why is such a grammar **context-free**? The simple reason is that the nonterminals appear singly on the left-hand sides of productions. This means that each nonterminal can be replaced by any right-hand side choice, no matter where the nonterminal might appear. In other words, there is no **context** under which only certain replacements can occur. For example, in the grammar just discussed, it makes sense to use the verb “pets” only when the subject is “girl”; this can be thought of as a context sensitivity. We can write out context-sensitive grammars by allowing context strings to appear on left-hand sides of the grammar rules. Some authors consider context sensitivities to be purely syntactic issues. We shall adopt the view, however, that anything not expressible using context-free grammars is a semantic, not a syntactic, issue. (Even some things that *are* expressible as context-free grammars are often better left to semantic descriptions, since they involve writing many extra productions; see Exercise 6.30.)

As an example of a context sensitivity, recall that, in the preceding grammar, articles at the beginning of sentences should be capitalized. One way of doing this is to rewrite the first rule as:

$$\text{sentence} \rightarrow \text{beginning noun-phrase verb-phrase} \text{ ' . '}$$

and then add the context-sensitive rule:

$$\text{beginning article} \rightarrow \text{The} \mid \text{A}$$

Now the derivation would look as follows:

$$\begin{aligned} \text{sentence} &\Rightarrow \text{beginning noun-phrase verb-phrase} \text{ . (new rule 1)} \\ &\Rightarrow \text{beginning article noun verb-phrase} \text{ . (rule 2)} \\ &\Rightarrow \text{The noun verb-phrase} \text{ . (new context-sensitive rule)} \\ &\Rightarrow \dots \end{aligned}$$

Context-free grammars have been studied extensively by formal language theorists and are now so well understood that it is natural to express the syntax of any programming language in BNF form. Indeed, doing so makes it easier to write translators for the language, since the parsing stage can be automated, as you will learn in Section 6.6.

A typical simple example of the use of a context-free grammar in a programming language is the description of simple integer arithmetic expressions with addition and multiplication given in Figure 6.4.

```

expr → expr + expr | expr * expr | ( expr ) | number
number → number digit | digit
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

Figure 6.4 A simple integer arithmetic expression grammar

Note the recursive nature of the rules: An expression can be the sum or product of two expressions, each of which can be further sums or products. Eventually, of course, this process must stop by selecting the *number* alternative, or we would never arrive at a string of terminals.

Note also that the recursion in the rule for *number* is used to generate a repeated sequence of digits. For example, the number 234 is constructed as in Figure 6.5.

```

number ⇒ number digit
        ⇒ number digit digit
        ⇒ digit digit digit
        ⇒ 2 digit digit
        ⇒ 23 digit
        ⇒ 234

```

Figure 6.5 A derivation for the *number* 234 using the grammar of Figure 6.4

This process could actually be performed by a scanner, as we have noted in the previous section, since it deals only with sequences of characters. Indeed, the grammar above has as its terminals only single character tokens such as + and 9, and the question of white space is ignored. We will return to this question in Section 6.7, but for simplicity we ignore scanning questions for the moment.

As a more complex example of a grammar in BNF, Figure 6.6 shows the beginning of a BNF description for the C language.

```

translation-unit → external-declaration
                  | translation-unit external-declaration

external-declaration → function-definition | declaration

function-definition → declaration-specifiers declarator
                      declaration-list compound-statement
                  | declaration-specifiers declarator compound-statement
                  | declarator declaration-list compound-statement
                  | declarator compound-statement

declaration → declaration-specifiers ‘;’
              | declaration-specifiers init-declarator-list ‘;’

init-declarator-list → init-declarator

```

Figure 6.6 Partial BNFs for C (adapted from Kernighan and Ritchie [1988]) (*continues*)

(continued)

```

| init-declarator-list ' , ' init-declarator

init-declarator → declarator | declarator '=' initializer
declarator → pointer direct-declarator | direct-declarator

pointer → '*' type-qualifier-list pointer | '*' type-qualifier-list
| '*' pointer | '*'

direct-declarator → ID
| '(' declarator ')' | direct_declarator '[' ']'
| direct_declarator '[' 'constant_expression' ']'
| direct_declarator '(' parameter_type_list ')'
| direct_declarator '(' identifier_list ')'
| direct_declarator '(' ' ' ')'
...
...
```

Figure 6.6 Partial BNFs for C (adapted from Kernighan and Ritchie [1988])

An alternative way of explaining how BNF rules construct the strings of a language is as follows. Given a grammar rule such as:

$$expr \rightarrow expr + expr \mid number$$

(abstracted from Figure 6.4), let **E** be the set of strings representing expressions, and let **N** represent the set of strings representing numbers. The preceding grammar rule can be viewed as a **set equation**:

$$\mathbf{E} = \mathbf{E} + \mathbf{E} \cup \mathbf{N}$$

where $\mathbf{E} + \mathbf{E}$ is the set constructed by concatenating all strings from **E** with the “+” symbol and then all strings from **E** and “ $\cup$ ” make up set union. Assuming that the set **N** has already been constructed, this represents a recursive equation for the set **E**. *The smallest set E satisfying this equation* can be taken to be the set defined by the grammar rule. Intuitively, this consists of the set:

$$\mathbf{N} \cup \mathbf{N} + \mathbf{N} \cup \mathbf{N} + \mathbf{N} + \mathbf{N} \cup \mathbf{N} + \mathbf{N} + \mathbf{N} + \mathbf{N} \cup \dots$$

In Exercise 6.53, you will have the chance to prove that this is the smallest set satisfying the given equation. Solutions to recursive equations appear frequently in formal descriptions of programming languages, and are a major object of study in the theory of programming languages.

6.3 Parse Trees and Abstract Syntax Trees

Syntax establishes structure, not meaning, but the meaning of a sentence (or program) is related to its syntax. For example, in English a sentence has a subject and a predicate. These are semantic concepts, since the subject (the “actor”) and the predicate (the “action”) determine the meaning of the sentence. A subject generally comes at the beginning of a sentence and is given by a noun phrase. Thus, in the syntax of an English sentence, a noun phrase is placed first and is subsequently associated with a subject. Similarly, in the grammar for expressions, when we write:

$$expr \rightarrow expr + expr$$

we expect to add the values of the two right-hand expressions to get the value of the left-hand expression. This process of associating the semantics of a construct to its syntactic structure is called **syntax-directed semantics**. We must, therefore, construct the syntax so that it reflects the semantics we will eventually attach to it as much as possible. (Syntax-directed semantics could just as easily have been called semantics-directed syntax.)

To make use of the syntactic structure of a program to determine its semantics, we must have a way of expressing this structure as determined by a derivation. A standard method for doing this is with a **parse tree**, which is a graphical depiction of the replacement process in a derivation. For example, the parse tree for the sentence “the girl sees a dog.” is shown in Figure 6.7.

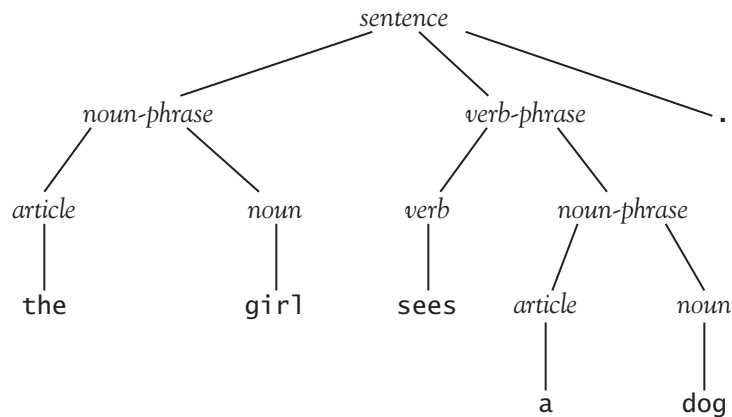


Figure 6.7: Parse tree for the sentence “the girl sees a dog.”

Similarly, the parse tree for the number 234 in the expression grammar is shown in Figure 6.8.

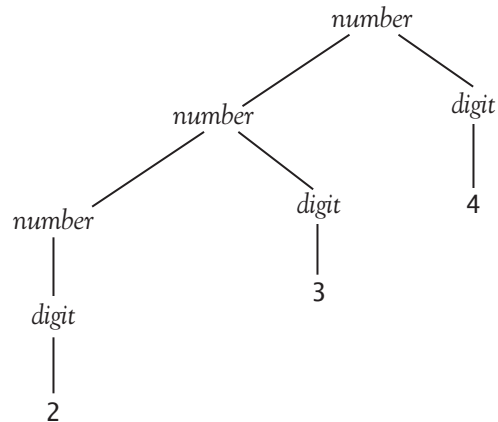


Figure 6.8: Parse tree for the number 234

A parse tree is labeled by nonterminals at interior nodes (nodes with at least one child) and terminals at leaves (nodes with no children). The structure of a parse tree is completely specified by the grammar rules of the language and a derivation of a particular sequence of terminals. The grammar rules specify

the structure of each interior node, and the derivation specifies which interior nodes are constructed. For example, the tree for the number 234 is specified by the grammar rules for *number* and the derivation of 234 as given in the previous section. Indeed, the first two steps of the derivation are:

$$\begin{aligned} \text{number} &\Rightarrow \text{number digit} && \text{(step 1)} \\ &\Rightarrow \text{number digit digit} && \text{(step 2)} \end{aligned}$$

and correspond to the construction of the two children of the root (step 1) and the two children of the left child of the root (step 2). Not surprisingly, there are exactly as many steps in the derivation of 234 as there are interior nodes in the parse tree.

All the terminals and nonterminals in a derivation are included in the parse tree. However, not all the terminals and nonterminals are necessary to determine completely the syntactic structure of an expression or sentence. For example, the structure of the number 234 can be completely determined from the tree shown in Figure 6.9.

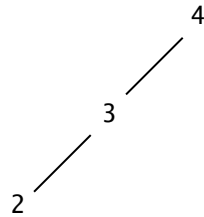
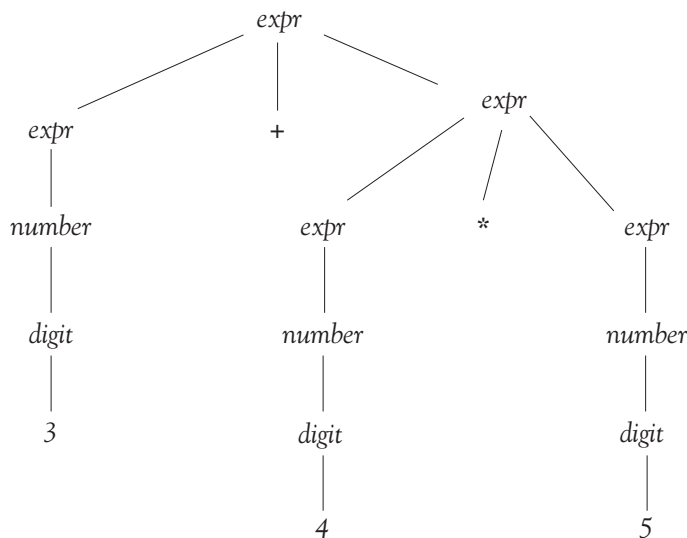


Figure 6.9: Parse tree for determining structure of the number 234

As another example, Figure 6.10 shows how a parse tree for $3 + 4 * 5$ can be condensed.

Complete parse tree



Condensed parse tree

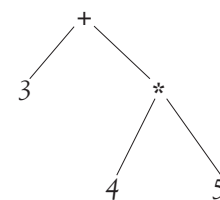


Figure 6.10: Condensing parse tree for $3 + 4 * 5$

Such trees are called **abstract syntax trees** or just **syntax trees**, because they abstract the essential structure of the parse tree. Abstract syntax trees may also do away with terminals that are redundant once the structure of the tree is determined. For example, the grammar rule:

$$\textit{if-statement} \rightarrow \textit{if} \, (\, \textit{expression} \,) \, \textit{statement} \, \textit{else} \, \textit{statement}$$

gives rise to the parse tree and abstract syntax tree shown in Figure 6.11.

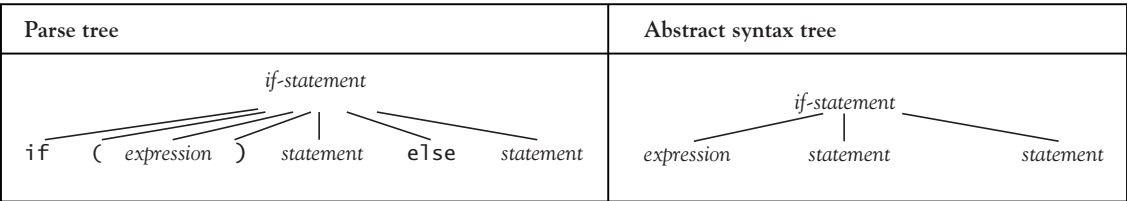


Figure 6.11: Parse tree and abstract syntax tree for grammar rule $\textit{if-statement} \rightarrow \textit{if} \, (\, \textit{expression} \,) \, \textit{statement} \, \textit{else} \, \textit{statement}$

It is possible to write out rules for abstract syntax in a similar way to the BNF rules for ordinary syntax. For example, the abstract syntax for an *if-statement* corresponding to the previous BNF rule could be written simply as:

$$\textit{if-statement} \rightarrow \textit{expression} \, \textit{statement} \, \textit{statement}$$

Of course, this can also be a little confusing, since this is not what we would see in an actual program. For this reason, abstract syntax is typically of less interest to the programmer and is often left unspecified. Sometimes ordinary syntax is **concrete syntax** to distinguish it from abstract syntax. Of course, abstract syntax is of great importance to the language designer and translator writer, since it is the abstract, not the concrete, syntax that expresses a language’s essential structure, and a translator will often construct a syntax tree rather than a full parse tree because it is more concise. In this text, rather than specifying both concrete and abstract syntax, we will focus on the concrete syntax and infer typical abstract structures for syntax trees directly from the concrete syntax.

6.4 Ambiguity, Associativity, and Precedence

Two different derivations can lead to the same parse tree or syntax tree: In the derivation for 234 in Figure 6.5, we could have chosen to replace the *digit* nonterminals first:

$$\begin{aligned} \textit{number} &\Rightarrow \textit{number} \, \textit{digit} \\ &\Rightarrow \textit{number} \, 4 \\ &\Rightarrow \textit{number} \, \textit{digit} \, 4 \\ &\Rightarrow \textit{number} \, 34 \\ &\dots \end{aligned}$$

but we would still have had the same parse tree (and syntax tree). However, different derivations can also lead to different parse trees. For example, if we construct $3 + 4 * 5$ from the expression grammar of Figure 6.4, we can use the derivation:

$$\begin{aligned}
 \text{expr} &\Rightarrow \text{expr} + \text{expr} \\
 &\Rightarrow \text{expr} + \text{expr} * \text{expr} \\
 &\quad (\text{replace the second } \text{expr} \text{ with } \text{expr} * \text{expr}) \\
 &\Rightarrow \text{number} + \text{expr} * \text{expr} \\
 &\dots
 \end{aligned}$$

or the derivation:

$$\begin{aligned}
 \text{expr} &\Rightarrow \text{expr} * \text{expr} \\
 &\Rightarrow \text{expr} + \text{expr} * \text{expr} \\
 &\quad (\text{replace the first } \text{expr} \text{ with } \text{expr} + \text{expr}) \\
 &\Rightarrow \text{number} + \text{expr} * \text{expr} \\
 &\dots
 \end{aligned}$$

These, in turn, lead to the two distinct parse trees shown in Figure 6.12. (Dashed lines in the remaining trees in this chapter indicate missing nodes in the parse trees that are irrelevant to the current discussion.)

This derivation can also lead to the two distinct abstract syntax trees shown in Figure 6.13.

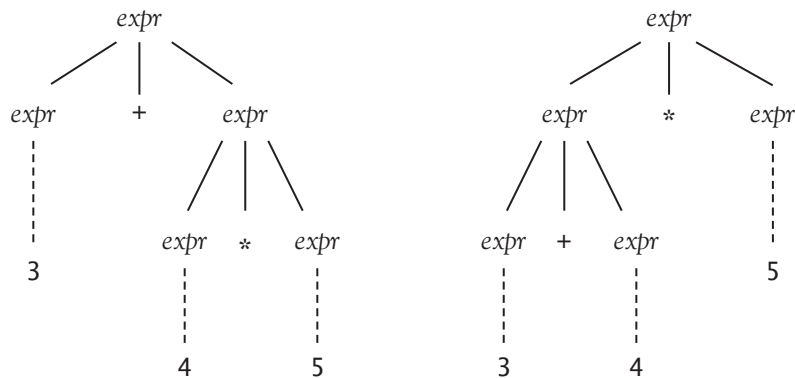


Figure 6.12: Two parse trees for $3 + 4 * 5$

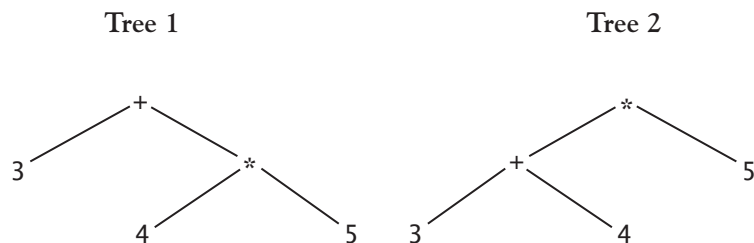


Figure 6.13 Two abstract syntax trees for $3 + 4 * 5$, indicating the ambiguity of the grammar of Figure 6.4

A grammar such as that in Figure 6.4, for which two distinct parse (or syntax) trees are possible for the same string, is **ambiguous**. Ambiguity can also be expressed directly in terms of derivations. Even though many derivations may in general correspond to the same parse tree, certain special derivations that are constructed in a special order can only correspond to unique parse trees. One kind of

derivation that has this property is a **leftmost derivation**, where the leftmost remaining nonterminal is singled out for replacement at each step. For example, the derivations in Figures 6.3 and 6.5 are leftmost, but the derivation at the beginning of this section (deriving the same string as Figure 6.5) is not. Each parse tree has a unique leftmost derivation, which can be constructed by a preorder traversal of the tree. Thus, the ambiguity of a grammar can also be tested by searching for two different leftmost derivations of the same string. If such leftmost derivations exist, then the grammar must be ambiguous, since each such derivation must correspond to a unique parse tree. For example, the ambiguity of the grammar of Figure 6.4 is also demonstrated by the two leftmost derivations for the string $3 + 4 * 5$ as given in Figure 6.14.

Leftmost Derivation 1 (Corresponding to Tree 1 of Figure 6.13)	Leftmost Derivation 2 (Corresponding to Tree 2 of Figure 6.13)
$expr \Rightarrow expr + expr$	$expr \Rightarrow expr * expr$
$\Rightarrow number + expr$	$\Rightarrow expr + expr * expr$
$\Rightarrow digit + expr$	$\Rightarrow number + expr * expr$
$\Rightarrow 3 + expr$	$\Rightarrow \dots$ (etc.)
$\Rightarrow 3 + expr * expr$	
$\Rightarrow 3 + number * expr$	
$\Rightarrow \dots$ (etc.)	

Figure 6.14 Two leftmost derivations for $3 + 4 * 5$, indicating the ambiguity of the grammar of Figure 6.4

Ambiguous grammars present difficulties, since no clear structure is expressed. To be useful, either the grammar must be revised to remove the ambiguity or a **disambiguating rule** must be stated to establish which structure is meant.

Which of the two parse trees (or leftmost derivations) is the correct one for the expression $3 + 4 * 5$? If we think of the semantics to be attached to the expression, we can understand what this decision means. The first syntax tree of Figure 6.13 implies that the multiplication operator $*$ is to be applied to the 4 and 5 (resulting in the value 20), and this result is added to 3 to get 23. The second syntax tree, on the other hand, says to add 3 and 4 first (getting 7) and then multiply by 5 to get 35. Thus, the operations are applied in a different order, and the resulting semantics are quite different.

If we take the usual meaning of the expression $3 + 4 * 5$ from mathematics, we would choose the first tree of Figure 6.13 over the second, since multiplication has precedence over addition. This is the usual choice in programming languages, although some languages (such as APL) make a different choice. How could we express the fact that multiplication should have precedence over addition? We could state a disambiguating rule separately from the grammar, or we could revise the grammar. The usual way to revise the grammar is to write a new grammar rule (called a “term”) that establishes a “precedence cascade” to force the matching of the “ $*$ ” at a lower point in the parse tree:

$$\begin{aligned} expr &\rightarrow expr + expr \mid term \\ term &\rightarrow term * term \mid (expr) \mid number \end{aligned}$$

We have not completely solved the ambiguity problem, however, because the rule for an *expr* still allows us to parse $3 + 4 + 5$ as either $(3 + 4) + 5$ or $3 + (4 + 5)$. In other words, we can make addition either **right-** or **left-associative**, as shown in Figure 6.15.

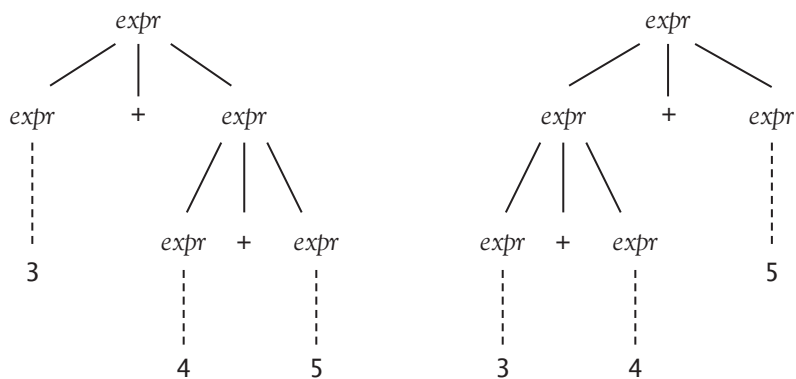


Figure 6.15: Addition as either right- or left-associative

In the case of addition, this does not affect the result, but if we were to include subtraction, it surely would: 8 - 4 - 2 = 2 if “-” is left-associative, but 8 - 4 - 2 = 6 if “-” is right-associative. What is needed is to replace the rule:

$$expr \rightarrow expr + expr$$

with either:

$$expr \rightarrow expr + term$$

or:

$$expr \rightarrow term + expr$$

The first rule is **left-recursive**, while the second rule is **right-recursive**. A left-recursive rule for an operation causes it to left-associate, as in the parse tree shown in Figure 6.16. Similarly, a right-recursive rule causes it to right-associate, as also shown in Figure 6.16.

A left-recursive rule for an operation causes it to left-associate.	A right-recursive rule for an operation causes it to right-associate.

Figure 6.16: Parse trees showing results of left- and right-recursive rules

The revised grammar for simple integer arithmetic expressions that expresses both precedence and associativity is given in Figure 6.17.

```

expr → expr + term | term
term → term * factor | factor
factor → ( expr ) | number
number → number digit | digit
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

Figure 6.17 Revised grammar for simple integer arithmetic expressions

The grammar of Figure 6.17 is now unambiguous (a proof of this requires more advanced techniques from parsing theory). Moreover, the syntax trees generated by this grammar correspond to the semantics of the arithmetic operations as they are usually defined. Sometimes the process of rewriting a grammar to eliminate ambiguity causes the grammar to become extremely complex, and in such cases we prefer to state a disambiguating rule (see the if-statement discussion in Chapter 9).

6.5 EBNFs and Syntax Diagrams

We noted earlier that the grammar rule:

$$number \rightarrow number\ digit \mid digit$$

generates a number as a sequence of digits:

$$\begin{aligned}
 number &\Rightarrow number\ digit \\
 &\Rightarrow number\ digit\ digit \\
 &\Rightarrow number\ digit\ digit\ digit \\
 &\Rightarrow digit \dots digit \\
 &\dots \\
 &\Rightarrow (\text{arbitrary repetitions of digit})
 \end{aligned}$$

Similarly, the rule:

$$expr \rightarrow expr + term \mid term$$

generates an expression as a sequence of terms separated by “+’s”:

$$\begin{aligned}
 expr &\Rightarrow expr + term \\
 &\Rightarrow expr + term + term \\
 &\Rightarrow expr + term + term + term \\
 &\dots \\
 &\Rightarrow term + \dots + term
 \end{aligned}$$

This situation occurs so frequently that a special notation for such grammar rules is adopted that expresses more clearly the repetitive nature of their structures:

$$\text{number} \rightarrow \text{digit} \{ \text{digit} \}$$

and:

$$\text{expr} \rightarrow \text{term} \{ + \text{term} \}$$

In this notation, the curly brackets $\{ \}$ stand for “zero or more repetitions of.” Thus, the rules express that a number is a sequence of one or more digits, and an expression is a term followed by zero or more repetitions of a $+$ and another term. In this notation, the curly brackets have become new meta-symbols, and this notation is called **extended Backus-Naur form**, or **EBNF** for short.

This new notation obscures the left associativity of the $+$ operator that is expressed by the left recursion in the original rule in BNF. We can get around this by simply assuming that any operator involved in a curly bracket repetition is left-associative. Indeed, if an operator were right-associative, the corresponding grammar rule would be right-recursive. Right-recursive rules are usually not written using curly brackets (the reason for this is explained more fully in the next section). The problem of expressing right associativity points out a flaw in EBNF grammars: Parse trees and syntax trees cannot be written directly from the grammar. Therefore, we will always use BNF notation to write parse trees.

A second common situation is for a structure to have an optional part, such as the optional else-part of an if-statement in C, which is expressed in BNF notation for C as follows:

$$\begin{aligned} \text{if-statement} \rightarrow & \text{if} (\text{expression}) \text{statement} \mid \\ & \text{if} (\text{expression}) \text{statement} \text{else statement} \end{aligned}$$

This is written more simply and expressively in EBNF, as follows:

$$\text{if-statement} \rightarrow \text{if} (\text{expression}) \text{statement} [\text{else statement}]$$

where the square brackets “[]” are new metasymbols indicating optional parts of the structure.

Another example is the grammar rule for *function-definition* in the BNF grammar for C; see Figure 6.6:

$$\begin{aligned} \text{function-definition} \rightarrow & \text{declaration-specifiers declarator} \\ & \text{declaration-list compound-statement} \\ & \mid \text{declaration-specifiers declarator compound-statement} \\ & \mid \text{declarator declaration-list compound-statement} \\ & \mid \text{declarator compound-statement} \end{aligned}$$

Using EBNF, this can be much more simply expressed as:

$$\begin{aligned} \text{function-definition} \rightarrow & [\text{declaration-specifiers}] \text{declarator} \\ & [\text{declaration-list}] \text{compound-statement} \end{aligned}$$

Right-associative (binary) operators can also be written using these new metasymbols. For example, if “@” is a right-associative operator with BNF:

$$\text{expr} \rightarrow \text{term} @ \text{expr} \mid \text{term}$$

then this rule can be rewritten in EBNF as follows:

$$\text{expr} \rightarrow \text{term} [@ \text{expr}]$$

For completeness, we write out the grammar of Figure 6.3 for simple integer arithmetic expressions in EBNF in Figure 6.18.

```
expr → term { + term }
term → factor { * factor }
factor → ( expr ) | number
number → digit { digit }
digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Figure 6.18 EBNF rules for simple integer arithmetic expressions

A sometimes-useful graphical representation for a grammar rule is the **syntax diagram**, which indicates the sequence of terminals and nonterminals encountered in the right-hand side of the rule. For example, syntax diagrams for *noun-phrase* and *article* of our simple English grammar presented in Section 6.2 would be drawn as shown in Figure 6.19. Figure 6.20 shows these two rules condensed into one.

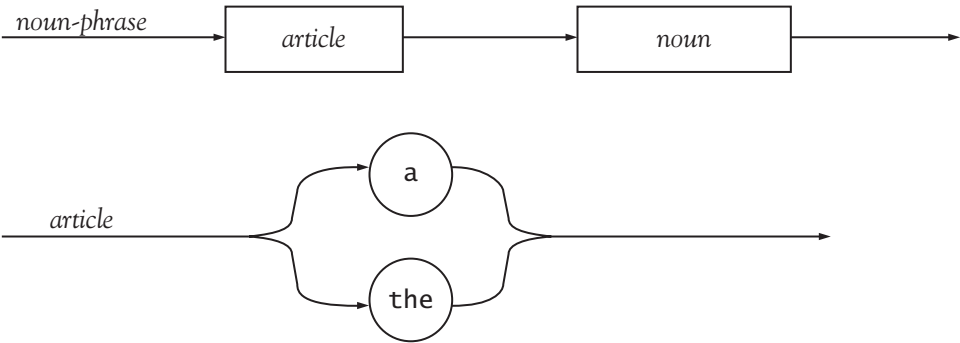


Figure 6.19: Syntax diagrams for *noun-phrase* and *article* of the simple English grammar presented in Section 6.2

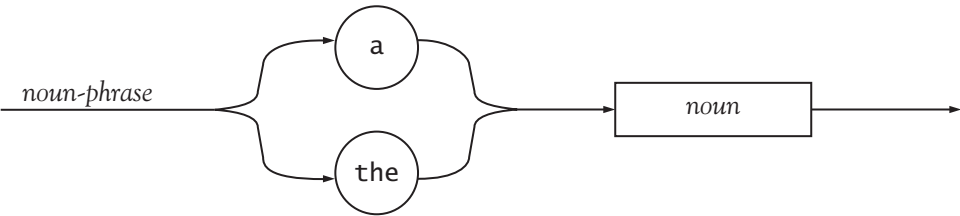


Figure 6.20: Condensed version of diagrams shown in Figure 6.19

Syntax diagrams use circles or ovals for terminals and squares or rectangles for nonterminals, connecting them with lines and arrows to indicate appropriate sequencing. Syntax diagrams can also condense several grammar rules into one diagram.

Syntax diagrams for the expression grammar in Figure 6.18 are given in Figure 6.21. Note the use of loops in the diagrams to express the repetition given by the curly brackets in the EBNFs.

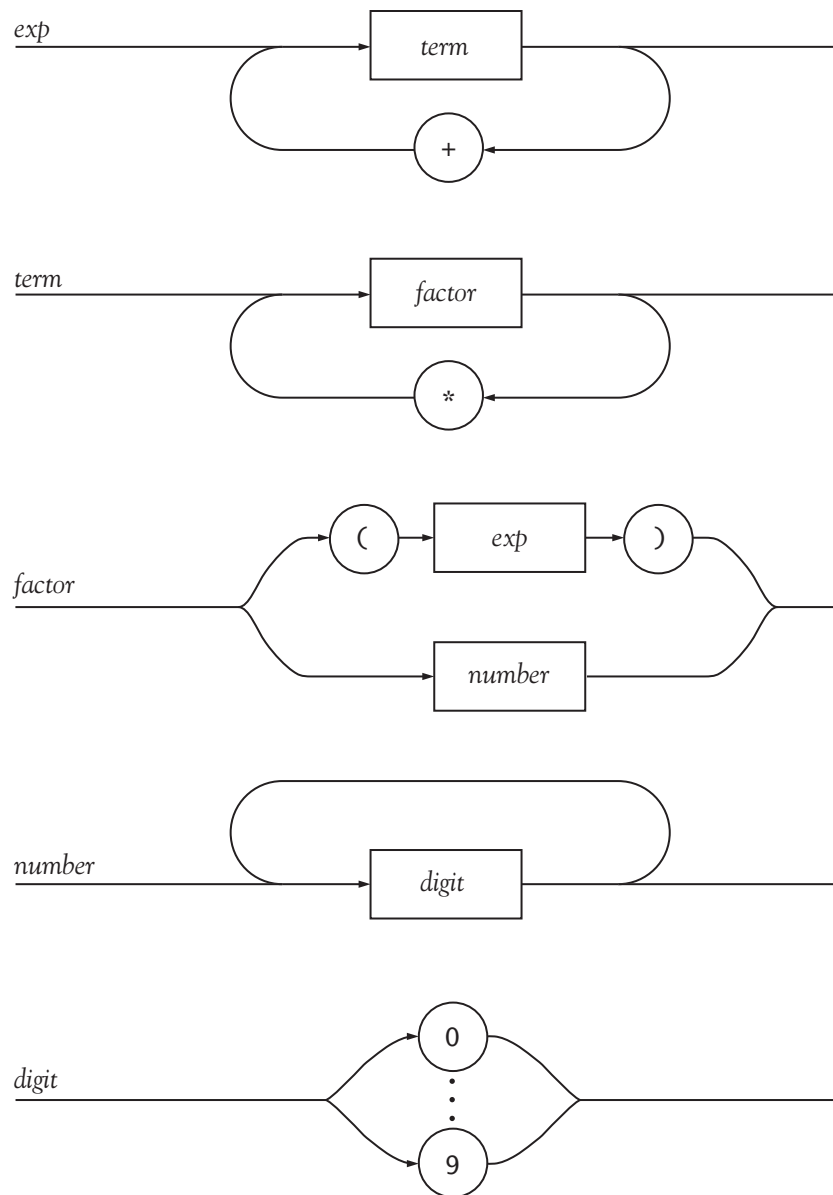


Figure 6.21: Syntax diagrams for a simple integer expression grammar

As an example of how to express the square brackets [] in syntax diagrams, Figure 6.22 shows the syntax diagram for the *if-statement* in C.

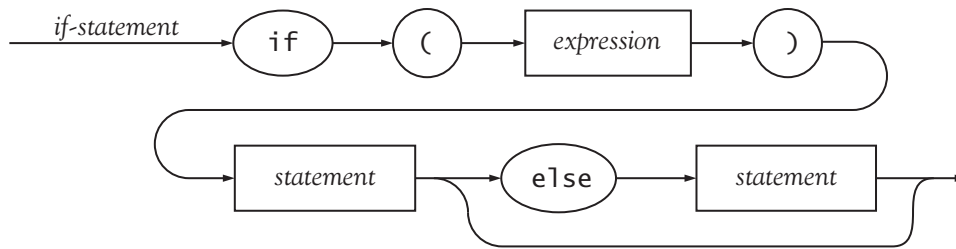


Figure 6.22 Syntax diagram for the *if-statement* in C

Syntax diagrams are always written from the EBNF notation, not the BNF notation, for reasons that are made clear in the next section. Thus, the diagram for *expr* shown in Figure 6.23 would be incorrect.

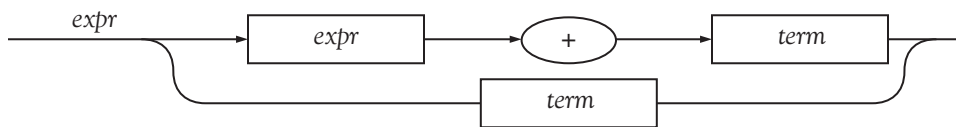


Figure 6.23: Incorrect syntax diagram for *expr*

Syntax diagrams are visually appealing but take up a great deal of space. As more programmers have become familiar with rules written directly in BNF or EBNF, syntax diagrams have been used less frequently, and now are seen much more rarely than in the past. They remain useful, however, as flow charts for writing parsing procedures, a topic to which we now turn.

6.6 Parsing Techniques and Tools

A grammar written in BNF, EBNF, or as syntax diagrams describes the strings of tokens that are syntactically legal in a programming language. The grammar, thus, also implicitly describes the actions that a parser must take to parse a string of tokens correctly and to construct, either implicitly or explicitly, a derivation or parse tree for the string. The simplest form of a parser is a **recognizer**—a program that accepts or rejects strings, based on whether they are legal strings in the language. More general parsers build parse trees (or, more likely, abstract syntax trees) and carry out other operations, such as calculating values for expressions.

Given a grammar in one of the forms we have discussed, how does it correspond to the actions of a parser? One method of parsing attempts to match an input with the right-hand sides of the grammar rules. When a match occurs, the right-hand side is replaced by, or **reduced** to, the nonterminal on the left. Such parsers are **bottom-up** parsers, since they construct derivations and parse trees from the leaves to the root. They are sometimes also called **shift-reduce** parsers, since they shift tokens onto a stack prior to reducing strings to nonterminals. In the other major parsing method, **top-down parsing**, nonterminals are expanded to match incoming tokens and directly construct a derivation. Both top-down and bottom-up parsing can be automated; that is, a program, called a **parser generator**, can be written that will automatically translate a BNF description into a parser. Since bottom-up parsing is somewhat more powerful than top-down parsing, it is usually the preferred method for parser generators (also called **compiler compilers**). One such parser generator in wide use is YACC (short for “yet another compiler compiler”), or its freeware version Bison, which we will study later in this section.

However, is an older, very effective method for constructing a parser from a grammar that is still in use. This method, which is called **recursive-descent parsing**, turns the nonterminals into a group of mutually recursive procedures whose actions are based on the right-hand sides of the BNFs.

The right-hand sides are interpreted in the procedures as follows: tokens are matched directly with input tokens as constructed by a scanner. Nonterminals are interpreted as calls to the procedures corresponding to the nonterminals.

As an example, in our simplified English grammar, procedures for *sentence*, *noun-phrase*, and *article* would be written as follows (in C-like pseudocode):

```
void sentence(){
    nounPhrase();
    verbPhrase();
}

void nounPhrase(){
    article();
    noun();
}

void article(){
    if (token == "a") match("a", "a expected");
    else if (token == "the") match("the", "the expected");
    else error("article expected");
}
```

In this pseudocode, we use strings to imitate tokens, and a global variable called *token* to hold the current token as constructed by a scanner. The *getToken* procedure of the scanner is called whenever a new token is desired. A match of a token corresponds to a successful test for that token, followed by a call to *getToken*:

```
void match(TokenType expected, char* message){
    if (token == expected) getToken();
    else error(message);
}
```

Each parsing procedure starts with the variable *token* already holding the first token that should be expected. Thus, a call to *getToken* must precede the first call to the *sentence* procedure. Likewise, each parsing procedure leaves behind the next token in the input stream when it is finished. The *error* procedure prints an error message and aborts the parse. (Note that errors need only be detected when actual tokens are expected, as in the procedure *article*. We could have checked for “a” or “the” in the *sentence* procedure, but in this scheme it is unnecessary.)

If we apply this process to the BNF description of Figure 6.17, we encounter a problem with the left-recursive rules such as the one for an expression:

$$expr \rightarrow expr + term \mid term$$

If we were to try to write this rule as a recursive-descent procedure, two fatal difficulties arise. First, the first choice in this rule ($expr \rightarrow expr + term$) would have the procedure immediately calling itself recursively, which would result in an infinite recursive loop. Second, there is no way to decide which of the two choices to take ($expr \rightarrow expr + term$ or $expr \rightarrow term$) until a + is seen (or not) much later in the input. The problem is caused by the existence of the left recursion in the first alternative of the grammar rule, since the right-recursive rule:

$$expr \rightarrow term + expr \mid term$$

has only a very modest problem and can be turned into a parsing procedure by simply calling `term()` and then testing for a "+":

```
void expr(){
    term();
    if (token == "+"){
        match("+", "+ expected");
        expr();
    }
}
```

Unfortunately, this grammar rule (and the associated parsing procedure) make + into a right-associative operator. How can we remove the problem caused by the left recursion, while at the same time retaining the left associative behavior of the + operation? The solution is implicit in the EBNF description of this same grammar rule, which expresses the recursion as a loop:

$$expr \rightarrow term \{ + term \}$$

This form of the grammar rule for *expr* corresponds to the recursive-descent code:

```
void expr(){
    term();
    while (token == "+"){
        match("+", "+ expected");
        term();
        /* perform left associative operations here */
    }
}
```

While this form of the grammar does suppress the explicit left associativity of +, the corresponding code *does* provide the opportunity for enforcing left associativity by timing whatever operations are to be performed (tree building or expression calculation) right after the second call to `term()` (at the end of the body of the loop, as indicated by the comment).

Thus, the curly brackets in EBNF represent **left recursion removal** by the use of a loop. While there are more general mechanisms than this for left recursion removal, this simple use of EBNF usually suffices in practice.

Right-recursive rules on the other hand, as we have already noted, present no such problem in recursive-descent parsing. However, the code that we write for a right-recursive rule such as:

$$expr \rightarrow term @ expr \mid term$$

(we change the name of the operation to @ to avoid confusion with +) also corresponds to the use of square brackets in EBNF to express this optional structure:

$$expr \rightarrow term [@ expr]$$

This process is called **left factoring**, since it is as if the *term* in both alternatives of the grammar rule is “factored out,” leaving either @ *expr* or nothing (thus making @ *expr* optional).

Another typical left-factoring situation arises when we have an if-statement with an optional else-part:

$$\begin{aligned} if\text{-statement} \rightarrow & \text{if (expression) statement} \mid \\ & \text{if (expression) statement else statement} \end{aligned}$$

This cannot be translated directly into code, since both alternatives begin with the same prefix. As in the previous example, in EBNF this is written with square brackets, and the common parts of the alternatives are “factored out”:

$$if\text{-statement} \rightarrow \text{if (expression) statement} [\text{else statement}]$$

This corresponds directly to the recursive-descent code to parse an *if-statement*:

```
void ifStatement(){
    match("if", "if expected");
    match("(", "( expected");
    expression();
    match(")", ") expected");
    statement();
    if (token == "else"){
        match("else", "else expected");
        statement();
    }
}
```

Thus, in both left-recursive and left-factoring situations, EBNF rules or syntax diagrams correspond naturally to the code of a recursive-descent parser. This actually is one of the main reasons for their use.

As a demonstration of the power and ease of recursive-descent parsing, Figure 6.24 provides the complete code in C for an integer calculator based on the grammar of Figure 6.18. This program accepts a single line of input containing a simple integer arithmetic expression consisting of numbers, the operators + and *, and parentheses, and prints the computed value of the expression, or an error message if the expression is not syntactically correct. As in the grammar, the calculator assumes all tokens are characters, and it does not permit spaces to appear in an expression. To make it easy to print the computed value, and also to make sure only a single expression is entered in the input line,

a new grammar rule is added to the grammar of Figure 6.18 (and the start symbol is changed to *command*):

$$command \rightarrow expr '\n'$$

In effect, this grammar rule makes the newline character ('`\n`') into the marker for the end of the input (just like the `$` token discussed earlier in this chapter).

Finally, we note that the timing of the computation of the results in each procedure ensures that the operators `+` and `*` are left-associative.

```
(1) #include <ctype.h>
(2) #include <stdlib.h>
(3) #include <stdio.h>

(4) int token; /* holds the current input character for the parse */

(5) /* declarations to allow arbitrary recursion */
(6) void command();
(7) int expr();
(8) int term();
(9) int factor();
(10) int number();
(11) int digit();

(12) void error(char* message){
(13)     printf("parse error: %s\n", message);
(14)     exit(1);
(15) }

(16) void getToken(){
(17)     /* tokens are characters */
(18)     token = getchar();
(19) }
(20) void match(char c, char* message){
(21)     if (token == c) getToken();
(22)     else error(message);
(23) }

(24) void command(){
(25)     /* command -> expr '\n' */
(26)     int result = expr();
(27)     if (token == '\n') /* end the parse and print the result */
(28)         printf("The result is: %d\n", result);
(29)     else error("tokens after end of expression");
(30) }
```

Figure 6.24 A calculator for simple integer arithmetic expressions using recursive-descent parsing (*continues*)

(continued)

```
(31) int expr(){
(32)     /* expr -> term { '+' term } */
(33)     int result = term();
(34)     while (token == '+'){
(35)         match('+', "+ expected");
(36)         result += term();
(37)     }
(38)     return result;
(39) }

(40) int term(){
(41)     /* term -> factor { '*' factor } */
(42)     int result = factor();
(43)     while (token == '*'){
(44)         match('*', "* expected");
(45)         result *= factor();
(46)     }
(47)     return result;
(48) }

(49) int factor(){
(50)     /* factor -> '(' expr ')' | number */
(51)     int result;
(52)     if (token == '('){
(53)         match('(', "( expected");
(54)         result = expr();
(55)         match(')', ") expected");
(56)     }
(57)     else
(58)         result = number();
(59)     return result;
(60) }

(61) int number(){
(62)     /* number -> digit { digit } */
(63)     int result = digit();
(64)     while (isdigit(token))
(65)         /* the value of a number with a new trailing digit
(66)          is its previous value shifted by a decimal place
(67)          plus the value of the new digit
(68)          */
(69)         result = 10 * result + digit();
(70)     return result;
(71) }
```

Figure 6.24 A calculator for simple integer arithmetic expressions using recursive-descent parsing (*continues*)

(continued)

```

(72) int digit(){
(73)     /* digit -> '0' | '1' | '2' | '3' | '4'
(74)         | '5' | '6' | '7' | '8' | '9' */
(75)     int result;
(76)     if (isdigit(token)){
(77)         /* the numeric value of a digit character
(78)         is the difference between its ascii value and the
(79)         ascii value of the character '0'
(80)         */
(81)         result = token - '0';
(82)         match(token, "( expected)");
(83)     }
(84)     else
(85)         error("digit expected");
(86)     return result;
(87) }

(88) void parse(){
(89)     getToken(); /* get the first token */
(90)     command(); /* call the parsing procedure for the start symbol */
(91) }

(92) int main(){
(93)     parse();
(94)     return 0;
(95) }

```

Figure 6.24 A calculator for simple integer arithmetic expressions using recursive-descent parsing

In this method for converting a grammar into a parser, the resulting parser bases its actions only on the next available token in the input stream (stored in the token variable in the code). This use of a single token to direct a parse is called **single-symbol lookahead**. A parser that commits itself to a particular action based only on this lookahead is called a **predictive parser** (sometimes laboriously referred to as a top-down parser with single-symbol lookahead and no backtracking).

Predictive parsers require that the grammar to be parsed satisfies certain conditions so that this decision-making process will work. The first condition is the ability to choose among several alternatives in a grammar rule. Suppose that a nonterminal A has the following choices:

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$$

(where each α_i stands for a string of tokens and nonterminals). To decide which choice to use, the tokens that begin each of the α_i must be different. Given a string α of tokens and nonterminals, we define **First**(α) to be the set of tokens that can begin the string α . For example, given the grammar rules:

$$\begin{aligned}
 factor &\rightarrow (\ expr \) \mid number \\
 number &\rightarrow digit \{ \ digit \} \\
 digit &\rightarrow 0 \mid 1 \mid \dots \mid 9
 \end{aligned}$$

we have:

$$\begin{aligned}\text{First}((\text{expr})) &= \{ (\} \\ \text{First}(\text{number}) &= \text{First}(\text{digit}) \\ &= \text{First}(0) \cup \text{First}(1) \cup \dots \cup \text{First}(9) \\ &= \{ 0, \dots, 9 \}\end{aligned}$$

and then:

$$\begin{aligned}\text{First}(\text{factor}) &= \text{First}((\text{expr})) \cup \text{First}(\text{number}) \\ &= \{ (, 0, 1, \dots, 9 \}\end{aligned}$$

The requirement that a predictive parser be able to distinguish between choices in a grammar rule can be stated in terms of First sets, as follows. Given the grammar rule:

$$A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$$

the First sets of any two choices must not have any tokens in common; that is,

$$\begin{aligned}\text{First}(\alpha_i) \cap \text{First}(\alpha_j) &= \emptyset \text{ for all } i \neq j \\ (\emptyset \text{ denotes the empty set})\end{aligned}$$

In the example of the grammar rule for a factor:

$$\text{factor} \rightarrow (\text{expr}) \mid \text{number}$$

this condition for predictive parsing is satisfied, since:

$$\text{First}((\text{expr})) \cap \text{First}(\text{number}) = \{ (\} \cap \{ 0, \dots, 9 \} = \emptyset$$

A second condition for predictive parsing arises when structures are optional. For example, to parse the grammar rule

$$\text{expr} \rightarrow \text{term} [@ \text{expr}]$$

we must test for the presence of the token “@” before we can be sure that the optional part is actually present:

```
void expr(){
    term();
    if (token == '@'){
        match('@', "@ expected");
        expr();
    }
}
```

However, if the token @ can also come after an expression, this test is insufficient. If @ is the next token in the input, it may be the start of the optional “@ *expr*” part, or it may be a token that comes after

the whole expression. Thus, the second requirement for predictive parsing is that, for any optional part, no token beginning the optional part can also come after the optional part.

We can formalize this condition in the following way. For any string A of tokens and nonterminals that appears on the right-hand side of a grammar rule, define **Follow**(a) to be the set of tokens that can follow a . The second condition for predictive parsing can now be stated in terms of Follow sets. Given a grammar rule in EBNF with an optional a ,

$$A \rightarrow \beta [\alpha] \sigma$$

we must have:

$$\text{First}(\alpha) \cap \text{Follow}(\alpha) = \emptyset$$

As an example of the computation of Follow sets, consider the grammar of Figure 6.18. Follow sets for the nonterminals can be computed as follows. From the rule:

$$\text{factor} \rightarrow (\text{expr})$$

we get that the token “)” is in $\text{Follow}(\text{expr})$. Since expr can also comprise a whole expression, expr may be followed by the end of the input, which we indicate by the special symbol $\$$. Thus, $\text{Follow}(\text{expr}) \ni \{ \text{) }, \$ \}$. From the rule:

$$\text{expr} \rightarrow \text{term} \{ + \text{term} \}$$

we obtain that “+” is in $\text{Follow}(\text{term})$. Since a term also appears as the last thing in an expr , anything that follows an expr can also follow a term . Thus,

$$\text{Follow}(\text{term}) = \{ \text{) }, \$, + \}$$

Finally, $\text{Follow}(\text{factor}) = \{ \text{) }, \$, +, * \}$ by a similar computation.

Note that the grammar of Figure 6.18 automatically satisfies the second condition for predictive parsing, since there are no optional structures inside square brackets. A more instructive example comes from Figure 6.6 (a small part of a BNF grammar for C), where the grammar rule for a *declaration* in EBNF is written as:

$$\text{declaration} \rightarrow \text{declaration-specifiers} [\text{init-declarator-list}] \text{ ‘;’}$$

By tracing through the rules for *init-declarator-list* in Figure 6.6, we can determine that $\text{First}(\text{init-declarator-list}) = \{ \text{ID}, *, (\}$ and $\text{Follow}(\text{init-declarator-list}) \ni \{ \text{ ‘;’}, \text{ ‘,’} \}$, so that:

$$\text{First}(\text{init-declarator-list}) \cap \text{Follow}(\text{init-declarator-list}) = \emptyset$$

thus satisfying the second condition for predictive parsing in this case also.

The computation of First and Follow sets can also be useful in recursive-descent parsing to obtain appropriate tests; see Exercise 6.49.

We mentioned at the beginning of this section that the process of converting grammar rules into a parser can be performed by a program known as a parser generator, or compiler-compiler. A parser generator takes a version of BNF or EBNF rules as its input and then generates an output that is a parser program in some language. This program can then be compiled to provide an executable parser. Of course, providing only a grammar to the parser generator will result in a recognizer.

To get the parser to construct a syntax tree or perform other operations, we must provide operations or actions to be performed that are associated with each grammar rule, that is, a syntax-directed scheme.

We mentioned earlier that YACC, which is available on most UNIX systems, is a widely used parser generator. Its freeware version, Bison, is available for many different systems (including all versions of UNIX). YACC was written by Steve Johnson in the mid-1970s; it generates a C program that uses a bottom-up algorithm to parse the grammar. The format of a YACC specification is as follows:

```
%{ /* code to insert at the beginning of the parser */
%}
/* other YACC definitions, if necessary */
%%
/* grammar and associated actions */
%%
/* auxiliary procedures */
```

The grammar is given in a BNF-like form, with associated actions written in C. Figure 6.25 shows a complete YACC/Bison description of a simple integer expression interpreter using the grammar of Figure 6.17 (Section 6.4). Note that it uses the original left-recursive BNF grammar—bottom-up parsers are not bothered by left recursion. The actions provided with the rules are essentially the same as in the recursive-descent calculator of Figure 6.24 (and the C program generated by YACC behaves exactly as that program from the user's perspective). The YACC/Bison convention for constructing values from a parse is that the value of the left-hand side of a production is indicated by the symbol "\$\$", while the value of the n th symbol of the right-hand side is given by \$ n . (These values are always integers unless specified otherwise.) For example, on line 7 of Figure 6.25, \$1 + \$3 represents the sum of the first and third symbols of the right-hand side of the rule:

```
expr : expr '+' term
```

so that the value of `expr` and `term` on the right are added. Note the different conventions for metasympols, nonterminals, and tokens.

```
(1) %{
(2) #include <stdio.h>
(3) %}

(4) %%
(5) command : expr '\n' { printf("The result is: %d\n", $1); }
(6)          ;
```

Figure 6.25 YACC/Bison specification of a calculator for simple integer arithmetic expressions (*continues*)

(continued)

```

(7)  expr      : expr '+' term { $$ = $1 + $3; }
(8)           u term { $$ = $1; }
(9)           ;
(10) term      : term '*' factor { $$ = $1 * $3; }
(11)          u factor { $$ = $1; }
(12)          ;

(13) factor    : number { $$ = $1; }
(14)          u '(' expr ')' { $$ = $2; }
(15)          ;

(16) number    : number digit { $$ = 10 * $1 + $2; }
(17)          u digit { $$ = $1; }
(18)          ;

(19) digit     : '0' { $$ = 0; }
(20)          u '1' { $$ = 1; }
(21)          u '2' { $$ = 2; }
(22)          u '3' { $$ = 3; }
(23)          u '4' { $$ = 4; }
(24)          u '5' { $$ = 5; }
(25)          u '6' { $$ = 6; }
(26)          u '7' { $$ = 7; }
(27)          u '8' { $$ = 8; }
(28)          u '9' { $$ = 9; }
(29)          ;

(30) %%
(31) main()
(32) { yyparse();
(33)   return 0;
(34) }

(35) int yylex(void)
(36) { static int done = 0; /* flag to end parse */
(37)   int c;
(38)   if (done) return 0; /* stop parse */
(39)   c = getchar();
(40)   if (c == '\n') done = 1; /* next call will end parse */
(41)   return c;
(42) }

(43) int yyerror(char *s)
(44) /* allows for printing error message */
(45) {   printf("%s\n",s);
(46) }

```

Figure 6.25 YACC/Bison specification of a calculator for simple integer arithmetic expressions

YACC/Bison generates a procedure `yyparse` from the grammar, so we have to provide a `main` procedure that calls `yyparse` (lines 31–34). YACC/Bison also assumes that tokens (aside from individual characters) are recognized by a scanner procedure called `yylex` similar to the `getToken` procedure of Figure 6.24. Thus, we provide a `yylex` in C that reads and returns a character (lines 35–42). One difference, however, between `yylex` and `getToken` is that YACC/Bison will only end a parse when `yylex` returns 0 (in the recursive-descent version, we could write special code that ended the parse on a newline character). Thus, `yylex` includes code to remember when a newline is reached and to return 0 on the next call after a newline (allowing the generated parser to match the newline directly before finishing; see Exercise 6.28). It would also be possible to generate `yylex` automatically from a **scanner generator** such as LEX/Flex, but we will not study this here. (For more on this, see the Notes and References at the end of this chapter.) Finally, an error procedure `yyerror` is needed to print error messages.

YACC/Bison is useful not only to the translator writer, but also to the language designer. Given a grammar, it provides a description of possible problems and ambiguities. However, to read this information we must have a more specific knowledge of bottom-up parsing techniques. The interested reader may consult the Notes and References at the end of this chapter.

6.7 Lexics vs. Syntax vs. Semantics

A context-free grammar typically includes a description of the tokens of a language by including the strings of characters that form the tokens in the grammar rules. For example, in the partial English grammar introduced in Section 6.2, the tokens are the English words “a,” “the,” “girl,” “dog,” “sees,” and “pets,” plus the “.” character. In the simple integer expression grammar of Section 6.3, the tokens are the arithmetic symbols “+” and “*,” the parentheses “(” and “),” and the digits 0 through 9. Specific details of formatting, such as the white-space conventions mentioned in Section 6.1, are left to the scanner and need to be stated as lexical conventions separate from the grammar.

Some typical token categories, such as literals or constants and identifiers, are not fixed sequences of characters in themselves, but are built up out of a fixed set of characters, such as the digits 0..9. These token categories can have their structure defined by the grammar, as for example the grammar rules for *number* and *digit* in the expression grammar. However, it is possible and even desirable to use a scanner to recognize these structures, since the full recursive power of a parser is not necessary, and the scanner can recognize them by a simple repetitive operation. Using a scanner makes the recognition of numbers and identifiers faster and simpler, and it reduces the size of the parser and the number of grammar rules.

To express the fact that a number in the expression grammar should be a token rather than represented by a nonterminal, we can rewrite the grammar of Figure 6.18 as in Figure 6.26.

```

expr → term { + term }
term → factor { * factor }
factor → ( expr ) | NUMBER

```

Figure 6.26 Numbers as tokens in simple integer arithmetic

By making the string `NUMBER` uppercase in the new grammar, we are saying that it is not a token to be recognized literally, but one whose structure is determined by the scanner. The structure of such tokens must then be specified as part of the lexical conventions of the language. As noted in Section 6.1, the structure of such tokens can be specified using regular expressions. However, many language designers choose to include the structure of such tokens as part of the grammar, with the understanding that an implementor may include these in the scanner instead of the parser. Thus, a description of a language using BNF, EBNF, or syntax diagrams may include not only the syntax but most of the lexical structure (or **lexics**) of a programming language as well. The boundary, therefore, between syntactic and lexical structure is not always clearly drawn, but depends on the point of view of the designer and implementor.

The same is true for syntax and semantics. We have been taking the approach that syntax is anything that can be defined with a context-free grammar and semantics is anything that cannot. However, many authors include properties that we would call semantic as syntactic properties of a language. Examples include such rules as declaration before use for variables and no redeclaration of identifiers within a procedure. These are rules that are context sensitive and cannot be written as context-free rules. Hence we prefer to think of them as semantic rather than syntactic rules.

Another conflict between syntax and semantics arises when languages require certain strings to be **predefined identifiers** rather than **reserved words**. Recall from Section 6.1 that reserved words are fixed strings of characters that are tokens themselves and that cannot be used as identifiers. By contrast, a predefined identifier is a fixed string that is given a predefined meaning in a language, but this meaning can be changed by redefining it within a program. As we have noted, the strings `if`, `while`, and `do` are reserved words in C, Java, and Ada. However, in Ada, all of the basic data type names, such as `integer` and `boolean`, are predefined identifiers that can be redefined. For example, the following declaration in Ada is perfectly legal:

```
integer: boolean;
```

After this declaration, the name `integer` has become a variable of type `boolean`. (You're not alone if you find this confusing.)

C, C++, and Java, on the other hand, make the basic types such as `int`, `double`, and `bool` (in C++) or `boolean` (in Java) reserved words that cannot be redefined. This is probably a better approach, since these names have a fixed semantics in every program that we would hardly want to change even if we could.

Finally, syntax and semantics can become interdependent in languages when semantic information must be used to distinguish ambiguous parsing situations. A simple situation where this occurs is in C, where type names and variable names must be distinguishable during parsing. Consider, for example, the C expression:

```
(x) - y
```

If `x` is a type name, this is a cast of the value `-y` to type `x`. However, if `x` is a variable name, then this is subtraction of `y` from `x`. Thus, a parser must have context information on identifiers available to disambiguate such situations. One might view this as a design flaw in the grammar of C. (Java has the same problem, but not Ada.) However, it is a relatively easy problem to overcome in practice, as you will see in later chapters.

6.8 Case Study: Building a Syntax Analyzer for TinyAda

We conclude this chapter by developing a parser for a small language that is similar in some ways to the Ada programming language. This little language, called TinyAda, is large enough to illustrate the syntactic features of many high-level languages, yet small enough to require a medium-sized programming project. In this section, we inaugurate the project by discussing a simple syntax analyzer. The next few chapters revisit the project to add code to analyze some semantics as well.

An EBNF grammar for TinyAda is shown in Figure 6.27. Note the semantic qualifiers, such as <type>, enclosed in angle brackets. You can ignore these for now, but we will return to them in the next few chapters.

In the following grammar

```

=      means "is defined as"
" "    enclose literal items
[ ]    enclose items which may be omitted
{ }    enclose items which may appear zero or more times
|      indicates a choice
( )    are used for grouping required choices
< >   enclose semantic qualifications

subprogramBody =
    subprogramSpecification "is"
    declarativePart
    "begin" sequenceOfStatements
    "end" [ <procedure>identifier ] ";"

declarativePart = { basicDeclaration }

basicDeclaration = objectDeclaration | numberDeclaration
                  | typeDeclaration | subprogramBody

objectDeclaration =
    identifierList ":" typeDefinition ";"

numberDeclaration =
    identifierList ":" "constant" ":@" <static>expression ";"

identifierList = identifier { "," identifier }

typeDeclaration = "type" identifier "is" typeDefinition ";"

typeDefinition = enumerationTypeDefinition | arrayTypeDefinition
                | range | <type>name

range = "range " simpleExpression ".." simpleExpression

```

Figure 6.27 An EBNF grammar for TinyAda (*continues*)

(continued)

```

index = range | <type>name

enumerationTypeDefinition = "(" identifierList ")"

arrayTypeDefinition = "array" "(" index { ",", index } ")" "of" <type>name

subprogramSpecification = "procedure" identifier [ formalPart ]

formalPart = "(" parameterSpecification { ";" parameterSpecification } ")"

parameterSpecification = identifierList ":" mode <type>name

mode = [ "in" ] | "in" "out" | "out"

sequenceOfStatements = statement { statement }

statement = simpleStatement | compoundStatement

simpleStatement = nullStatement | assignmentStatement
                | procedureCallStatement | exitStatement

compoundStatement = ifStatement | loopStatement

nullStatement = "null" ";"

assignmentStatement = <variable>name "!=" expression ";"

ifStatement =
    "if" condition "then" sequenceOfStatements
    { "elsif" condition "then" sequenceOfStatements }
    [ "else" sequenceOfStatements ]
    "end" "if" ";"

loopStatement =
    [ iterationScheme ] "loop" sequenceOfStatements "end" "loop" ";"

iterationScheme = "while" condition

exitStatement = "exit" [ "when" condition ] ";"

procedureCallStatement = <procedure>name [ actualParameterPart ] ";"

actualParameterPart = "(" expression { ",", expression } ")"

```

Figure 6.27 An EBNF grammar for TinyAda (*continues*)

(continued)

```

condition = <boolean>expression

expression = relation { "and" relation } | { "or" relation }

relation = simpleExpression [ relationalOperator simpleExpression ]

simpleExpression =
    [ unaryAddingOperator ] term { binaryAddingOperator term }

term = factor { multiplyingOperator factor }

factor = primary [ "*" primary ] | "not" primary

primary = numericLiteral | stringLiteral | name | "(" expression ")"

name = identifier [ indexedComponent ]

indexedComponent = "(" expression { "," expression } ")"

relationalOperator = "=" | "/=" | "<" | "<=" | ">" | ">="

binaryAddingOperator = "+" | "-"

unaryAddingOperator = "+" | "-"

multiplyingOperator = "*" | "/" | "mod"

```

Figure 6.27 An EBNF grammar for TinyAda

As you can see from the grammar, TinyAda includes several kinds of declarations (constant, variable, type, and procedure), statements (assignment, selection, loop, and procedure call), and expressions. The top-level phrase in this little language, called `subprogramBody`, is a procedure declaration, which might serve the same role for a TinyAda program as the `main` function of the C language does in the execution of a program. Note also that the rules for declarations, statements, and expressions are indirectly recursive, allowing for nested declarations, statements, and expressions.

Figure 6.28 shows a short TinyAda program, which includes two procedure declarations and several constant declarations, type declarations, variable declarations, statements, and expressions.

```

procedure TEST is

    COLUMN_MAX : constant := 10;
    ROW_MAX : constant := COLUMN_MAX;
    type COLUMN_INDEX is range 1..COLUMN_MAX;
    type ROW_INDEX is range 1..ROW_MAX;
    type MATRIX is array(COLUMN_INDEX, ROW_INDEX) of INTEGER;

```

Figure 6.28 A TinyAda program (*continues*)

(continued)

```

A : MATRIX;
I : INTEGER;

procedure INIT_MATRIX(X : in INTEGER; Y : out MATRIX) is

    I, J : INTEGER;

begin
    I := 1;
    while I <= COLUMN_MAX loop
        J := 1;
        while J <= ROW_MAX loop
            Y(I, J) := X;
            J := J + 1;
        end loop;
        I := I + 1;
    end loop;
end INIT_MATRIX;

begin
    I := 1;
    INIT_MATRIX(I, A);
end TEST;

```

Figure 6.28 A TinyAda program

Although TinyAda is not case sensitive, we use lowercase for reserved words and uppercase for identifiers in program examples.

The initial version of our parser is known as a **parsing shell**. The purpose of this program is to apply the grammar rules to a source program and report any syntax or lexical errors. The parser is a mere shell, because it simply checks whether the tokens in each phrase are of the correct types, without performing any additional analysis. In later chapters, we show how to fill in this shell with additional mechanisms for semantic analysis.

The complete program consists of a scanner and a parser, but it is structured in terms of four cooperating components, each of which is implemented as a Java class. Here are their names, roles, and responsibilities:

1. The `Token` class represents tokens in the language. For simple syntax analysis, a token object need only include a code for the token's type, such as was used in earlier examples in this chapter. However, a token can include other attributes, such as an identifier name, a constant value, and a data type, as we will see in later chapters.
2. The `Char io` class (short for character I/O) converts the source program's text into a stream of characters for the scanner, thus enabling the scanner to focus on lexical analysis rather than low-level text processing. The `Char io` class also handles the output of any error messages.

3. The `Scanner` class recognizes and generates tokens in a stream of characters and returns these tokens to the parser. The `Scanner` class also detects any lexical errors.
4. The `Parser` class uses a recursive descent strategy to recognize phrases in a stream of tokens. Unlike the scanner, which continues to generate tokens after lexical errors, the parser halts execution upon encountering the first syntax error in a source program.

The flow of data among these classes is shown in Figure 6.29.

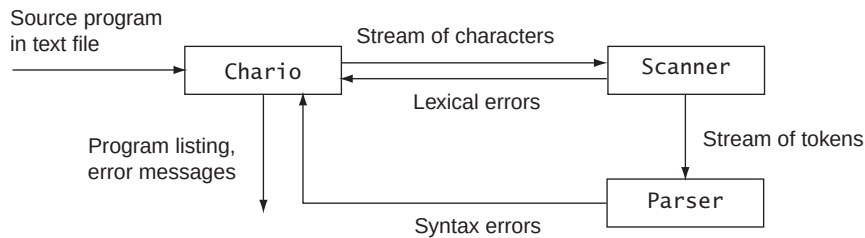


Figure 6.29 Data flow in the TinyAda syntax analyzer

Complete source code for the `Token`, `Chario`, and `Scanner` classes and a partially completed `Parser` class can be found on the book's Web site. Also available are two user interfaces for the program, one that is terminal-based and one that is GUI-based. If you want to execute a completed version of the parser, executable jar files for both versions are available as well. In this section, we focus on the development of several parsing methods, and leave the rest as exercises for the reader.

The `Parser` object receives tokens from the `Scanner` object and sends error messages to the `Chario` object. At program startup, the parser's constructor receives these two objects as parameters and saves references to them in instance variables. The parser also includes instance variables for the current token in the input stream and for several sets of tokens. These sets include some of TinyAda's operator symbols and the tokens that begin various declarations and statements in the language (described earlier in this chapter as First Sets). Figure 6.30 shows the Java code for the parser's instance variable declarations and their initialization.

```

public class Parser extends Object{

    private Chario chario;
    private Scanner scanner;
    private Token token;

    private Set<Integer> addingOperator,
                        multiplyingOperator,
                        relationalOperator,
                        basicDeclarationHandles,
                        statementHandles;

```

Figure 6.30 The constructor and data for the `Parser` class (*continues*)

(continued)

```

public Parser(Chario c, Scanner s){
    chario = c;
    scanner = s;
    initHandles();
    token = scanner.nextToken();
}

private void initHandles(){
    addingOperator = new HashSet<Integer>();
    addingOperator.add(Token.PLUS);
    addingOperator.add(Token.MINUS);
    multiplyingOperator = new HashSet<Integer>();
    multiplyingOperator.add(Token.MUL);
    multiplyingOperator.add(Token.DIV);
    multiplyingOperator.add(Token.MOD);
    relationalOperator = new HashSet<Integer>();
    relationalOperator.add(Token.EQ);
    relationalOperator.add(Token.NE);
    relationalOperator.add(Token.LE);
    relationalOperator.add(Token.GE);
    relationalOperator.add(Token.LT);
    relationalOperator.add(Token.GT);
    basicDeclarationHandles = new HashSet<Integer>();
    basicDeclarationHandles.add(Token.TYPE);
    basicDeclarationHandles.add(Token.ID);
    basicDeclarationHandles.add(Token.PROC);
    statementHandles = new HashSet<Integer>();
    statementHandles.add(Token.EXIT);
    statementHandles.add(Token.ID);
    statementHandles.add(Token.IF);
    statementHandles.add(Token.LOOP);
    statementHandles.add(Token.NULL);
    statementHandles.add(Token.WHILE);
}

// Other parsing methods go here
}

```

Figure 6.30 The constructor and data for the Parser class

After the main application program instantiates the parser, it runs the method `parse` on this object to commence the syntax analysis. This method will either return normally if the source program has no syntax errors, or throw an exception when the first syntax error is encountered. The utility methods `accept` and `fatalError` collaborate to check for an expected token and throw an exception if an error occurs. Here is the code for these three Parser methods:


```

public void parse(){
    subprogramBody();
    accept(Token.EOF, "extra symbols after logical end of program");
}

private void accept(int expected, String errorMessage){
    if (token.code != expected)
        fatalError(errorMessage);
    token = scanner.nextToken();
}

private void fatalError(String errorMessage){
    chario.putError(errorMessage);
    throw new RuntimeException("Fatal error");
}

```

Note that the `accept` method calls the `Scanner` method `nextToken` to advance the scan to the next token if the current token's code is the expected one. The `fatalError` method calls the `Chario` method `putError` to print an error message before throwing the exception. Finally note that the `parse` method calls the `subprogramBody` method, which corresponds to the top-level rule in the TinyAda grammar. After processing a complete source program, this method calls the `accept` method to ensure that the end of the source file has been reached.

All that remains to implement the parser is to translate the grammar rules into parsing methods. We mentioned earlier that some coders find it easier first to translate the EBNF rules to syntax diagrams, which then can serve as flow charts for the parsing methods. This option is left as an exercise for the reader. Before we explore the implementation of several methods, we need to make several more points about their design:

1. With few exceptions, each parsing method is responsible for analyzing a phrase in the source program that corresponds to exactly one rule in the grammar. Occasionally, when the first few tokens in a phrase appear in two rules (see `numberDeclaration` and `objectDeclaration` in the grammar), it will be convenient to merge the processing for both rules into a single method.
2. Each parsing method assumes that the parser's `token` variable refers to the first token to be processed in that parsing method's grammar rule. This is why, before parsing starts, the `Scanner` method `nextToken` must be run once, to obtain the first token for the parsing method `subprogramBody`.
3. Each parsing method leaves behind the next token following the phrase just processed. The `accept` method is designed to accomplish that most of the time.
4. Each parsing method expects no parameters and returns no value. With the exception of the current token, the parsing methods do not need to share any information, because the rules for simple syntax analysis are context-free. In later chapters, we will enable the parsing methods to communicate in order to share the context-sensitive information needed for semantic analysis.

To give the flavor of the coding process, let's develop several methods from the rules. The first one, `subprogramBody`, processes the phrase defined by the top-level rule. The EBNF rule appears in a comment above the method header.

```
/*
subprogramBody =
    subprogramSpecification "is"
    declarativePart
    "begin" sequenceOfStatements
    "end" [ <procedure>identifier ] ";"
*/
private void subprogramBody(){
    subprogramSpecification();
    accept(Token.IS, "'is' expected");
    declarativePart();
    accept(Token.BEGIN, "'begin' expected");
    sequenceOfStatements();
    accept(Token.END, "'end' expected");
    if (token.code == Token.ID)
        token = scanner.nextToken();
    accept(Token.SEMI, "semicolon expected");
}
```

As you can see, this method usually calls the `accept` method to process a terminal symbol (or token) and calls another parsing method to process a nonterminal symbol (or phrase). There is one `if` statement, which processes the optional identifier enclosed in `[ ]` symbols in the EBNF.

Our next two methods collaborate to process basic declarations.

```
/*
declarativePart = { basicDeclaration }
*/
private void declarativePart(){
    while (basicDeclarationHandles.contains(token))
        basicDeclaration();
}

/*
basicDeclaration = objectDeclaration | numberDeclaration
                  | typeDeclaration | subprogramBody
*/
private void basicDeclaration(){
    switch (token.code){
        case Token.ID:
            numberOrObjectDeclaration();
            break;
```

(continues)

(continued)

```

        case Token.TYPE:
            typeDeclaration();
            break;
        case Token.PROC:
            subprogramBody();
            break;
        default: fatalError("error in declaration part");
    }
}

```

A declarative part consists of zero or more basic declarations. The `declarativePart` method must, therefore, call the `basicDeclaration` method within a loop. This loop runs as long as the current token is a member of the set of basic declaration handles, which are the four different types of tokens that can begin a basic declaration. The `basicDeclaration` method then decides which type of basic declaration to process, based on the handle or current token. The actual processing of each declaration is, naturally, a matter of passing the buck on to another parsing method.

The last parsing method that we show here belongs to the processing of expressions. A simple expression consists of an optional unary adding operator followed by a term. This term is then followed by zero or more pairs of binary adding operators and terms.

```

/*
simpleExpression =
    [ unaryAddingOperator ] term { binaryAddingOperator term }
*/
private void simpleExpression(){
    if (addingOperator.contains(token))
        token = scanner.nextToken();
    term();
    while (addingOperator.contains(token)){
        token = scanner.nextToken();
        term();
    }
}

```

Once again, an option in the grammar translates to an `if` statement in the method. In this case, there are actually two options (the two unary adding operators), but the code is simplified by testing the current token for membership in a set of tokens. The same technique is used to control the entry into the `while` loop to process the zero or more items following the first term. Note also that after each optional token is found, it is consumed from the input stream by calling the `Scanner` method `nextToken`. This will guarantee that the parsing method `term` sees the appropriate token when it begins its processing.

The development of the remaining parsing methods is left as an exercise for the reader. Although there is a large number of these methods, their structure is simple and fairly regular. You can write them in a top-down fashion, and use stubs for many of the methods if you want to test your code before

completing them all. If you do that, be sure to use example TinyAda programs that exercise only the methods you have completed. You should then test any completed methods with TinyAda programs that exercise their error-detection capabilities.

Exercises

- 6.1 (a) The C programming language distinguishes character constants from string constants by using single quotation marks for characters and double quotation marks for strings. Thus, 'c' is the character c, while "c" is a string of length 1 consisting of the single character c. Why do you think this distinction is made? Is it useful?
 (b) Python, on the other hand, uses double quotation marks for both characters and strings (thus, "c" is either a character or string, depending on context). Discuss the advantages and disadvantages of these two approaches.
- 6.2 Devise a test in one or more of the following languages to determine whether comments are considered white space: (a) C, (b) Java, (c) Ada, and (d) Python. What is the result of performing your test?
- 6.3 Discuss the pros and cons of ignoring or requiring “white space” (i.e., blanks, end-of-lines, and tabs) when recognizing tokens.
- 6.4 Many programming languages (like C) do not allow nested comments. Why is this requirement made? Modula-2 and a few other languages do allow nested comments. Why is this useful?
- 6.5 (a) Describe the strings that are represented by the regular expression:

$$[0-9]^+((E|e)(\+|\-)?[0-9]^+)?$$

 (b) Write a regular expression for C identifiers consisting of letters, digits, and the underscore character '_', and starting with a letter or underscore.
- 6.6 (a) Numeric constants may be signed or unsigned in a programming language (e.g., 123 is an unsigned integer constant, but -123 is a signed integer constant). However, there is a problem associated with signed constants. Describe it. (*Hint:* Consider expressions involving subtraction.)
 (b) Can you think of a solution to the problem of part (a)?
- 6.7 Rewrite the scanner program of Figure 6.1 by packaging the code for scanning an integer literal as a function.
- 6.8 In the simplified English grammar of Figure 6.2, there are only finitely many legal sentences. How many are there? Why?
- 6.9 (a) Suppose that white space was completely ignored (as in FORTRAN) in the grammar of Figure 6.2, so that sentences could be written as, for example, "thegirlseesadog." Can this grammar still be parsed? Explain.
 (b) Answer the previous question again, assuming that the nouns theorist and orator were added to the grammar.
- 6.10 Translate the BNF rules of Figure 6.6 into EBNF.
- 6.11 Add subtraction and division to the (a) BNF, (b) EBNF, and (c) syntax diagrams of simple integer arithmetic expressions (Figures 6.9, 6.10, and 6.11). Be sure to give them the appropriate precedences.

- 6.12 Add the integer remainder and power operations to (a) the arithmetic BNF or (b) EBNF of Figures 6.9 or 6.10. Use % for the remainder operation and ^ for the power operation. Recall that the remainder operation is left-associative and has the same precedence as multiplication, but that power is right-associative (and has greater precedence than multiplication, so $2 \wedge 2 \wedge 3 = 256$, not 64).
- 6.13 Unary minuses can be added in several ways to the arithmetic expression grammar of Figure 6.17 or Figure 6.18. Revise the BNF and EBNF for each of the cases that follow so that it satisfies the stated rule:
- (a) At most, one unary minus is allowed in each expression, and it must come at the beginning of an expression, so $-2 + 3$ is legal (and equals 1) and $-2 + (-3)$ is legal, but $-2 + -3$ is not.
 - (b) At most, one unary minus is allowed before a number or left parenthesis, so $-2 + -3$ and $-2 * -3$ are legal, but $--2$ and $-2 + --3$ are not.
 - (c) Arbitrarily many unary minuses are allowed before numbers and left parentheses, so everything above is legal.
- 6.14 Using the grammar of Figure 6.17, draw parse trees and abstract syntax trees for the arithmetic expressions:
- (a) $((2))$
 - (b) $3 + 4 * 5 + 6 * 7$
 - (c) $3 * 4 + 5 * 6 + 7$
 - (d) $3 * (4 + 5) * (6 + 7)$
 - (e) $(2 + (3 + (4 + 5)))$
- 6.15 Finish writing the pseudocode for a recursive-descent recognizer for the English grammar of Section 6.2 that was begun in Section 6.6.
- 6.16 Translate the pseudocode of the previous exercise into a working recognizer program in C or another programming language of your choice. (*Hint:* This will require a scanner to separate the text into token strings.)
- 6.17 Revise the program of the previous exercise so that it randomly *generates* legal sentences in the grammar. (This will require the use of a random number generator.)
- 6.18 Modify the recursive-descent calculator program of Figure 6.24 to use the grammar of Figure 6.26 and the scanner of Figure 6.1 (so that the calculator also skips blanks appropriately).
- 6.19 Add subtraction and division to either (a) the calculator program of Figure 6.24, or (b) your answer to the previous exercise.
- 6.20 Add the remainder and power operations as described in Exercise 6.12 to (a) the program of Figure 6.24, (b) your answer to Exercise 6.18, or (c) your answer to Exercise 6.19.
- 6.21 Rewrite the program of Figure 6.24 or your answer to any of the previous three exercises (except for the remainder operation) to produce floating-point answers, as well as accept floating-point constants using the following revised grammar rules (in EBNF):

... (rules for *expr* and *term* as before)
factor $\rightarrow$ (*expr*) | *decimal-number*
decimal-number $\rightarrow$ *number* [. *number*]
 ... (rules for *number* and *digit* as before)

- 6.22 To eliminate the left recursion in the simple integer arithmetic grammar, one might be tempted to write:

$$\text{expr} \rightarrow \text{term} + \text{term}$$

Why is this wrong?

- 6.23 Modify the YACC/Bison program of Figure 6.25 to use the grammar of Figure 6.26 and the scanner of Figure 6.1 (so that the calculator also skips blanks appropriately). This will require some additions to the YACC definition and changes to the scanner, as follows. First, YACC already has a global variable similar to the `numval` variable of Figure 6.1, with the name `yyval`, so `numval` must be replaced by this variable. Second, YACC must define the tokens itself, rather than use an enum such as in Figure 6.1; this is done by placing a definition as follows in the YACC definition:

```
%{ /* code to insert at the beginning of the parser */
%}
#token NUMBER PLUS TIMES ...
%%
etc....
```

- 6.24 Add subtraction and division to either **(a)** the YACC/Bison program of Figure 6.25, or **(b)** your answer to the previous exercise.
- 6.25 Add the remainder and power operations as described in Exercise 6.12 to **(a)** the YACC/Bison program of Figure 6.25, **(b)** your answer to Exercise 6.23, or **(c)** your answer to Exercise 6.24.
- 6.26 Repeat Exercise 6.21 for the YACC/Bison program of Figure 6.25. This will require that the type of the parsing result be redefined to `double`, and this requires the insertion of a new definition into the YACC definition as follows:

```
%{
#define YYSTYPE double
...
%}
...
%%
etc....
```

- 6.27 **(a)** Explain why the code for a command in Figure 6.24 (lines 23–29) does not call `match(' \n')`. **(b)** Explain what the behavior of the code of Figure 6.24 *would* be if command *did* call `match(' \n')`.
- 6.28 The YACC/Bison code of Figure 6.25 could be simplified by using a newline to end the parse, as in the code of Figure 6.24. We would simply change the grammar rule for `command` (Figure 6.25, line 5) to remove the newline:

```
command : expr { printf("The result is: %d\n", $1); }
```

and change the `yylex` procedure (Figure 6.25, lines 35–42) to:

```
int yylex(void){
    int c;
    c = getchar();
    if (c == '\n') return 0; /* newline will end parse */
    return c;
}
```

Unfortunately, this changes the behavior of the YACC/Bison parser so that it is no longer the same as that of the recursive-descent version.

- (a) Describe the different behavior. (*Hint*: Consider a missing operator, as in “3 4”.)
 - (b) Explain why you think this different behavior occurs.
- 6.29 Capitalization of articles at the beginning of a sentence was viewed as a context sensitivity in the text. However, in the simple English grammar of Figure 6.2, capitalization can be achieved by adding only context-free rules. How would you do this?
- 6.30 It was stated in this chapter that it is not possible to include in the BNF description the rule that there should be no redeclaration of variables. Strictly speaking, this is not true if only finitely many variable names are allowed. Describe how one could include the requirement that there be no redeclaration of variables in the grammar for a language, if only finitely many identifiers are allowed. Why isn't it a good idea?
- 6.31 The text notes that it is more efficient to let a scanner recognize a structure such as an unsigned integer constant, which is just a repetition of digits. However, an expression is also just a repetition of terms and pluses:

$$expr \rightarrow term \{ + term \}$$

Why can't a scanner recognize all expressions?

- 6.32 Write a BNF description for a statement-sequence as a sequence of statements separated by semicolons. (Assume that statements are defined elsewhere.) Then, translate your BNF into EBNF and/or syntax diagrams.
- 6.33 Write a BNF description for a statement-sequence as a sequence of statements in which each statement is terminated by a semicolon. (Assume that statements are defined elsewhere.) Then, translate your BNF into EBNF and/or syntax diagrams.
- 6.34 C uses the semicolon as a statement terminator (as in the previous exercise) but also allows a statement to be empty (that is, consisting of only a semicolon), so that the following is a legal C program:

```
main(){
    ; ; ; ; ; ; ; ;
    return 0;
}
```

Discuss the advantages and disadvantages of this.

- 6.35 (a) List the predefined identifiers in Python and describe their meanings.
 (b) Does C have predefined identifiers? Explain.
 (c) Does Ada have predefined identifiers? Explain.
- 6.36 (a) One measure of the complexity of a language is the number of reserved words in its syntax. Compare the number of reserved words in C, C++, Ada, and Java.
 (b) One could argue that the preceding comparison is misleading because of the use of predefined identifiers in Pascal and standard (predefined) libraries in C++, Java, and Ada. Discuss the pros and cons of adding the number of predefined identifiers or the number of library identifiers into the comparison.
- 6.37 Here is a legal Pascal program:

```
program yecch;
  var true, false: boolean;
  begin
    true := 1 = 0;
    false := true;
    ...
    ...
  end.
```

What values do `true` and `false` have in this program? What design principle does this violate? Is this program possible in Ada? In C? In C++? In Java?

- 6.38 Is it possible to have a language without any reserved words? Discuss.
- 6.39 Some languages use format to distinguish the beginning and ending of structures, as follows:

```
if (x == 0)
  (* all indented statements here are part of
  the if *)
else
  (* all indented statements here are part of
  the else *)
(* statements that are not indented are outside the else *)
```

Discuss the advantages and disadvantages of this for (a) writing programs in the language, and (b) writing a translator. (This rule, sometimes called the **Offside rule**, is used in Haskell; see Landin [1966].)

- 6.40 A number is defined in the grammar of Figure 6.17 using a left-recursive rule. However, it could also be defined using a right-recursive rule:

$$\textit{number} \rightarrow \textit{digit number} \mid \textit{digit}$$

Which is better, or does it matter? Why?

- 6.41 In Exercise 6.21, a decimal constant was defined using the rule:

$$\textit{decimal-number} \rightarrow \textit{number} [. \textit{number}]$$

This causes a problem when computing the value of the fractional part of a decimal number (to the right of the decimal point), because of the left-recursive rule for a *number*.

- (a) Describe the problem. Is this problem as significant when a scanner recognizes a decimal number as it is when a parser does the recognition?
 - (b) A solution to the problem of part (a) is to write the fractional number as a right-recursive rule, as in the previous exercise. Write out BNF rules expressing this solution, and explain why it is better.
 - (c) Implement your solution of part (b) in the code of Exercise 6.21 or Exercise 6.26.
- 6.42 Given the following BNF:

$$\begin{aligned} \text{expr} &\rightarrow (\text{list}) \mid a \\ \text{list} &\rightarrow \text{list} , \text{expr} \mid \text{expr} \end{aligned}$$

- (a) Write EBNF rules and/or syntax diagrams for the language.
 - (b) Draw the parse tree for $((a, a), a, (a))$.
 - (c) Write a recursive-descent recognizer for the language.
- 6.43 One way of defining the abstract syntax for the expressions of Figure 6.17 is as follows:

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{expr} \mid \text{expr} * \text{expr} \mid \text{number} \\ \text{number} &\rightarrow \text{digit} \{ \text{digit} \} \\ \text{digit} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

- (a) Why are there no precedences or associativities expressed in this grammar?
 - (b) Why are there no parentheses in this grammar?
 - (c) The rule for *number* is written in EBNF in this grammar, yet we stated in the text that this obscures the structure of the parse tree. Why is the use of EBNF legitimate here? Why did we not use EBNF in the rule for *expr*?
- 6.44 Write data type definitions in C, C++, or Java for an *expr* abstract syntax tree, as given in the text and the previous exercise.
- 6.45 Some languages, like ML and Haskell, allow the abstract syntax tree to be defined in a way that is virtually identical to syntax rules. Write data type definitions in ML or Haskell that follow the abstract syntax for *expr* described in Exercise 6.43.
- 6.46 Compute First and Follow sets for the nonterminals of the grammar of Exercise 6.42.
- 6.47 Show that any left-recursive grammar rule does not satisfy the first condition for predictive parsing.
- 6.48 Show that the following grammar does not satisfy the second rule of predictive parsing:

$$\begin{aligned} \text{stmt} &\rightarrow \text{if-stmt} \mid \text{other} \\ \text{if-stmt} &\rightarrow \text{if } \text{stmt} [\text{else } \text{stmt}] \end{aligned}$$

- 6.49 Given the following grammar in EBNF:

$$\begin{aligned} \text{expr} &\rightarrow (\text{list}) \mid a \\ \text{list} &\rightarrow \text{expr} [\text{list}] \end{aligned}$$

- (a) Show that the two conditions for predictive parsing are satisfied.
- (b) Write a recursive-descent recognizer for the language.

- 6.50 In Section 6.7, the definition of Follow sets was somewhat nonstandard. More commonly, Follow sets are defined for nonterminals only rather than for strings, and optional structures are accommodated by allowing a nonterminal to become empty. Thus, the grammar rule:

$$\begin{aligned} \text{if-statement} &\rightarrow \text{if (expression) statement} \\ &\quad | \text{if (expression) statement else statement} \end{aligned}$$

is replaced by the two rules:

$$\begin{aligned} \text{if-statement} &\rightarrow \text{if (expression) statement else-part} \\ \text{else-part} &\rightarrow \text{else statement} \mid \varepsilon \end{aligned}$$

where the symbol ε (Greek epsilon) is a new metasymbol standing for the empty string. A nonterminal A can then be recognized as optional if it either becomes ε directly, or there is a derivation beginning with A that derives ε . In this case, we add ε to $\text{First}(A)$. Rewrite the second condition for predictive parsing using this convention, and apply this technique to the grammars of Exercises 6.42 and 6.49.

- 6.51 The second condition for predictive parsing, involving Follow sets, is actually much stronger than it needs to be. In fact, for every optional construct β :

$$A \rightarrow \alpha [\beta] \gamma$$

a parser only needs to be able to tell when β is present and when it isn't. State a condition that will do this without using Follow sets.

- 6.52 The second condition for predictive parsing is suspect (even more than the previous exercise indicates) because there is a natural disambiguating rule in those cases where it is not satisfied. What is it? Explain its effect when parsing the grammar of Exercise 6.48.
- 6.53 Given the following grammar in BNF:

$$\text{string} \rightarrow \text{string string} \mid \text{a}$$

this corresponds to the set equation:

$$S = SS \cup \{a\}$$

Show that the set $S_0 = \{a, aa, aaa, aaaa, \dots\}$ is the smallest set satisfying the equation. (*Hint:* First show that S_0 does satisfy the equation by showing set inclusion in both directions. Then, using induction on the length of strings in S_0 , show that, given any set S' that satisfies the equation, S_0 must be contained in S' .)

- 6.54 According to Wirth [1976], data structures based on syntax diagrams can be used by a “generic” recursive-descent parser that will parse any set of grammar rules that satisfy the two conditions for predictive parsing. A suitable data structure is given by the following C declarations:

```
typedef int TokenKind;
typedef struct RuleRec* RulePtr;
typedef struct RuleRec{
    RulePtr next, other;
```

(continues)

(continued)

```
int isToken;
union{
    TokenKind name;
    RulePtr rule;
} token_or_rule;
} RuleRec;
```

The next field is used to point to the next item in the grammar rule, and the other field is used to point to choices given by the | metasymbol. Thus, the data structure for the grammar rule:

$$factor \rightarrow (\textit{expr}) \mid \textit{number}$$

from Figure 6.18 would look like the one shown in Figure 6.31. The fields of this data structure are shown in Figure 6.32.

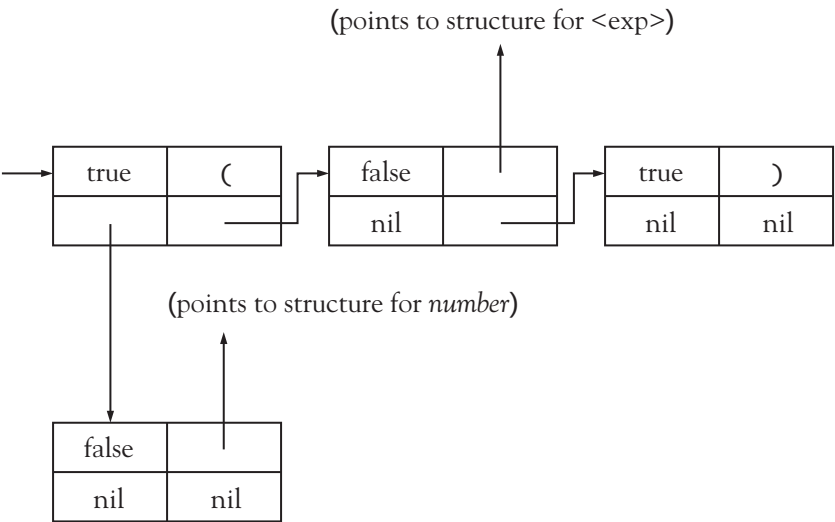


Figure 6.31: Data structure for the grammar rule $factor \rightarrow (\textit{expr}) \mid \textit{number}$

isToken	name/rule
other	next

Figure 6.32: Fields of the data structure shown in Figure 6.31

- (a) Draw the data structures for the rest of the grammar rules in Figure 6.18. (Hint: For repetitive and optional structures you will need a special token to represent the empty string.)

- (b) Write a generic parse procedure that uses these data structures to recognize an input string, assuming the existence of *getToken* and *error* procedures.
 - (c) Write a parser-generator that reads EBNF rules (either from a file or standard input) and generates the preceding data structures.
- 6.55 Draw syntax diagrams that express the rules in the EBNF grammar for TinyAda of Figure 6.15.
- 6.56 Complete the implementation of the TinyAda parser discussed in Section 6.8.

Notes and References

Many of the topics discussed in this chapter are treated in more detail in Aho, Sethi, and Ullman [1986] and Louden [2004]. An overview similar to this chapter, but with a little more depth, can be found in Louden [1997b]. EBNF and syntax diagrams are discussed in Wirth [1976] and in Horowitz [1984]. Context-free grammars and regular expressions are explained in Hopcroft and Ullman [1979] and Sipser [1997]. The original description of context-free grammars is in Chomsky [1956]. An early use of BNF in the description of Algo160 is in Naur [1963a]. A description of general algorithms to compute First and Follow sets, and to perform left-recursion removal, are found in Aho, Sethi, and Ullman [1986], Louden [1997], and Wirth [1976]. Our description of Follow sets is for EBNF grammars and is slightly nonstandard; see Exercise 6.50. YACC is described in more detail in Johnson [1975] and LEX in Lesk [1975]. A BNF description of C can be found in Kernighan and Ritchie [1988], for C++ in Stroustrup [1997], for Java in Gosling et al. [2000], for Pascal in Cooper [1983], and for Ada in Horowitz [1987]. All of these BNF descriptions can be found in many other places as well.

CHAPTER

Basic Semantics

7.1	Attributes, Binding, and Semantic Functions	257
7.2	Declarations, Blocks, and Scope	260
7.3	The Symbol Table	269
7.4	Name Resolution and Overloading	282
7.5	Allocation, Lifetimes, and the Environment	289
7.6	Variables and Constants	297
7.7	Aliases, Dangling References, and Garbage	303
7.8	Case Study: Initial Static Semantic Analysis of TinyAda	309

In Chapter 1, we made the distinction between the syntax of a programming language (what the language constructs look like) and its semantics (what the language constructs actually do). In Chapter 6, you saw how the syntactic structure of a language can be precisely specified using context-free grammar rules in Backus-Naur form (BNF). In this chapter, we will introduce the semantics of programming languages in more detail.

Specifying the semantics of a programming language is a more difficult task than specifying its syntax. This is not surprising, given that semantics has to do with meaning, which can be subtle and open to interpretation. There are several ways to specify semantics:

1. *Language reference manual.* This is the most common way to specify semantics. The expanding use of English descriptions has resulted in clearer and more precise reference manuals, but they still suffer from the lack of precision inherent in natural language descriptions and also may have omissions and ambiguities.
2. *Defining translator.* The advantage of specifying semantics by defining a translator is that questions about a language can be answered by experiment (as in chemistry or physics). A drawback is that questions about program behavior cannot be answered in advance—we must execute a program to discover what it does. Another drawback is that bugs and machine dependencies in the translator become parts of the language semantics, possibly unintentionally. Also, the translator may not be portable to all machines and may not be generally available.
3. *Formal definition.* Formal, mathematical methods are precise, but they are also complex and abstract, and require study to understand. Different formal methods are available, with the choice of method depending on its intended use. Perhaps the best formal method to use for the description of the translation and execution of programs is denotational semantics, which describes semantics using a series of functions.

In this chapter, we will specify semantics by using a hybrid of the type of informal description you might find in a manual combined with the simplified functions used in denotational descriptions. We will provide abstractions of the operations that occur during the translation and execution of programs in general, whatever the language, but we will concentrate on the details of Algol-like languages, such as C, C++, Java, Pascal, and Ada. We conclude this chapter with a case study on the semantic analysis of programs in the TinyAda language. More formal methods of describing semantics are studied in Chapter 12.

7.1 Attributes, Binding, and Semantic Functions

A fundamental abstraction mechanism in any programming language is the use of **names**, or **identifiers**, to denote language entities or constructs. In most languages, variables, procedures, and constants can have names assigned by the programmer. A fundamental step in describing the semantics of a language is to describe the conventions that determine the meaning of each name used in a program.

In addition to names, a description of the semantics of most programming languages includes the concepts of **location** and **value**. Values are any storable quantities, such as the integers, the reals, or even array values consisting of a sequence of the values stored at each index of the array. Locations are places values can be stored. Locations are like addresses in the memory of a specific computer, but we can think of them more abstractly than that. To simplify, we can think of locations as being numbered by integers starting at 0 and going up to some maximum location number.

The meaning of a name is determined by the properties, or **attributes**, associated with the name. For example, the C declaration¹

```
const int n = 5;
```

makes *n* into an integer constant with value 5. That is, it associates to the name *n* the data type attribute “integer constant” and the value attribute 5. (In C, the constant attribute—that *n* can never change its value—is part of the data type; in other languages this may be different.) The C declaration

```
int x;
```

associates the attribute “variable” and the data type “integer” to the name *x*. The C declaration

```
double f(int n){
    ...
}
```

associates the attribute “function” and the following additional attributes to the name *f*:

1. The number, names, and data types of its parameters (in this case, one parameter with name *n* and data type “integer”)
2. The data type of its returned value (in this case, “double”)
3. The body of code to be executed when *f* is called (in this case, we have not written this code but just indicated it with three dots)

Declarations are not the only language constructs that can associate attributes to names. For example, the assignment:

```
x = 2;
```

associates the new attribute “value 2” to the variable *x*. And, if *y* is a pointer variable² declared as:

```
int* y;
```

the C++ statement

```
y = new int;
```

¹ In C, C++, Pascal, and Ada, some declarations are called “definitions”; the distinctions will be explained later.

² Pointers are discussed more fully in Sections 8.5 and 8.7.

allocates memory for an integer variable (that is, associates a location attribute to it) and assigns this location to `*y`, that is, associates a new value attribute to `y`.

The process of associating an attribute with a name is called **binding**. In some languages, constructs that cause values to be bound to names (such as the C constant declaration earlier) are in fact called bindings rather than declarations. For example, in the ML code

```
let val x = 2; val y = 3 in x + y
```

this is a `let` expression which binds the value 2 to `x` and the value 3 to `y` (these are called `let`-bindings).

In purely functional languages such as Haskell, a value can be bound to a name only once. Thus, in these languages there is no need for a separate location binding, to store a value that can be reset later in the program by an assignment statement.

An attribute can be classified according to the time during the translation/execution process when it is computed and bound to a name. This is called the **binding time** of the attribute. Binding times can be classified into two general categories: **static binding** and **dynamic binding**. Static binding occurs prior to execution, while dynamic binding occurs during execution. An attribute that is bound statically is a static attribute, while an attribute that is bound dynamically is a dynamic attribute.

Languages differ substantially in which attributes are bound statically and which are bound dynamically. Often, functional languages have more dynamic binding than imperative languages. Binding times can also depend on the translator. Interpreters, for example, can translate and execute code simultaneously and so can establish most bindings dynamically, while compilers can perform many bindings statically. To make the discussion of attributes and binding independent of such translator issues, we usually refer to the binding time of an attribute as the earliest time that the language rules permit the attribute to be bound. Thus, an attribute that *could* be statically bound, but that is dynamically bound by a translator, is still referred to as a static attribute.

As examples of binding times, consider the previous examples of attributes. In the declaration

```
const int n = 2;
```

the value 2 is bound statically to the name `n`, and in the declaration

```
int x;
```

the data type “integer” is bound statically to the name `x`. A similar statement holds for the function declaration.

On the other hand, the assignment `x = 2` binds the value 2 dynamically to `x` when the assignment statement is executed. And the C++ statement

```
y = new int;
```

dynamically binds a storage location to `*y` and assigns that location as the value of `y`.

Binding times can be further refined into subcategories of dynamic and static binding. A static attribute can be bound during parsing or semantic analysis (translation time), during the linking of the program with libraries (link time), or during the loading of the program for execution (load time). For example, the body of an externally defined function will not be bound until link time, and the location of a global variable is bound at load time, since its location does not change during the execution of the program. Similarly, a dynamic attribute can be bound at different times during execution—for example, on entry or exit from a procedure, or on entry or exit from the entire program.

Names can be bound to attributes even prior to translation time. Predefined identifiers, such as the Pascal data types `boolean` and `char`, have their meanings (and hence attributes) specified by the

language definition. Data type `boolean`, for example, is specified as having the two values `true` and `false`. Some predefined identifiers, such as data type `integer` and constant `maxint`, have their attributes specified by the language definition *and* by the implementation. The language definition specifies that data type `integer` has values consisting of a subset of the integers and that `maxint` is a constant, while the implementation specifies the value of `maxint` and the actual range of data type `integer`.³

Thus, we have the following possible binding times for attributes of names:

- Language definition time
- Language implementation time
- Translation time (“compile-time”)
- Link time
- Load time
- Execution time (“runtime”)

All binding times in this list, except for the last, represent static binding. (Dynamic binding times during execution were examined in Chapters 3 and 5.)

Bindings must be maintained by a translator so that appropriate meanings are given to names during translation and execution. A translator does this by creating a data structure to maintain the information. Since we are not interested in the details of this data structure, but only its properties, we can think of it abstractly as a function that expresses the binding of attributes to names. This function is a fundamental part of language semantics and is usually called the **symbol table**. Mathematically, the symbol table is a function from names to attributes, which we could write as $\text{SymbolTable} : \text{Names} \rightarrow \text{Attributes}$, or more graphically as shown in Figure 7.1.



Figure 7.1 Mapping names to attributes in a symbol table

This function will change as translation and/or execution proceeds to reflect additions and deletions of bindings within the program being translated and/or executed.

Viewed in this way, the parsing phase of translation includes three types of analysis:

1. Lexical analysis, which determines whether a string of characters represents a token in the vocabulary of the language
2. Syntax analysis, which determines whether a sequence of tokens represents a phrase in the context-free grammar of the language
3. Static semantic analysis, which is concerned with establishing the attributes of names in declarations and ensuring that the use of these names in references conforms to their declared attributes

³ This assumes that a program does not redefine the meanings of these names in a declaration. Redefinition is a possibility since these predefined identifiers are not reserved words.

During the execution of a compiled program, attributes such as locations and values must also be maintained. A compiler generates code that maintains these attributes in data structures during execution. The memory allocation part of this process—that is, the binding of names to storage locations is usually considered separately and is called the **environment**, as shown in Figure 7.2.



Figure 7.2 Mapping names to locations in an environment

Finally, the bindings of storage locations to values is called the **memory**, since it abstracts the memory of an actual computer (sometimes it is also called the **store** or the **state**). See Figure 7.3.

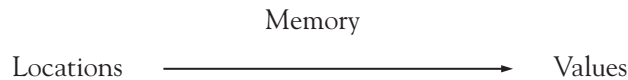


Figure 7.3 Mapping locations to values in a memory

7.2 Declarations, Blocks, and Scope

Declarations are, as you have seen, a principal method for establishing bindings. Bindings can be determined by a declaration either **implicitly** or **explicitly**. For example, the C declaration

```
int x;
```

establishes the data type of *x* explicitly using the keyword `int`, but the exact location of *x* during execution is only bound implicitly and, indeed, could be either static or dynamic, depending on the location of this declaration in the program. Similarly, the value of *x* is either implicitly zero or undefined, depending on the location of the declaration. On the other hand, the declaration

```
int x = 0;
```

explicitly binds 0 as the initial value of *x*.

Not only can bindings be implicit in a declaration, but the entire declaration itself may be implicit. For example, in some languages simply using the name of the variable causes it to be declared, although the first use usually takes the form of an assignment statement that binds a value to the name.

Sometimes a language will have different names for declarations that bind certain attributes but not others. For example, in C and C++, declarations that bind *all* potential attributes are called **definitions**, while declarations that only *partially* specify attributes are called simply declarations. For example, the function declaration (or **prototype**)

```
double f(int x);
```

specifies only the data type of the function *f* (i.e., the types of its parameters and return value) but does not specify the code to implement *f*. Similarly,

```
struct x;
```

specifies an **incomplete type** in C or C++, so this declaration is also not a definition. (Such specialized declarations are used to resolve recursive definitions and in cases where the complete definition can only be found at link time—see subsequent remarks in this chapter.)

In dynamically typed languages such as Lisp, Smalltalk, and Python, variable names typically have only a value binding, with the type attributes belonging to the associated values.

Declarations commonly appear in particular language constructs. One such standard language construct, typically called a **block**, consists of a sequence of declarations followed by a sequence of statements. Within a block, the sequence of statements following a declaration are surrounded by syntactic markers such as braces or begin-end pairs. In C, blocks are called **compound statements** and appear as the body of functions in function definitions, and also anywhere an ordinary program statement could appear. Thus,

```
void p (){
    double r, z; /* the block of p */
    ...
    { int x, y; /* another, nested, block */
      x = 2;
      y = 0;
      x += 1;
    }
    ...
}
```

establishes two blocks, one of which represents the body of `p` and the other nested inside the body of `p`.

In addition to declarations associated with blocks, C also has an “external” or “global” set of declarations outside any compound statement:

```
int x;
double y;
/* these are external to all functions
   and so global */

main()
{ int i, j; /* these are associated with the block of main only */
  ...
}
```

Declarations that are associated with a specific block are also called **local**, while declarations in surrounding blocks (or global declarations) are called **nonlocal** declarations. For example, in the previous definition of the procedure `p`, variables `r` and `z` are local to `p` but are nonlocal from within the second block nested inside `p`.

The notion of blocks containing declarations began with Algol60, and all Algol descendants exhibit this **block structure** in various ways. For example, in Ada, a block is formed by a `begin`-`end` pair preceded by the keyword `declare`:

```
declare x: integer;
        y: boolean;
begin
  x := 2;
  y := true;
  x := x + 1;
  ...
end;
```

In block-structured languages, blocks can be nested to an arbitrary depth, and names can be redeclared within nested blocks. Instead of identifying each new declared name with a unique integer (counting from 0), we can associate with each declared name a **level number** and an **offset**. This pair of numbers is called the name's **lexical address**. Names declared in the outermost level have the level number 0. The level number increases as we move into each nested block. The offsets of the names declared within each nested block are then numbered beginning with 0. Thus, in the following Ada code segment, the declarations of `x` and `y` in the outer block have the lexical addresses (0, 0) and (0, 1), whereas the same names in the nested block have the lexical addresses (1, 0) and (1, 1).

```
declare x: integer;
        y: boolean;
begin
  x := 2;
  y := true;
  x := x + 1;
  declare x: integer;
          y: boolean;
  begin
    x := 3;
    y := false;
    x := x * 6;
  end;
end;
```

Other language constructs besides blocks are important sources of declarations. For example, in C, a `struct` definition is composed of local variable (or **member**) declarations, which are within it:

```
struct A{
  int x; /* local to A */
  double y;
```

(continues)

(continued)

```
    struct{
        int* x; /* nested member declarations */
        char y;
    } z;
};
```

Note how the syntax with braces is almost, but not quite, that of a block. There is an extra semicolon at the end.

Similarly, in object-oriented languages, the **class** is an important source of declarations. Indeed, in pure object-oriented languages such as Java and Smalltalk, the class is the *only* declaration that does not itself need to be inside another class declaration. In other words, in pure object-oriented languages the class is the *primary* source of declarations. Consider the following example from Java (keeping in mind that in Java, functions are called methods and members are called fields):

```
/* Java example of class declaration */

public class IntWithGcd{
    ...
    /* local declaration of intValue method */
    public int intValue(){
        return value;
    }

    /* local declaration of gcd method */
    public int gcd( int v ){
        ...
    }

    /* local declaration of value field */
    private int value;
}
```

Finally, declarations can also be collected into larger groups as a way of organizing programs and providing special behavior. **Ada packages** and **tasks** belong to this category, as do **Java packages** (rather different from Ada packages), ML, Haskell, and Python **modules**, and C++ **namespaces**. These will be treated in more detail in Chapter 11.

Declarations bind various attributes to names, depending on the kind of declaration. The **scope of a binding** is the region of the program over which the binding is maintained. We can also refer to the scope of a declaration if all the bindings established by the declaration (or at least all of the bindings we are interested in at a particular moment) have identical scopes. Sometimes, by abuse of language, we refer to the scope of a name, but this is dangerous, since the same name may be involved in several different declarations, each with a different scope. For example, the following C code contains two declarations of the name *x*, with different meanings and different scopes:

```

void p(){
    int x;
    ...
}

void q(){
    char x;
    ...
}

```

In a block-structured language like C (and most modern languages), where blocks can be nested, the scope of a binding is limited to the block in which its associated declaration appears (and other blocks contained within it). Such a scope rule is called **lexical scope** because it follows the structure of the blocks as they appear in the written code.

A simple example of scope in C is given in Figure 7.4.

```

(1)  int x;

(2)  void p(){
(3)      char y;
(4)      ...
(5)  } /* p */

(6)  void q(){
(7)      double z;
(8)      ...
(9)  } /* q */

(10) main(){
(11)     int w[10];
(12)     ...
(13) }

```

Figure 7.4 Simple C program demonstrating scope

In Figure 7.4, the declarations of variable *x* (line 1) and procedures *p* (lines 2–5), *q* (lines 6–9), and *main* (lines 10–13) are global. The declarations of *y* (line 3), *z* (line 7), and *w* (line 11), on the other hand, are associated with the blocks of procedures *p*, *q*, and *main*, respectively. They are local to these functions, and their declarations are valid only for those functions. C has the further rule that the scope of a declaration begins at the point of the declaration itself. This is the so-called **declaration before use** rule: The scope of a declaration extends from the point just after the declaration to the end of the block in which it is located.⁴

⁴ C's scope rules are actually a bit more complicated, and we discuss a few of these complications further on in the chapter.

We repeat the example of Figure 7.4 in Figure 7.5, drawing brackets to indicate the scope of each declaration.

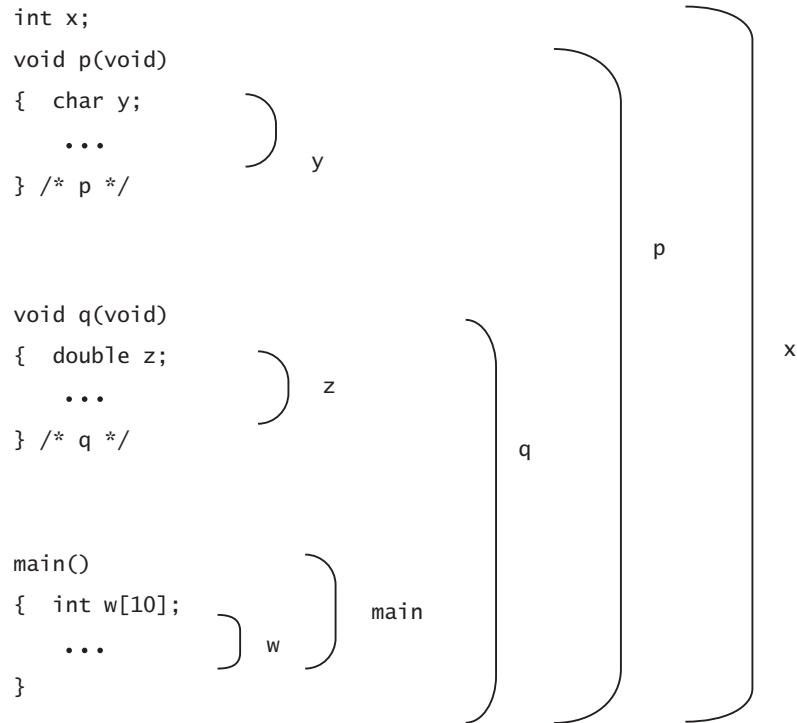


Figure 7.5 C Program from Figure 7.4 with brackets indicating scope

Blocks in other Algol-like languages establish similar scope rules. For example, in the following Ada code,

```

B1: declare
  x: integer;
  y: boolean;
begin
  x := 2;
  y := false;
B2: declare
  a, b: integer;
begin
  if y then a := x;
  else b := x;
  end if;
end B2;
...
end B1;

```

the scope of the declarations of *x* and *y* is all of block B1 (including block B2), while the scope of the declarations of *a* and *b* is block B2 only.

One feature of block structure is that declarations in nested blocks take precedence over previous declarations. Consider the following example:

```
(1) int x;

(2) void p(){
(3)     char x;
(4)     x = 'a'; /* assigns to char x */
(5)     ...
(6) }

(7) main(){
(8)     x = 2; /* assigns to global x */
(9)     ...
(10) }
```

The declaration of *x* in *p* (line 3) takes precedence over the global declaration of *x* (line 1) in the body of *p*. Thus, the global integer *x* cannot be accessed from within *p*. The global declaration of *x* is said to have a **scope hole** inside *p*, and the local declaration of *x* is said to **shadow** its global declaration. For this reason, a distinction is sometimes made between the scope and the visibility of a declaration. **Visibility** includes only those regions of a program where the bindings of a declaration apply, while scope includes scope holes (since the bindings still exist but are hidden from view). In C++, the **scope resolution operator** `::` (double colon) can be used to access such hidden declarations (as long as they are global). For example:

```
int x;

void p(){
char x;
    x = 'a'; // assigns to char x
    ::x = 42; // assigns to global int x
    ...
}

main(){
x = 2; // assigns to global x
    ...
}
```

This demonstrates the fact that operators and declaration modifiers can alter the accessibility and scope of declarations—a widespread phenomenon in programming languages. For example, Ada allows

any named enclosing scope to be accessed using a dot notation similar to record structure access, so the above C++ behavior can also be achieved in Ada, as follows:

```
B1: declare
  a: integer;
  y: boolean;
begin
  a := 2;
  y := false;
  B2: declare
    a, b: integer; -- local a now hides a in B1
  begin
    if y then a := B1.a; -- select the a in B1
    else b := B1.a;
    end if;
  end B2;
  ...
end B1;
```

This is called **visibility by selection** in Ada.

C++ also uses the scope resolution operator to enter the scope of a class from the “outside” to provide missing definitions, since C++ only requires declarations, not complete definitions, within a class declaration:

```
/* C++ example of class declaration similar to previous Java example */
class IntWithGcd{

  public:
  /* local declaration of intValue function - code for intValue() is not
  provided */
  int intValue();

  int gcd ( int v ); /* local declaration of gcd */

  private:
  int value; /* local declaration of value member */
};

/* use scope resolution to give actual definition of intValue */
int IntWithGcd::intValue(){
  return value;
}
```

One further example in this vein is the way additional keyword modifiers in a declaration can alter the scope of the declaration. In C, global variable declarations can actually be accessed across files, using the `extern` keyword:

File 1:

```
extern int x; /* uses x from somewhere else */
...
```

File 2:

```
int x; /* a global x that can be accessed by other files */
...
```

Now if File 1 and File 2 are compiled separately and then linked together, the external `x` in File 1 will be identified by the linker with the `x` provided in File 2. If, on the other hand, when writing File 2, the programmer wishes to restrict the scope of `x` to File 2 alone and, thus, prevent external references, the programmer may use the `static` keyword:

File 2:

```
/* a global x that can only be seen within this file */
static int x;
...
```

Now an attempt to link File 1 with File 2 will result in an error: undefined reference to `x`. This is a primitive version of the kind of scope-modifying access control that classes and modules provide, which is discussed in detail in Chapters 5 and 11.

Scope rules need also to be constructed in such a way that recursive, or self-referential, declarations are possible when they make sense. Since functions must be allowed to be recursive, this means that the declaration of a function name has scope beginning *before* the block of the function body is entered:

```
int factorial (int n){
    /* scope of factorial begins here */
    /* factorial can be called here */
    ...
}
```

On the other hand, recursive variable declarations don't, in general, make sense:

```
int x = x + 1; /* ??? */
```

This declaration should probably be an error, and indeed Java and Ada flag this as an error, even though the scope of the `x` begins at the `=` sign. In Ada, this is true because self-references in variable declarations are specifically outlawed. In Java, this is due to the fact that the self-reference is flagged as an uninitialized use. C and C++, however, do not flag this as an error if `x` is a local variable⁵, but simply add one to the (probably random) uninitialized value of `x`.

⁵ Global variables in C and C++ cannot have such an initializer expression, since their initial values must be compile-time quantities; see Section 7.6.2.

A special scoping situation, related to recursion, occurs in class declarations in object-oriented languages. Local declarations inside a class declaration generally have scope that extends backwards to include the entire class (thus, partially suspending the declaration before use rule). For example, in the previous `IntWithGcd` example in Java, the local data `value` was referenced in the class declaration even before it was defined. This rule exists so that the order of declarations within a class does not matter. Translators for such languages must generally perform two passes to parse a program. In the first pass, the parser enters all of the declared names into a symbol table. In the second pass, the parser looks up each reference to a name in the table to verify that a declaration exists.

The way the symbol table processes declarations must correspond to the scope of each declaration. Thus, different scope rules require different behavior by, and even different structure within, the symbol table. In the next section, we examine the basic ways that a symbol table can help in analyzing scope in a block-structured language.

7.3 The Symbol Table

A symbol table is like a dictionary: It must support the insertion, lookup, and deletion⁶ of names with associated attributes, representing the bindings in declarations. A symbol can be implemented with any number of data structures to allow for efficient access, such as hash tables and binary search trees. However, the maintenance of scope information in a lexically scoped language with block structure requires that declarations be processed in a stacklike fashion. That is, on entry into a block, all declarations of that block are processed and the corresponding bindings added to the symbol table. Then, on exit from the block, the bindings provided by the declarations are removed, restoring any previous bindings that may have existed. This process is called **scope analysis**. Without restricting our view of a symbol table to any particular data structure, we may nevertheless view the symbol table schematically as a collection of names, each of which has a stack of declarations associated with it, such that the declaration on top of the stack is the one whose scope is currently active. To see how this works, consider the C program of Figure 7.6.

```
(1) int x;
(2) char y;

(3) void p(){
(4)     double x;
(5)     ...
(6)     { int y[10];
(7)         ...
(8)     }
(9)     ...
(10) }

(11) void q(){
(12)     int y;
```

Figure 7.6 C program demonstrating symbol table structure (*continues*)

⁶ Deletion, of course, may not mean complete erasure from the symbol table, but simply marking it as no longer active, since bindings may need to be revisited even after a scope is exited by the translator.

(continued)

```
(13)    ...
(14) }

(15) main(){
(16)     char x;
(17)     ...
(18) }
```

Figure 7.6 C program demonstrating symbol table structure

The names in this program are `x`, `y`, `p`, `q`, and `main`, but `x` and `y` are each associated with three different declarations with different scopes. Right after the processing of the variable declaration at the beginning of the body of `p` (line 5 of figure 7.6), the symbol table can be represented as in Figure 7.7. (Since all function definitions are global in C, we do not indicate this explicitly in the attributes of Figure 7.7 and subsequent figures.)

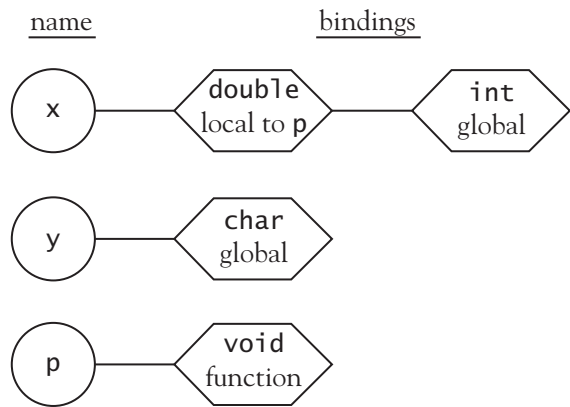


Figure 7.7 Symbol table structure at line 5 of Figure 7.6

After the processing of the declaration of the nested block in `p`, with the local declaration of `y` (i.e., at line 7 of Figure 7.6), the symbol table is as shown in Figure 7.8.

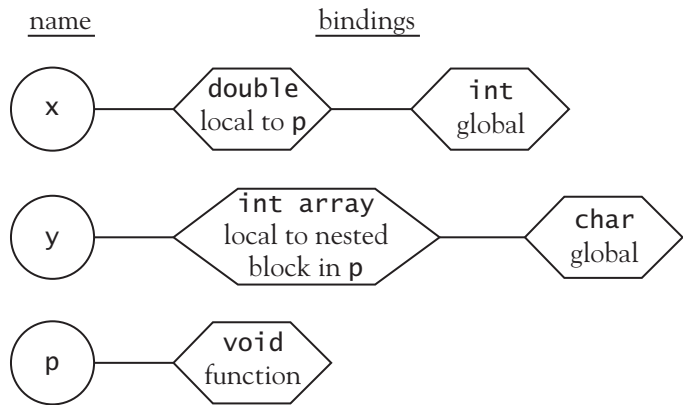


Figure 7.8 Symbol table structure at line 7 of Figure 7.6

Then, when the processing of *p* has finished (line 10 of Figure 7.6), the symbol table becomes as shown in Figure 7.9 (with the local declaration of *y* popped from the stack of *y* declarations at line 8 and then the local declaration of *x* popped from the *x* stack at line 10).

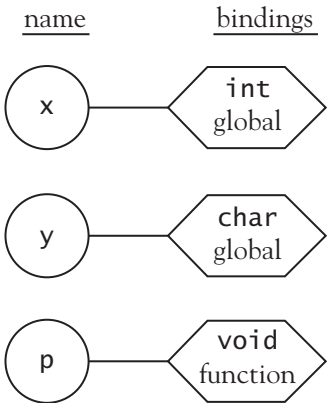


Figure 7.9 Symbol table structure at line 10 of Figure 7.6

After *q* is entered (line 13), the symbol table becomes like the one shown in Figure 7.10.

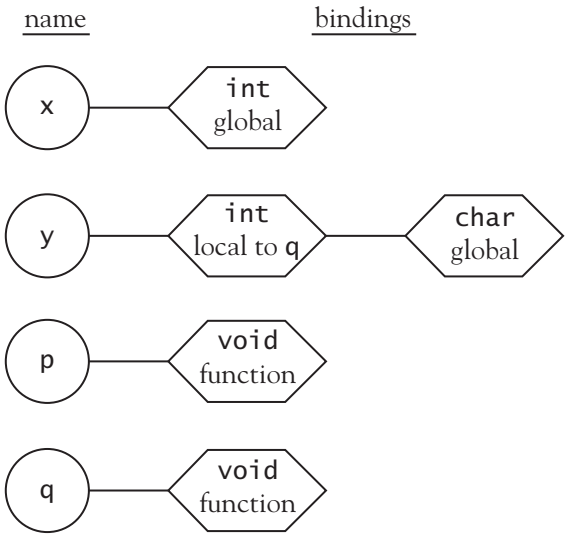


Figure 7.10 Symbol table structure at line 13 of Figure 7.6

When `q` exits (line 14), the symbol table is as shown in Figure 7.11.

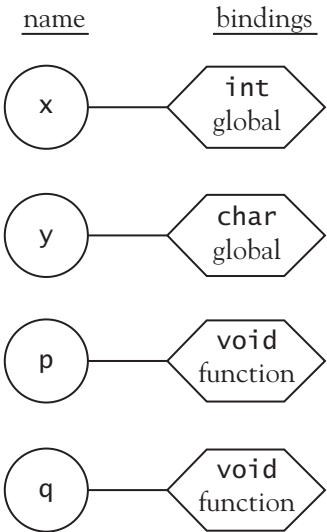


Figure 7.11 Symbol table structure at line 14 of Figure 7.6

Finally, after `main` is entered (line 17), the symbol table is as shown in Figure 7.12.

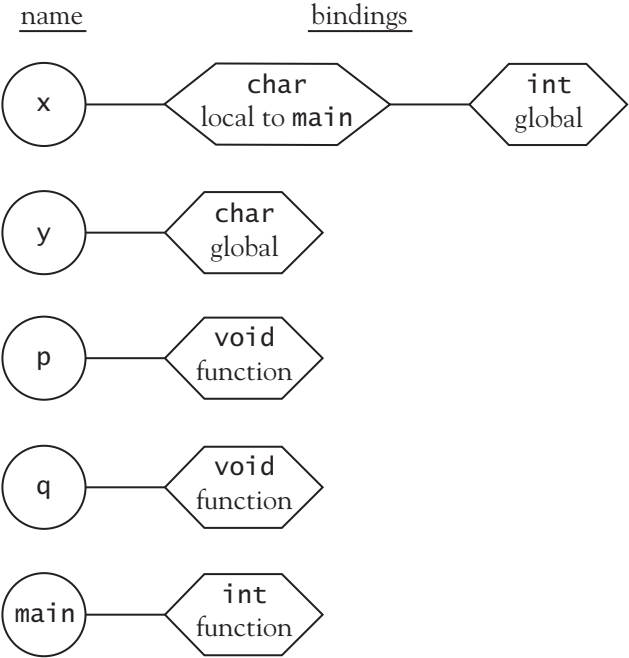


Figure 7.12 Symbol table structure at line 17 of Figure 7.6

Note that this process maintains the appropriate scope information, including the scope holes for the global declaration of `x` inside `p` and the global declaration of `y` inside `q`.

This representation of the symbol table assumes that the symbol table processes the declarations statically—that is, prior to execution. This is the case if the symbol table is managed by a compiler, and the bindings of the declarations are all static. However, if the symbol table is managed dynamically, that is, during execution, then declarations are processed as they are encountered along an execution path through the program. This results in a different scope rule, which is usually called **dynamic scoping**, and our previous lexical scoping rule is sometimes called **static scoping**.

To show the difference between static and dynamic scoping, let us add some code to the example of Figure 7.6, so that an execution path is established, and also give some values to the variables, so that some output can be generated to emphasize this distinction. The revised example is given in Figure 7.13.

```
(1) #include <stdio.h>

(2) int x = 1;
(3) char y = 'a';
(4) void p(){
(5)     double x = 2.5;
(6)     printf("%c\n",y);
(7)     { int y[10];
(8)     }
(9) }

(10) void q(){
(11)     int y = 42;
(12)     printf("%d\n",x);
(13)     p();
(14) }

(15) main(){
(16)     char x = 'b';
(17)     q();
(18)     return 0;
(19) }
```

Figure 7.13 C program of Figure 7.6 with added code

Now consider what would happen if the symbol table for the code of Figure 7.13 were constructed dynamically as execution proceeds. First, execution begins with `main`, and all the global declarations must be processed before `main` begins execution, since `main` must know about all the declarations that occur before it. Thus, the symbol table at the beginning of the execution (line 17) is as shown in Figure 7.14.

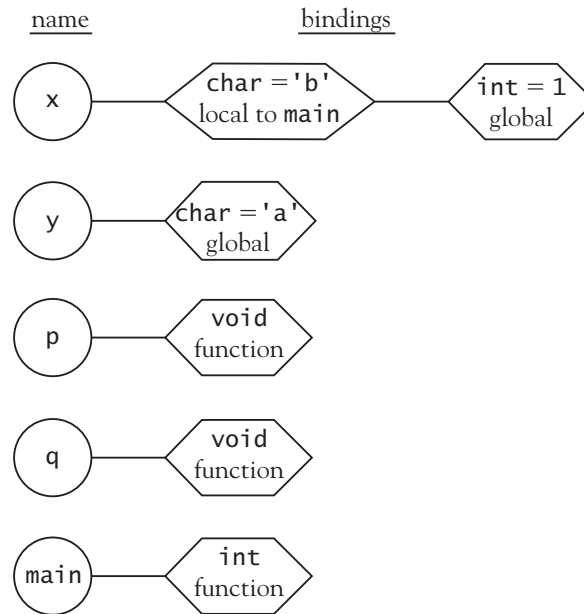


Figure 7.14 Symbol table structure at line 17 of Figure 7.13 using dynamic scope

Note that this is the same symbol table used when `main` is processed statically (Figure 7.12), except that we have now added value attributes to the picture. Note we have yet to process the bodies of any of the other functions.

Now `main` proceeds to call `q`, and we begin processing the body of `q`. The symbol table on entry into `q` (line 12) is then as shown in Figure 7.15.

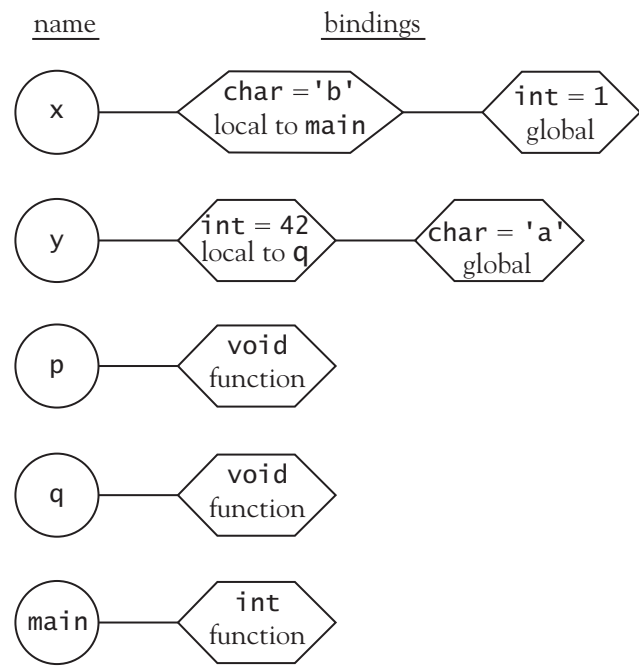


Figure 7.15 Symbol table structure at line 12 of Figure 7.13 using dynamic scope

This is now quite different from the symbol table on entry into q using static processing (Figure 7.10). Note also that *each* call to q may have a different symbol table on entry, depending on the execution path to that call, whereas when using lexical scoping, each procedure has only one symbol table associated with its entry. (Since there is only one call to q in this example, this doesn't happen in this case.)

To continue with the example, q now calls p, and the symbol table becomes like the one shown in Figure 7.16.

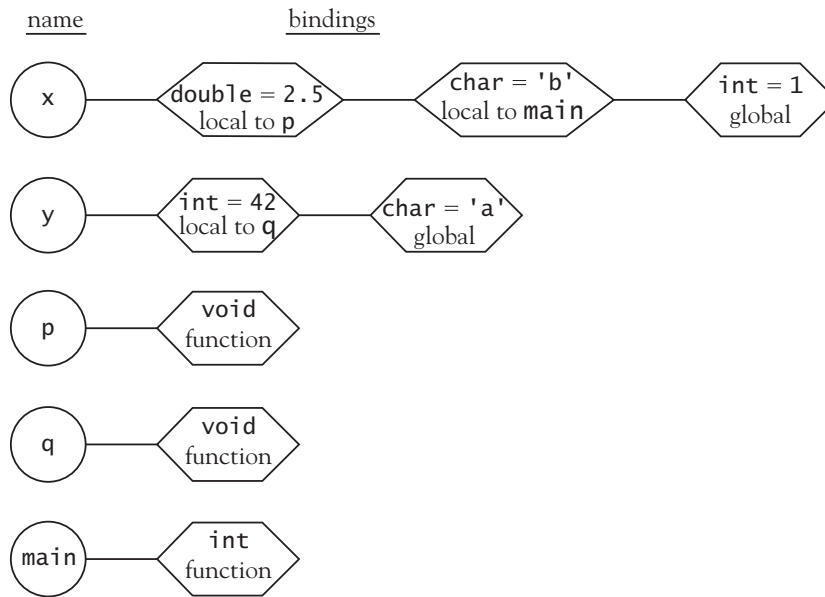


Figure 7.16 Symbol table structure at line 6 of Figure 7.13 using dynamic scope

Now consider how dynamic scoping will affect the semantics of this program and produce different output. First, note that the actual output of this program (using lexical scoping, which is the standard for most languages, including C) is:

```
1
a
```

since in the first `printf` statement (line 6), the reference is to the global `y`, regardless of the execution path, and in the second `printf` statement (line 12) the reference is to the global `x`, again regardless of the execution path, and the values of these variables are ‘a’ and 1 throughout the program.

However, using dynamic scoping,⁷ the nonlocal references to `y` and `x` inside `p` and `q`, respectively, can change, depending on the execution path. In this program, the `printf` statement at line 12 of Figure 7.13 is reached with the symbol table as in Figure 7.15 (with a new declaration of `x` as the character “b” inside `main`). Thus, the `printf` statement will print this character interpreted as an integer (because of the format string “%d\n”) as the ASCII value 98. Second, the `printf` reference to `y` inside `p` (line 6 of Figure 7.13) is now the integer `y` with value 42 defined inside `q`, as shown in Figure 7.16. This value will now be interpreted as a character (ASCII 42 = ‘*’), and the program will print the following:

```
98
*
```

⁷ Of course, since C uses lexical scope, we cannot actually execute this program this way. We are using C syntax only as a convenience.

This example brings up many of the issues that make dynamic scoping problematic, and why few languages use it. The first, and perhaps foremost problem, is that under dynamic scoping, when a nonlocal name is used in an expression or statement, the declaration that applies to that name cannot be determined by simply reading the program. Instead, the program must be executed, or its execution traced by hand, to find the applicable declaration (and different executions may lead to different results). Therefore, the semantics of a function can change radically as execution proceeds. You saw this in the example above, where the reference to `y` in the `printf` statement on line 6 of Figure 7.13 cannot be known until execution time (if we wanted, we could have rewritten this example to make the `y` reference actually depend on user input). Thus, the semantics of `p` change during execution.

The example demonstrates another serious problem: Since nonlocal variable references cannot be predicted prior to execution, neither can the data types of these variables. In the example, the reference to `y` in the `printf` statement on line 6 of Figure 7.13 is assumed to be a character (from the data type of the global `y`), and we therefore write a character format directive `"%c"` in the format string of the `printf` function. However, with dynamic scoping, the variable `y` could have any type when `p` is executed. That means this formatting directive is likely to be wrong. In the example, we were lucky that C has very liberal conversion rules between two basic data types such as `char` and `int`. However, imagine what would happen if the global `y` or the `y` local to `q` were a `double` or even a user-defined structure—we still are likely not to get a runtime error in C, but predicting the output can be difficult (and machine dependent!). In other, stricter languages like Ada or Pascal, this would definitely lead to a runtime error—not at all what we would like. Thus, static binding of data types (static typing) and dynamic scoping are inherently incompatible. (See Chapter 8 for more on this issue.)

Nevertheless, dynamic scoping remains a possible option for highly dynamic, interpreted languages when we do not expect programs to be extremely large. The reason is that the runtime environment is made considerably simpler by using dynamic scoping in an interpreter. Indeed, at the end of Section 7.1, we noted that the environment in an interpreter includes the symbol table, so it may seem that it is impossible to maintain lexical scope in an interpreter, since, by definition, the symbol table is maintained dynamically. This is not the case. However, maintaining lexical scope dynamically does require some extra structure and bookkeeping (see Chapter 10). Thus, languages like APL, Snobol, and Perl (all interpreted languages with relatively small expected programs) have traditionally opted for dynamic scoping.⁸

Early dialects of Lisp, too, were dynamically scoped. The inventor of Lisp, John McCarthy, has stated that he felt this was a bug in the initial implementation of Lisp in 1958 (McCarthy [1981], page 180). However, as mentioned in Chapter 3, the popular Scheme dialect of Lisp has used lexical scoping from its inception (Steele and Gabriel [1996], page 249), and Common Lisp (Steele [1982] [1984]) offers lexical scoping as its default option.

Aside from the question of lexical versus dynamic scope, there is significant additional complexity surrounding the structure and behavior of the symbol table that we have not yet discussed. Indeed, the symbol table as we have discussed it so far—a single table for an entire program, with insertions on entry

⁸ Recent versions of Perl now offer lexical scoping.

into a scope and deletions on exit—is appropriate only for the simplest languages, such as C and Pascal, with a strict declaration before use requirement, and where scopes cannot be reaccessed once they have been exited during processing by a translator.

Actually, even this description does not fully cover all situations in C and Pascal. Consider, for example, struct declarations in C (or record declarations in Pascal) as given in Figure 7.17. Clearly, each of the two struct declarations in this code (lines 1–5 and 7–11 of Figure 7.17) must contain the further declarations of the data fields within each struct, and these declarations must be accessible (using the “dot” notation of member selection) whenever the struct variables themselves (*x* and *y*) are in scope. This means two things: (1) A struct declaration actually contains a local symbol table itself as an attribute (which contains the member declarations), and (2) this local symbol table cannot be deleted until the struct variable itself is deleted from the “global” symbol table of the program.

```
(1) struct{
(2)     int a;
(3)     char b;
(4)     double c;
(5) } x = {1, 'a', 2.5};

(6) void p(){
(7)     struct{
(8)         double a;
(9)         int b;
(10)        char c;
(11)    } y = {1.2, 2, 'b'};
(12)    printf("%d, %c, %g\n", x.a, x.b, x.c);
(13)    printf("%f, %d, %c\n", y.a, y.b, y.c);
(14) }

(15) main(){
(16)    p();
(17)    return 0;
(18) }
```

Figure 7.17 Code example illustrating scope of local declarations in a C struct

Thus, inside *p* (line 12 of Figure 7.17), the symbol table for the preceding code might look as shown in Figure 7.18, with the symbol table attribute labeled as “syntab”).

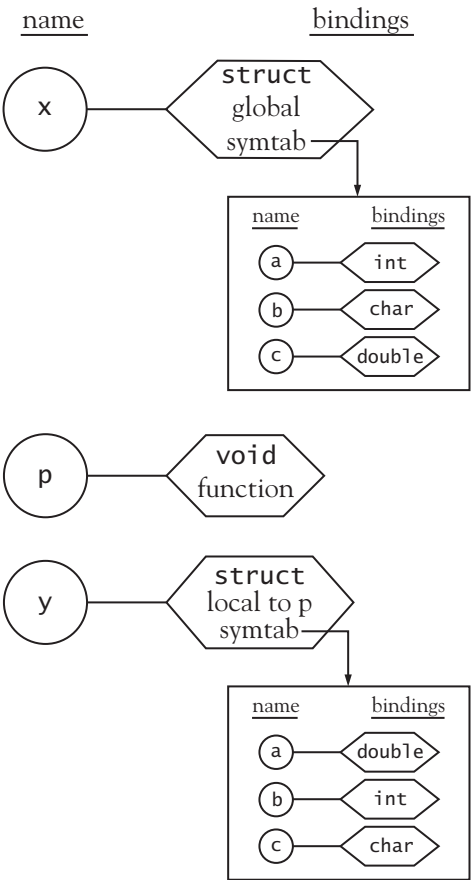


Figure 7.18 Representation of the symbol table structure at line 12 of the program of Figure 7.17, showing local `struct` symbol tables

Any scoping structure that can be referenced directly in a language must also have its own symbol table. Examples include all named scopes in Ada; classes, structs, and namespaces in C++; and classes and packages in Java. Thus, a more typical structure for the symbol table of a program in any of these languages is to have a table for each scope. When each new scope is entered during the parsing of a source program, a new symbol table for it is pushed onto a stack of symbol tables. The names declared in this scope are then entered into this table. If a duplicate name is encountered at this point, the name has already been declared in the same scope and an error is returned. When a reference to a name is encountered, a search for it begins in the table at the top of the stack. If an entry for the name is not found in this table, the search continues in the next table below it, and so on, until an entry is found or the bottom of the stack is reached. If the bottom of the stack is reached, the name is undeclared and an error is returned.

As a second example of this phenomenon, we rewrite the C example of Figure 7.13 into Ada as shown in Figure 7.19.

```

(1) with Text_IO; use Text_IO;
(2) with Ada.Integer_Text_IO; use Ada.Integer_Text_IO;

(3) procedure ex is
(4)   x: integer := 1;
(5)   y: character := 'a';

(6)   procedure p is
(7)     x: float := 2.5;
(8)     begin
(9)       put(y); new_line;
(10)    A: declare
(11)      y: array (1..10) of integer;
(12)      begin
(13)        y(1) := 2;
(14)        put(y(1)); new_line;
(15)        put(ex.y); new_line;
(16)      end A;
(17)    end p;

(18)   procedure q is
(19)     y: integer := 42;
(20)     begin
(21)       put(x); new_line;
(22)       p;
(23)     end q;

(24)   begin
(25)     declare
(26)       x: character := 'b';
(27)       begin
(28)         q;
(29)         put(ex.x); new_line;
(30)       end;
(31)   end ex;

```

Figure 7.19 Ada code corresponding to Figure 7.13

A possible organization for the symbol table of this program after entry into block A inside p (line 12 of Figure 7.19) is shown in Figure 7.20 (we ignore the question of how to represent the imported packages). Note in Figure 7.20 that we no longer have to mention which specific scope each declaration belongs to, since membership in a particular symbol table already gives us that information. Also, note that if a lookup in an inner table fails, the lookup must proceed with the next outermost table. Thus, as indicated in the diagram with dotted arrows, there must be links from each inner scope to the next outer scope.

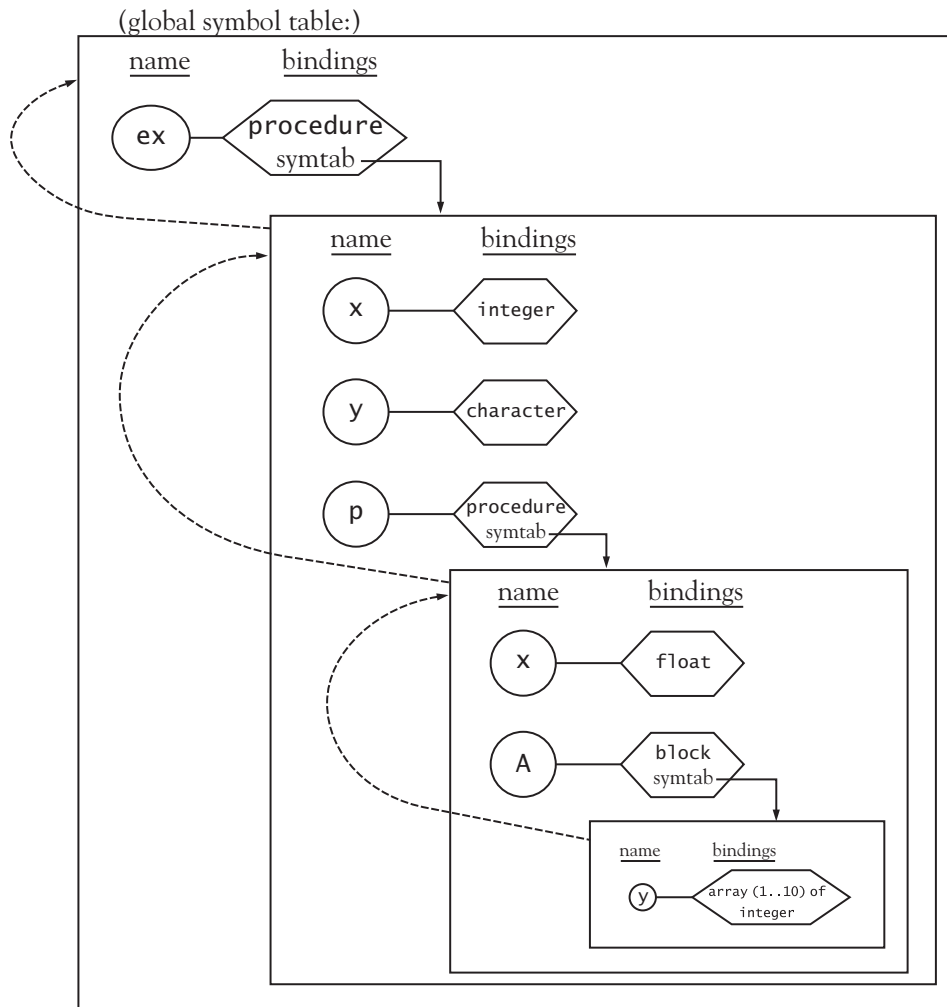


Figure 7.20 Symbol table structure at line 12 of Figure 7.19

Consider, for example, how the lookup of `ex.y` in line 15 of Figure 7.19 would proceed from within block A in function p. First, `ex` would be looked up in the symbol table of A. Since this lookup fails, the next outermost symbol table would be consulted (the symbol table of p) by following the dotted line. This lookup also fails, and the next symbol table—that of `ex`—is consulted. This fails too, and finally the outermost symbol table is arrived at, which has only the entry for `ex`. This is the entry one we want. With the entry found, the lookup of `y` proceeds (the `ex.` in `ex.y` is stripped off for this lookup). The character variable `y` is found. This is the variable that is meant in line 15.

A final question about symbol tables and scope is to what extent the same name can be reused for different functions, variables, and other entities within the same scope. Such reuse, which is called overloading, is discussed, along with related issues, in the next section.

7.4 Name Resolution and Overloading

An important question about declarations and the operation of the symbol table in a programming language is to what extent the same name can be used to refer to different things in a program. One might at first consider that this should not be allowed, as it can lead to serious confusion and unreadability. However, consider, for example, the addition operation denoted by a `+` sign, which is typically a built-in binary infix⁹ operator in most languages and whose result is the sum of its two operands. In virtually all languages, this simple `+` sign refers to at least two (and often more) completely different operations: integer addition and floating-point addition (typically denoted in assembly language by two different symbols `ADDI` and `ADDF`, or similar names). The `+` operator is then said to be **overloaded**. Clearly this reuse of the same symbol does not cause confusion, since these operations are closely related (and are even “the same” in some mathematical sense). Just as clearly, it would be a major annoyance if a programming language required programmers to use two different symbols for these operations (say `+%` for integer addition and `+#` for floating-point addition).

How does a translator disambiguate (distinguish between) these two uses of the “`+`” symbol?

It does so by looking at the data types of the operands. For example, in C, if we write `2+3`, we mean integer addition, and if we write `2.1+3.2`, we mean floating-point addition. Of course, `2+3.2` is still potentially ambiguous, and the rules of a language must deal in some way with this case (most languages automatically convert `2` to `2.0`, but Ada says this is an error).

It is, therefore, a major advantage for a language to allow overloading of operators and function names based in some way on data types. C++ and Ada allow extensive overloading of both function names and operators. Java also allows overloading, but only on function names, not operators.¹⁰ The functional language Haskell also allows overloading of both functions and operators, and even allows new operators to be defined (which is not allowed in C++ or Ada). Several versions of FORTRAN (Fortran90/95 and later) also allow limited forms of overloading. We discuss overloading here using C++, Ada, and Java (which have similar, but not identical, mechanisms). Overloading in Haskell was described briefly in Chapter 3. See the Notes and References for overloading in Fortran 90/95.

The basic method for allowing overloaded functions is to expand the operation of the lookup operation in the symbol table to perform lookups based not only on the name of a function but also on the number of its parameters and their data types. This process of choosing a unique function among many with the same name, which represents an expansion of the symbol table capabilities, is called **overload resolution**.

To see how overload resolution works, we consider the simple but useful example of a `max` function for numbers. This function can be defined for integers, doubles, and all other numeric types, and may have two, three, or four (or even more) parameters. For our purposes, we define only three of the possible `max` functions (in C++ syntax)¹¹ in Figure 7.21.

⁹ That is, written between its operands.

¹⁰ At least, not in the current version of Java—a proposal exists to allow operator overloading in future versions.

¹¹ These functions can be easily turned into Ada code, but Java does not have freestanding functions; to write this example in Java, we would write these functions as static methods inside the class containing the `main` function.


```

int max(int x, int y){ // max #1
    return x > y ? x : y;
}

double max(double x, double y){ // max #2
    return x > y ? x : y;
}

int max(int x, int y, int z){ // max #3
    return x > y ? (x > z ? x : z) : (y > z ? y : z);
}

```

Figure 7.21 Three overloaded `max` functions in C++

We will refer to these as `max #1`, `max #2`, and `max #3`, respectively.

Now consider the following calls:

```

max(2,3); // calls max #1
max(2.1,3.2); // calls max #2
max(1,3,2); // calls max #3

```

The symbol table can easily determine the appropriate `max` function for each of these calls from the information contained in each call (the **calling context**)—it just needs to count the number of parameters and then look (in the two-parameter case) to see if the parameter values are integers or doubles.

Difficulties do arise in determining which of several overloaded definitions should be used when several different definitions of an overloaded function are equally likely in a particular calling context under the rules of the language. Consider, for example, the following call:

```
max(2.1,3); // which max?
```

Here the answer depends on the language rules for converting a value of one type to another; see Chapter 8. In C++, for example, this call is **ambiguous**: C++ allows conversions both from integer to double and from double to integer (automatic truncation). Thus, the preceding call could be converted to either:

```
max(2,3); // max #1
```

or

```
max(2.1,3.0); // max #2
```

and the language rules do not say which conversion should be preferred. In Ada, too, the call `max(2.1,3)` is illegal, but for a different reason: No automatic conversions at all are allowed in Ada. In Ada, all parameter types must match a definition exactly. On the other hand, this call is perfectly legitimate in Java since Java permits conversions that do not lose information. Thus, Java would convert 3 to 3.0 and call `max #2` since the other possible call involves converting 2.1 to 2 and would result in information loss (the fractional part would disappear). Note also that the call:

```
max(2.1,4,3);
```

is legal in C++ (and results in 2.1 being truncated to 2), but it is not legal in either Ada or Java. On the other hand, suppose we add the definitions of Figure 7.22 to those of Figure 7.21.

```
double max(int x, double y){ // max #4
    return x > y ? (double) x : y;
}

double max(double x, int y){ // max #5
    return x > y ? x : (double) y;
}
```

Figure 7.22 Two more overloaded `max` functions in C++ (see Figure 7.21)

Then, the calls `max(2.1, 3)` and `max(2, 3.2)` become legal in C++ and Ada, since now there is an exact match for each call. Of course, these extra definitions are unnecessary (but not harmful) in Java.

Thus, automatic conversions as they exist in C++ and Java significantly complicate the process of overload resolution (but less so in Java than C++ because of Java's more restrictive conversion policy).

An additional issue in overload resolution is to what extent additional information in a calling context, besides the number and types of the parameter values, can be used. Ada, for example, allows the return type and even the names of the parameters in a definition to be used. Consider the Ada program in Figure 7.23.

```
(1) procedure overload is
(2)   function max(x: integer; y: integer) -- max #1
(3)     return integer is
(4)   begin
(5)     if x > y then return x;
(6)     else return y;
(7)     end if;
(8)   end max;

(9)   function max(x: integer; y: integer) -- max #2
(10)    return float is
(11)  begin
(12)    if x > y then return float(x);
(13)    else return float(y);
(14)    end if;
(15)  end max;

(16) a: integer;
(17) b: float;
(18) begin -- max_test
(19)   a := max(2,3); -- call to max # 1
(20)   b := max(2,3); -- call to max # 2
(21) end overload;
```

Figure 7.23 An example of `max` function overloading in Ada

Because line 19 of Figure 7.23 assigns the result of the `max` call to `integer a`, while line 20 assigns the result to `float b`, Ada can still resolve the overloading because there are no automatic conversions. C++ and Java, on the other hand, ignore the return type (if they didn't, the rules for overload resolution would become even more complex, with more room for ambiguity), so this program would be an error in either of those languages.

Both Ada and C++ (but not Java) also allow built-in operators to be extended by overloading. For example, suppose that we have a data structure in C++:

```
typedef struct { int i; double d; } IntDouble;
```

We can then define a “+” and a “<” (less than) operation on `IntDouble` as follows:

```
bool operator < (IntDouble x, IntDouble y){
    return x.i < y.i && x.d < y.d;
}

IntDouble operator + (IntDouble x, IntDouble y){
    IntDouble z;
    z.i = x.i + y.i;
    z.d = x.d + y.d;
    return z;
}
```

Now code such as:

```
IntDouble x, y;
...
if (x < y) x = x + y;
else y = x + y;
```

is possible. A complete, runnable example of this C++ code is given in Figure 7.24. A corresponding Ada example is given in Figure 7.25.

```
#include <iostream>

using namespace std;

typedef struct { int i; double d; } IntDouble;

bool operator < (IntDouble x, IntDouble y){
    return x.i < y.i && x.d < y.d;
}
```

Figure 7.24 A complete C++ program illustrating operator overloading (*continues*)

(continued)

```

IntDouble operator + (IntDouble x, IntDouble y){
    IntDouble z;
    z.i = x.i + y.i;
    z.d = x.d + y.d;
    return z;
}

int main(){
    IntDouble x = {1,2.1}, y = {5,3.4};
    if (x < y) x = x + y;
    else y = x + y;
    cout << x.i << " " << x.d << endl;
    return 0;
}

```

Figure 7.24 A complete C++ program illustrating operator overloading

```

with Text_IO; use Text_IO;
with Ada.Integer_Text_IO;
use Ada.Integer_Text_IO;
with Ada.Float_Text_IO;
use Ada.Float_Text_IO;

procedure opover is

type IntDouble is
record
    i: Integer;
    d: Float;
end record;

function "<" (x,y: IntDouble) return Boolean is
begin
    return x.i < y.i and x.d < y.d;
end "<";

function "+" (x,y: IntDouble) return IntDouble is
    z: IntDouble;
begin
    z.i := x.i + y.i;
    z.d := x.d + y.d;
    return z;

```

Figure 7.25 A complete Ada program illustrating operator overloading analogous to the C++ program of Figure 7.24 *(continues)*

(continued)

```

end "+";
x, y: IntDouble;

begin
  x := (1,2.1);
  y := (5,3.4);
  if (x < y) then x := x + y;
  else y := x + y;
  end if;
  put(x.i); put(" "); put(x.d); new_line;
end opover;

```

Figure 7.25 A complete Ada program illustrating operator overloading analogous to the C++ program of Figure 7.24

Of course, when overloading a built-in operator, we must accept the syntactic properties of the operator. That is, we cannot change their associativity or precedence. Indeed, there is no *semantic* difference between operators and functions, only the *syntactic* difference that operators are written in infix form, while function calls are always written in prefix form. In fact, in Ada, all operators also have a prefix form: "+"(3, 2) means the same as 3 + 2. C++ also allows prefix form for operators but only for overloaded ones applied to user-defined types:

```

// x and y are IntDoubles as above
x = operator + (x, y);

```

In Ada one can even redefine the built-in operators using overloading, while in C++ this is prohibited.

In Ada the operators that are allowed to be overloaded are those that traditionally have been viewed as infix or prefix mathematical operators. In C++, however, a much wider class of operations are viewed as overloadable operators. For example, the subscript operation can be viewed as a binary operator (with the "[" part of the operator symbol written infix and the "]" part of the operator symbol written postfix). These operations in C++ can only be overridden using object-oriented constructs (in the parlance of object-oriented programming, these overloaded functions must be "member functions" defined in a C++ class). Indeed, as you learned in Chapter 5, further opportunities for overloading are provided by object-oriented techniques.

Up to this point, we have discussed only overloading of functions—by far the most important case. Now let's consider an additional potential for overloading: reusing the same name for things of completely different kinds. For example, could we use the same name to indicate both a data type and a variable, or both a variable and a function? In most languages, this is not permitted, and with good reason: It can be extremely confusing, and programs have little to gain by using such overloading (as opposed to function or operator overloading, which is very useful).

Occasionally, however, such overloading comes in handy as a way of limiting the number of names that one actually has to think about in a program. Take, for example, a typical C definition of a recursive type:

```
struct A{
    int data;
    struct A * next;
};
```

This is “old-style” C in which the data type is given by the struct name A, but the keyword struct must also always appear. Using a typedef one can remove the need for repeating the struct:

```
typedef struct A A;
struct A{
    int data;
    A * next;
};
```

Notice that we have used the same name, A, both for the struct name and the typedef name. This is legal in C, and it is useful in this case, since it reduces the number of names that we need to remember.¹² This implies that struct names in C occupy a different symbol table than other names, so the principle of unique names for types (and variables) can be preserved.¹³ Some languages have extreme forms of this principle, with different symbol tables for each of the major kinds of definitions, so that one could, for example, use the same name for a type, a function, and a variable (why this might be useful is another question).

Java is a good example of this extreme form of name overloading, where the code of Figure 7.26 is perfectly acceptable to a Java compiler.¹⁴

```
class A
{
    A A(A A)
    {
        A:
        for(;;)
        {
            if (A.A(A) == A) break A;
        }
        return A;
    }
}
```

Figure 7.26 Java class definition showing overloading of the same name for different language constructs (adapted from Arnold, Gosling, and Holmes [2000], p. 153)

Figure 7.26 includes a definition of a class, a method (a function), a parameter, and a label, all of which share the name A. In Exercise 7.20, you will have the chance to explain which instances of a name represent definitions and which represent references.

¹² In C++, the typedef is unnecessary, since the definition of struct A automatically creates a typedef named A.

¹³ In fact, C implementations often simply add a prefix like struct_ to a struct name and use the same symbol table, so the struct name A becomes struct_A and remains unique.

¹⁴ In Java, the different symbol tables for different kinds of definitions are called **namespaces**, not to be confused with the namespaces of C++.

7.5 Allocation, Lifetimes, and the Environment

Having considered the symbol table in some detail, we need also to study the environment, which maintains the bindings of names to locations. We will introduce the basics of environments without functions here, as a comparison to the symbol table, but defer the study of environments in the presence of functions and procedures until Chapter 10.

Depending on the language, the environment may be constructed statically (at load time), dynamically (at execution time), or with a mixture of the two. A language that uses a completely static environment is FORTRAN—all locations are bound statically. A language that uses a completely dynamic environment is LISP—all locations are bound during execution. C, C++, Ada, Java, and other Algol-style languages are in the middle—some allocation is performed statically, while other allocation is performed dynamically.

Not all names in a program are bound to locations. In a compiled language, names of constants and data types may represent purely compile-time quantities that have no existence at load or execution time. For example, the C global constant declaration

```
const int MAX = 10;
```

can be used by a compiler to replace all uses of MAX by the value 10. The name MAX is never allocated a location and indeed has disappeared altogether from the program when it executes.

Declarations are used to construct the environment as well as the symbol table. In a compiler, the declarations are used to indicate what allocation code the compiler is to generate as the declaration is processed. In an interpreter, the symbol table and the environment are combined, so attribute binding by a declaration in an interpreter includes the binding of locations.

Typically, in a block-structured language, global variables can be allocated statically, since their meanings are fixed throughout the program. Local variables, however, are allocated dynamically when execution reaches the block in question. In the last section you saw that, in a block-structured language, the symbol table uses a stack-like mechanism to maintain the bindings of the declarations. Similarly, the environment in a block-structured language binds locations to local variables in a stack-based fashion. To see how this works, consider the following C program fragment of Figure 7.27 with nested blocks.

```
(1) A: { int x;
(2)     char y;
(3)     ...
(4)   B: { double x;
(5)       int a;
(6)       ...
(7)   } /* end B */
(8)   C: { char y;
(9)       int b;
(10)      ...
(11)   D: { int x;
(12)        double y;
(13)      ...
```

Figure 7.27 A C program with nested blocks to demonstrate allocation by the environment (*continues*)

(continued)

```
(14)      } /* end D */
(15)      ...
(16)      } /* end C */
(17)      ...
(18)  } /* end A */
```

Figure 7.27 A C program with nested blocks to demonstrate allocation by the environment

When the code is executed and a block is entered, memory for the variables declared at the beginning of each block is allocated. When a block is exited, this memory is deallocated. If we view the environment as a linear sequence of storage locations, with locations allocated from the top in descending order, then the environment at line 3 of Figure 7.27 (after the entry into A) looks as shown in Figure 7.28 (ignoring the size of memory allocated for each variable).

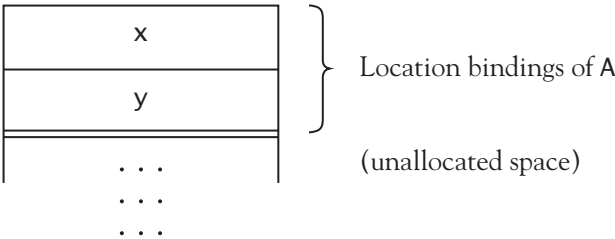


Figure 7.28 The environment at line 3 of Figure 7.27 after the entry into A

The environment after entry into B is shown in Figure 7.29.

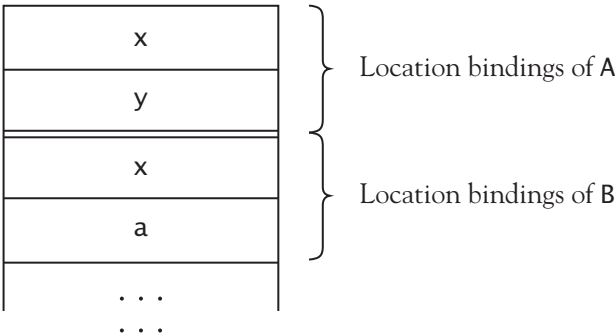


Figure 7.29 The environment at line 6 of Figure 7.27 after the entry into B

On exit from block B (line 7) the environment returns to the environment as it existed just after the entry into A. Then, when block C is entered (lines 8–9), memory for the variables of C is allocated and the environment is as shown in Figure 7.30.

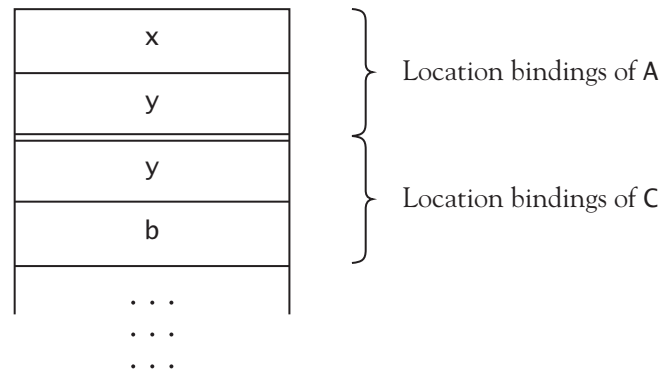


Figure 7.30 The environment at line 10 of Figure 7.27 after the entry into C

Notice that the variables *y* and *b* of block C are now allocated the same memory space that previously was allocated to the variables *x* and *a* of block B. This is okay, since we are now outside the scope of those variables, and they will never again be referenced.

Finally, on entry into block D (lines 11–12), the environment is as shown in Figure 7.31.

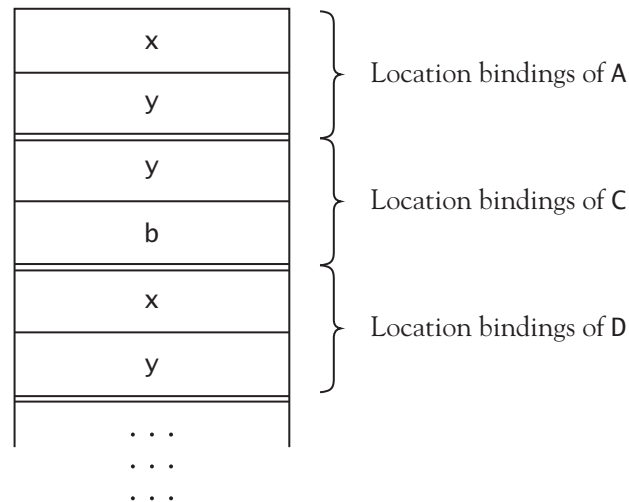


Figure 7.31 The environment at line 1 of Figure 7.27 after the entry into D

On exit from each block the location bindings of that block are successively deallocated, until, just prior to exit from block A, we have again recovered the original environment of A. In this way, the environment behaves like a stack. (Traditionally, environments are drawn upside down from the usual way stacks are depicted.)

This behavior of the environment in allocating and deallocating space for blocks is relatively simple. Procedure and function blocks are more complicated. Consider the following procedure declaration in C syntax:

```
void p(){
    int x;
    double y;
    ...
} /* p */
```

During execution, this declaration will not itself trigger an execution of the block of *p*, and the local variables *x* and *y* of *p* will not be allocated at the point of the declaration. Instead, memory for the variables *x* and *y* will be allocated only when *p* is called. Also, each time *p* is called, new memory for the local variables will be allocated. Thus, every call to *p* results in a region of memory being allocated in the environment. We refer to each call to *p* as an **activation** of *p* and the corresponding region of allocated memory as an **activation record**. A more complete description of the structure of activation records, and the information necessary to maintain them, is postponed to the discussion of procedures in Chapter 10.

It should be clear from these examples that, in a block-structured language with lexical scope, the same name may be associated with several different locations (though only one of these can be accessed at any one time). For instance, in the environment of the previous nested blocks example, the name *x* is bound to two different locations during the execution of block *D* (lines 11–14), and the name *y* is bound to three different locations (though only the *x* and *y* of *D* are accessible at that time). We must therefore distinguish among a name, an allocated location, and the declaration that causes them to be bound. We will call the allocated location an **object**. That is, an object is an area of storage that is allocated in the environment as a result of the processing of a declaration. According to this definition, variables and procedures in C are associated with objects, but global constants and data types are not (since type and global constant declarations do not result in storage allocation).¹⁵ The **lifetime** or **extent** of an object is the duration of its allocation in the environment. The lifetimes of objects can extend beyond the region of a program where they may be accessed. For example, in the previous block example, the declaration of integer *x* in block *A* defines an object whose lifetime extends through block *B*, even though the declaration has a scope hole in *B*, and the object is not accessible from inside *B*. Similarly, it is possible for the reverse to happen: As explained later in this chapter in Section 7.7, an object can be accessible beyond its lifetime.

When pointers are available in a language, a further extension of the structure of the environment is necessary. A **pointer** is an object whose stored value is a reference to another object.

In C, the processing of the declaration

```
int* x;
```

by the environment causes the allocation of a pointer variable *x*, but *not* the allocation of an object to which *x* points. Indeed, *x* may have an undefined value, which could be any arbitrary location in memory. To permit the initialization of pointers that do not point to an allocated object, and to allow a program to determine whether a pointer variable points to allocated memory, C allows the integer 0 to be used as an

¹⁵ This notion of object is not the same as that in object-oriented programming languages; see Chapter 5.

address that cannot be the address of any allocated object, and various C libraries give the name `NULL` to 0, so that one can write in C:

```
int* x = NULL;
```

Other languages generally reserve a special symbol for this address (`null` in Java and Ada, `nil` in Pascal), so that it cannot be used as an identifier.¹⁶ With this initialization, `x` can be tested to see if it points to an allocated object:

```
if (x != NULL) *x = 2;
```

For `x` to point to an allocated object, we must manually allocate it by the use of an allocation routine. Again, C is unusual in that there is no built-in allocation routine specified by the language, but the `stdlib` library module contains several functions to allocate and deallocate memory. The most commonly used of these are the `malloc` (for *memory allocation*) and `free` functions. Thus,

```
x = (int*) malloc(sizeof(int));
```

allocates a new integer variable and at the same time assigns its location to be the value of `x`. The `malloc` function returns the location it allocates and must be given the size of the data for which it is to allocate space. This can be given in implementation-independent form using the built-in `sizeof` function, which is given a data type and returns its (implementation-dependent) size. The `malloc` function must also be cast to the data type of the variable to which its result is being assigned, by putting the data type in parentheses before the function, since the address it returns is an *anonymous* pointer (a `void*` in C). (Casts and anonymous pointers are explained in Chapter 8.)

Once `x` has been assigned the address of an allocated integer variable, this new integer object can be accessed by using the expression `*x`. The variable `x` is said to be **dereferenced** using the unary `*` operator.¹⁷ We can then assign integer values to `*x` and refer to those values as we would with an ordinary variable, as in:

```
*x = 2;
printf("%d\n", *x);
```

`*x` can also be deallocated by calling the `free` procedure, as in:

```
free(x);
```

(Note that dereferencing is not used here, although it is `*x` that is being freed, not `x` itself.)

C++ simplifies the dynamic allocation of C by adding built-in operators `new` and `delete`, which are reserved names. Thus, the previous code in C++ would become:

```
int* x = new int; // C++
*x = 2;
cout << *x << endl; // output in C++
delete x;
```

¹⁶ Though technically, `null` is a literal, in practice there is little difference between this literal use and that of a reserved word.

¹⁷ While this use of `*` is made to look similar to the way `x` is defined, this is an entirely different use of the `*` symbol from its purpose in a declaration.

Note the special syntax in this code—`new` and `delete` are used as unary operators, rather than functions, since no parentheses are required. Pascal is similar to C++, only the `delete` procedure is called `dispose`.

To allow for arbitrary allocation and deallocation using `new` and `delete` (or `malloc` and `free`), the environment must have an area in memory from which locations can be allocated in response to calls to `new`, and to which locations can be returned in response to calls to `delete`. Such an area is traditionally called a **heap** (although it has nothing to do with the heap data structure). Allocation on the heap is usually referred to as **dynamic allocation**, even though allocation of local variables is also dynamic, as we have seen. To distinguish these two forms of dynamic allocation, allocation of local variables according to the stack-based scheme described earlier is sometimes called **stack-based** or **automatic**, since it occurs automatically under control of the runtime system. (A more appropriate term for pointer allocation using `new` and `delete` might be *manual* allocation, since it occurs under programmer control.)

In a typical implementation of the environment, the stack (for automatic allocation) and the heap (for dynamic allocation) are kept in different sections of memory, and global variables are also allocated in a separate, static area. Although these three memory sections could be placed anywhere in memory, a common strategy is to place the three adjacent to each other, with the global area first, the stack next, and the heap last, with the heap and stack growing in opposite directions (to avoid stack/heap collisions in case there is no fixed boundary between them). This is usually depicted as shown in Figure 7.32.

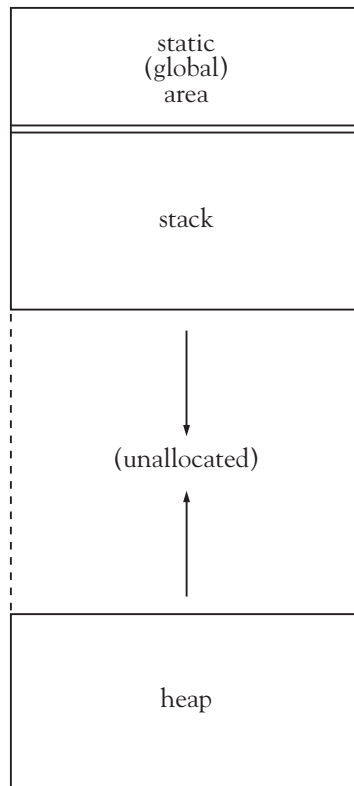


Figure 7.32 Structure of a typical environment with a stack and a heap

Note that, although the heap is shown as growing in only one direction, storage can be released anywhere in the allocated part of the heap, and that storage needs to be reusable. Thus, a simple stack protocol will not work for the heap. Chapter 10 discusses this question in more detail.

A further complication in the management of the heap is that many languages require that heap deallocation be managed automatically (like the stack, except that allocation may still be under user control). For example, virtually all functional languages require that the heap be completely automatic, with no allocation or deallocation under programmer control. Java, on the other hand, places allocation, but not deallocation, under programmer control. Java also further restricts heap allocation to objects only, that is, variables of reference types. Thus, the previous example would look as follows in Java:

```
// in Java must use class Integer, not int
// also, cannot allocate without assigning a value
Integer x = new Integer(2);
// print the actual integer value of x, i.e. 2
System.out.println(x);
// no delete operation allowed
```

Note how this code also does not have any explicit dereferencing (such as `*x` in C); Java forbids the use of such dereferencing—indeed, it is not even part of Java’s syntax, as there is no “dereference” operator.

Languages like Java and the functional languages, such as Scheme, ML, and Haskell, do not allow much programmer control over heap allocation and deallocation, nor explicit pointer manipulation, such as dereferencing. The reason for this is that these are all inherently unsafe operations and can introduce seriously faulty runtime behavior that is very difficult to analyze and fix (as any C or C++ programmer can attest). Moreover, such faulty program behavior can, if extreme enough, compromise the entire operating system of a computer and cause an entire system or network to fail, or, worse, to act in malicious ways. For example, one entry point for computer viruses has traditionally been the unsafe access to addresses allowed by explicit pointer references in programs. This inherent lack of safety for heap allocation is studied further in Section 7.7.

One final, somewhat unusual, example of a language mechanism for heap allocation is that of Ada. Ada also has a `new` operation but no corresponding `delete` operation, similar to Java. However, Ada allows a `delete` operation to be user-defined using a standard language utility called `Ada.Unchecked_Deallocation`. This makes it more difficult for the programmer to use, as well as alerting anyone reading the code that this code is potentially unsafe, while at the same time allowing explicit programmer control of this feature as in C/C++. Figure 7.33 shows the preceding Java example rewritten in Ada (in its own block, to provide for the necessary declarations).

```
(1) declare
(2)   type Intptr is access Integer;
(3)   procedure delete_int is
(4)     new Ada.Unchecked_Deallocation(Integer, Intptr);
(5)     -- generic procedure instantiation in Ada
```

Figure 7.33 Ada code showing manual dynamic deallocation (*continues*)

(continued)

```
-- see Chapter 8
(6)  x: Intptr := new Integer;
(7)  begin
(8)  x.all := 2;
(9)  put(x.all); new_line;
(10) delete_int(x);
(11) end;
```

Figure 7.33 Ada code showing manual dynamic deallocation

Note how the keyword `access` is used in Figure 7.33 (line 2) to indicate a pointer, and the dereference operator (line 8 of Figure 7.33) is `.all`. This comes from the expectation of Ada (as with Java) that heap allocation will be used primarily for record (class) structures. The `.all` refers to *all fields* (in our case, there are no fields, only an integer, but one must still use the dereferencer). Unlike Java, Ada does not require `x` to be a record, however.

To summarize, in a block-structured language with heap allocation, there are three kinds of allocation in the environment: static (for global variables), automatic (for local variables), and dynamic (for heap allocation). These categories are also referred to as the **storage class** of the variable. Some languages, such as C, allow a declaration to specify a storage class as well as a data type. Typically, this is used in C to change the allocation of a local variable to static:

```
int f(){
  static int x;
  ...
}
```

Now `x` is allocated only once, and it has the same meaning (and value) in all calls to `f`. This combination of local scope with global lifetime can be used to preserve local state across calls to `f`, while at the same time preventing code outside the function from changing this state. For example, Figure 7.34 shows the code for a function in C that returns the number of times it has been called, together with some interesting main program code.

```
(1) int p(){
(2)     static int p_count = 0;
(3)     /* initialized only once - not each call! */
(4)     p_count += 1;
(5)     return p_count;
(6) }

(7) main(){
(8)     int i;
```

Figure 7.34 C program demonstrating the use of a local static variable (continues)

(continued)

```
(9)    for (i = 0; i < 10; i++){
(10)    if (p() % 3) printf("%d\n",p());
(11)    }
(12)    return 0;
(13) }
```

Figure 7.34 C program demonstrating the use of a local static variable

7.6 Variables and Constants

Although references to variables and constants look the same in many programming languages, their roles and semantics in a program are very different. This section clarifies and distinguishes their basic semantics.

7.6.1 Variables

A **variable** is an object whose stored value can change during execution. A variable can be thought of as being completely specified by its attributes, which include its name, its location, its value, and other attributes such as data type and size of memory storage. A schematic representation of a variable can be drawn as shown in Figure 7.35.

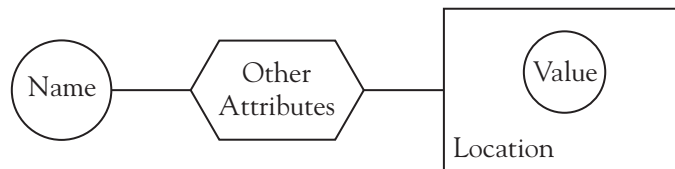


Figure 7.35 Schematic representation of a variable and its attributes

This picture singles out the name, location, and value of a variable as being its principal attributes. Often we will want to concentrate on these alone, and then we picture a variable in the diagram shown in Figure 7.36, which is known as a **box-and-circle diagram**.

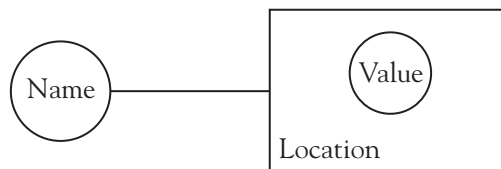


Figure 7.36 Schematic representation of a variable, its value, and its location

In a box-and-circle diagram, the line between the name and the location box represents the binding of the name to the location by the environment. The circle inside the box represents the value bound by the memory—that is, the value stored at that location.

The principal way a variable changes its value is through the **assignment** statement $x = e$, where x is a variable name and e is an expression. The semantics of this statement are that e is evaluated to a value, which is then copied into the location of x . If e is a variable name, say, y , then the assignment (in C syntax):

$$x = y$$

can be viewed as shown in Figure 7.37, with the double arrow indicating copying.

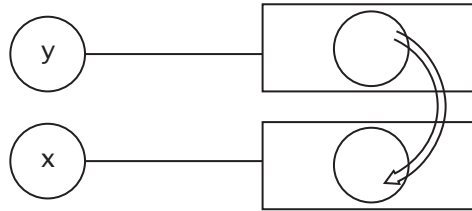


Figure 7.37 Assignment with copying of values

Since a variable has both a location and a value stored at that location, it is important to distinguish clearly between the two. However, this distinction is obscured in the assignment statement, in which y on the right-hand side stands for the value of y and x on the left-hand side stands for the location of x . For this reason, the value stored in the location of a variable is sometimes called its **r-value** (for right-hand side value), while the location of a variable is its **l-value** (for left-hand side value). A few languages make this distinction explicit. ML is one modern language that does so. For example, in ML, variables are explicitly thought of as locations, or references to values. If x is an integer variable in ML, then its type is “reference to integer,” and to obtain its value we must write $!x$, which dereferences x to produce its value. Thus, to increment x we must write in ML:

$$x := !x + 1;$$

and the assignment of the value of the variable y to x would be written in ML as:

$$x := !y;$$

In C, in addition to the standard automatic dereferencing of l-values to r-values, there is an explicit **address of operator** $\&$ that turns a reference into a pointer. This allows the address of a variable to be explicitly fetched as a pointer (which can then be dereferenced using the standard $*$ operator on pointers). Thus, given the C declaration

```
int x;
```

$\&x$ is a pointer to x (the “address” of x) and $*\&x$ is again x itself (both as an l-value and an r-value; see Exercise 7.24). Also, C allows the mixing of expressions with assignments, where both the r-value and l-value of a variable are involved. Thus, to increment the integer x by 1, one can write either $x = x + 1$ or $x += 1$ (indicating that x is both added to 1 and reassigned). The mixing of r-values, l-values, and pointers in C can lead to confusing and unsafe situations. For example,

$$x + 1 = 2;$$

is illegal ($x + 1$ is not an l-value), but

```
*(&x + 1) = 2;
```

is legal. In this case, $\&x$ is a pointer, to which 1 is added using address arithmetic, and then 2 is stored at the resulting address; as you can imagine, this is unsafe!

These features of C—the mixing of references and values, the use of the $\&$ operator, and the distinction between a reference and a pointer—all make C vulnerable to extremely obscure and even dangerous or improper use of variables, many of which cannot easily be caught either during translation or execution. They are, however, a consequence of C's design goal as a fast, unsafe language for systems programming.

Ada95 also has an “address-of” operator: If x has an l-value, then x' access is a pointer to x (in Ada pointer types are called **access types**).¹⁸ Ada95 applies strict rules, however, that limit the use of these types in ways that make it much more difficult to misuse them than in C. This is discussed in further detail in Chapter 10.

In some languages, assignment has a different meaning. Locations are copied instead of values. In this form of assignment, called **assignment by sharing**, case $x = y$ has the result of binding the location of y to x instead of its value, as shown in Figure 7.38.

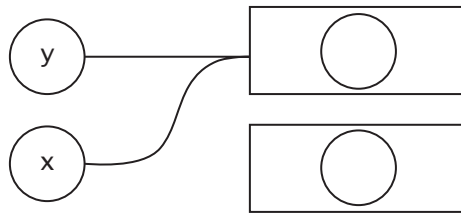


Figure 7.38 The result of assignment by sharing

An alternative, called **assignment by cloning**, is to allocate a new location, copy the value of y , and then bind x to the new location, as shown in Figure 7.39.¹⁹

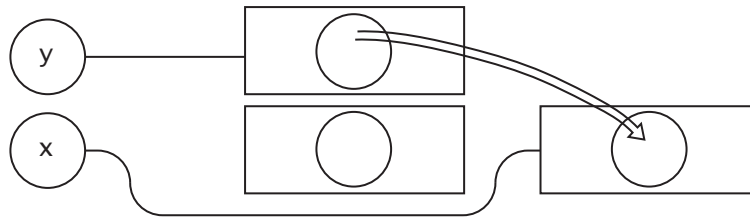


Figure 7.39 The result of assignment by cloning

In both cases this interpretation of assignment is sometimes referred to as **pointer semantics** or **reference semantics** to distinguish it from the more usual semantics, which is sometimes referred to

¹⁸ The apostrophe is used in Ada to fetch various attributes of program entities; thus, x' access fetches the **access** attribute of the variable x , that is, the address of x as a pointer.

¹⁹ A *clone* is an exact copy of an object in memory.

as **storage semantics** or **value semantics**. Java, for example, uses assignment by sharing for all object variables, but not for simple data; see Section 7.7 for an example. Other languages that use reference semantics are Smalltalk, LISP, and Python.

Figure 7.40 shows assignment by sharing, with the name *x* being associated directly to a new location (that of *y*). This is difficult to achieve for a compiler, since the symbol table commonly does not exist during execution. Standard implementations of assignment by sharing (such as in Java) use pointers and implicit dereferencing. In this view of assignment by sharing, *x* and *y* are implicitly pointers (with implicit automatically allocated pointer values).

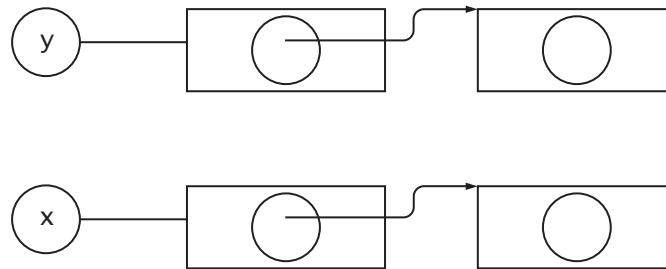


Figure 7.40 Variables as implicit references to objects

Then the assignment $x = y$ has the effect shown in Figure 7.41. Note that this is the same as if we wrote $x = y$ for pointer variables *x* and *y* in C.

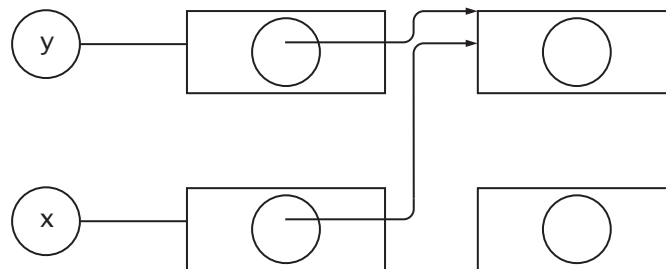


Figure 7.41 Assignment by sharing of references

However, in the absence of address-of and dereference operators (& and * in C), this implementation is indistinguishable from the original picture. (See Exercise 7.25 and Section 7.7.1.)

7.6.2 Constants

A **constant** is a language entity that has a fixed value for the duration of its existence in a program. As shown in Figure 7.42, a constant is like a variable, except that it has no location attribute, but a value only.

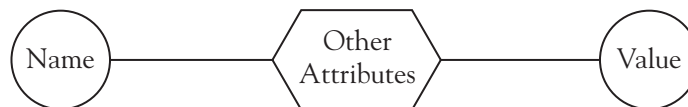


Figure 7.42 A constant and its attributes

We sometimes say that a constant has **value semantics** instead of the storage semantics of a variable. This does not mean that a constant is not stored in memory. It is possible for a constant to have a value that is known only at execution time. In this case, its value must be stored in memory, but, unlike a variable, once this value is computed, it cannot change, and the location of the constant cannot be explicitly referred to by a program.

This notion of a constant is **symbolic**; that is, a constant is essentially a name for a value. Sometimes representations of values, like the sequence of digits 42 or the representation of a character such as “a,” are called constants. To distinguish them from the constants in a constant declaration, we sometimes refer to these representations of values as **literals**.

Many languages—especially the functional languages—emphasize the use of constants over variables because of their much simpler semantics: Each constant name stands for only one value, and, once the value is created, it remains the same regardless of the position or use of the name within the code. Some languages, such as Haskell, even prohibit variables altogether (the way some languages prohibit goto’s) and rely entirely on constants for computations. (For more on this, see Chapter 3.)

As we have noted, constants can be static or dynamic. A static constant is one whose value can be computed prior to execution, while a dynamic constant has a value that can be computed only during execution. Static constants may also be of two kinds: Some constants can be computed at translation time (compile time), while others can be computed at load time (like the static location of a global variable), or right at the beginning of the execution of the program. This distinction is important, since a compile-time constant can, in principle, be used by a compiler to improve the efficiency of a program and need not actually occupy memory, while a load-time or dynamic constant must be computed either on startup or as execution proceeds and must be stored in memory. Unfortunately, the word “static” is used sometimes to refer to compile-time constants and sometimes to load-time constants. We shall try to be precise in the following description by referring to static translation-time constants as **compile-time constants** (even though the language may be interpreted rather than compiled), and we shall restrict the use of the term **static constant** to load-time constants. We can also make a distinction between general constants and **manifest constants**: A manifest constant is a name for a literal, while a constant can be more general.

Consider, for example, the following C code:

```
#include <stdio.h>
#include <time.h>

const int a = 2;
const int b = 27+2*2;
/* warning - illegal C code! */
const int c = (int) time(0);

int f( int x){
    const int d = x+1;
    return b+c;
}
...
```

In this code, `a` and `b` are compile-time constants (`a` is a manifest constant), while `c` is a static (load-time) constant, and `d` is a dynamic constant. In C, the `const` attribute can be applied to any variable in a program and simply indicates that the value of a variable, once set, cannot be changed; other criteria determine whether a variable is static or not (such as the global scope in which `a`, `b`, and `c` are defined above). In C, however, load-time constants cannot be defined because of a separate rule restricting initial values of global variables to a narrow subset of compile-time expressions. Thus, in C, we cannot even write:

```
const int a = 2;
int b = 27 + a * a; /* also illegal in C */
```

in the preceding program, because the initial value of `b` cannot be computed from the constant `a`. Instead, it must be computed from literals only (with a few small exceptions). However, in C++, which removes these restrictions, the preceding program is legal.

Ada is similar to C++ in that a constant declaration such as:

```
time : constant integer := integer(seconds(clock));
```

may appear anywhere in a program (and is static if global, and dynamic if local).

Java is similar to C++ and Ada, with two exceptions. First, in Java, a constant is indicated by using the keyword `final` (indicating that it gets only one, or a “final” value). Second, to get a static constant, one must use the keyword `static` (Java is structured so that there are no global variables as such). For example, the complete Java program in Figure 7.43 prints out the date and time at the moment when the program begins to run.

```
import java.util.Date;

class PrintDate{
    public static void main(String[] args){
        System.out.println(now);
    }
    static final Date now = new Date();
}
```

Figure 7.43 Java program demonstrating a load-time (static final) constant variable

One issue often suppressed or overlooked in language discussions is that function definitions in virtually all languages are definitions of constants whose values are functions. Indeed, a C definition such as:

```
int gcd( int u, int v){
    if (v == 0) return u;
    else return gcd(v, u % v);
}
```

defines the name `gcd` to be a (compile-time) constant whose value is a function with two integer parameters, returning an integer result, and whose operation is given by the code of its body. Notice how this

definition differs from that of a function *variable* in C, to which we could assign the value of `gcd`, as given in Figure 7.44.

```
(1) int (*fun_var)(int,int) = gcd;

(2) main(){
(3)     printf("%d\n", fun_var(15,10));
(4)     return 0;
(5) }
```

Figure 7.44 C code showing the use of a function variable

C is a little inconsistent in the code of Figure 7.44, in that we must define a function variable like `fun_var` as a pointer²⁰, but when assigning it a value and calling it, use of pointer syntax is not required. C also has no way of writing a function *literal* (that is, a function value, without giving it a name such as `gcd`). As we would expect, functional languages do a much better job of clarifying the distinction between function constants, function variables, and function literals (also called anonymous functions). For example, in ML, a function literal (in this case the integer square function) can be written as:

```
fn(x:int) => x * x
```

The square function can also be defined as a function constant (a constant is a `val` in ML) as:

```
val square = fn(x:int) => x * x;
```

as well as in more traditional C-like syntax:

```
fun square (x: int) = x * x;
```

An unnamed function literal can also be used directly in expressions without ever giving it a name. For example,

```
(fn(x:int) => x * x) 2;
```

evaluates the (unnamed) “square” function, passes it the value 2, and returns the value 4:

```
val it = 4 : int
```

Similar syntax is possible in Scheme and Haskell (see Chapter 3).

7.7 Aliases, Dangling References, and Garbage

This section describes several of the problems that arise with the naming and dynamic allocation conventions of programming languages, particularly the conventional block-structured languages C, C++, and Ada. As a programmer, you can learn how to avoid these problematic situations. As a language designer, you can build solutions to these problems into your language, so that the programmer doesn’t have to be concerned with them at all (Java programmers are good examples of this). A few of these language design solutions are discussed in this section.

²⁰ This is necessary because without the pointer syntax this would be a function declaration or prototype.

7.7.1 Aliases

An **alias** occurs when the same object is bound to two different names at the same time. Aliasing can occur in several ways. One is during procedure call and is studied in Chapter 10. Another is through the use of pointer variables. A simple example in C is given by the following code:

```
(1) int *x, *y;
(2) x = (int *) malloc(sizeof(int));
(3) *x = 1;
(4) y = x;    /* *x and *y now aliases */
(5) *y = 2;
(6) printf("%d\n", *x);
```

After the assignment of x to y (line 4), $*y$ and $*x$ both refer to the same variable, and the preceding code prints 2. We can see this clearly if we record the effect of the preceding code in our box-and-circle diagrams of Section 7.6, as follows.

After the declarations (line 1), both x and y have been allocated in the environment, but the values of both are undefined. We indicate that in Figure 7.45 by shading in the circles indicating values.

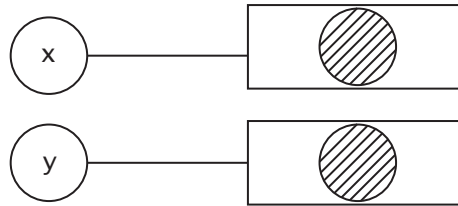


Figure 7.45 Allocation of storage for pointers x and y

After line 2, $*x$ has been allocated, and x has been assigned a value equal to the location of $*x$, but $*x$ is still undefined. See Figure 7.46.

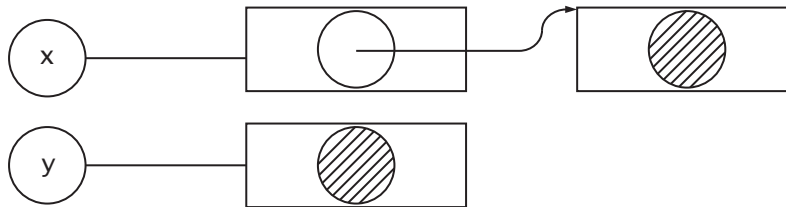


Figure 7.46 Allocation of storage for $*x$

After line 3, the situation is as shown in Figure 7.47.

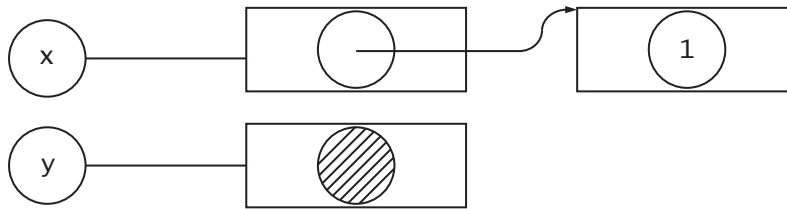


Figure 7.47 Result of $*x = 1$

Line 4 now copies the value of x to y, and so makes $*y$ and $*x$ aliases of each other. (Note that x and y are not aliases of each other.) See Figure 7.48.

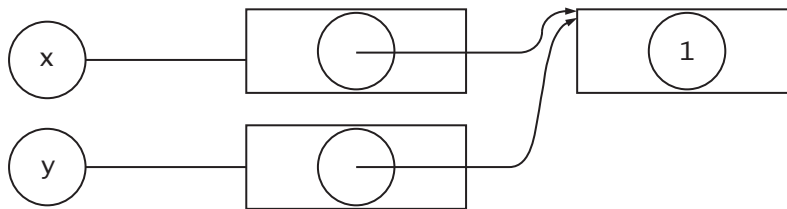


Figure 7.48 Result of $y = x$

Finally, line 5 results in the diagram shown in Figure 7.49.

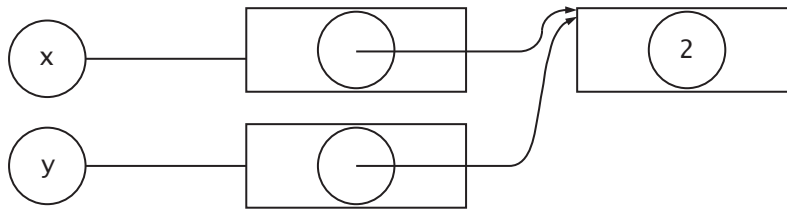


Figure 7.49 Result of $*y = 2$

Aliases present a problem in that they cause potentially harmful **side effects**. For our purposes, we define a side effect of a statement to be any change in the value of a variable that persists beyond the execution of the statement.²¹ According to this definition, side effects are not all harmful, since an assignment is explicitly intended to cause one. However, side effects that are changes to variables whose names do not directly appear in the statement are potentially harmful in that the side effect cannot be determined from the written code. In the preceding example, the assignment $*y = 2$ also changed $*x$, even though no hint of that change appears in the statement that causes it (line 5). We must instead read the earlier statements in this code to discover that this is happening.

Aliasing due to pointer assignment is difficult to control and is one of the reasons that programming with pointers is so difficult. One language that does attempt to limit aliasing, not only by pointers, but throughout the language, is Euclid. (For more on this, see the Notes and References at the end of this chapter.)

²¹ Other definitions of side effect exist; see Exercise 7.23.

A third way that aliases can be created is through assignment by sharing, as described in the previous section, since assignment by sharing implicitly uses pointers. Java is a major example of this kind of aliasing. Consider the following Java program:

```
(1) class ArrTest{
(2)     public static void main(String[] args){
(3)         int[] x = {1,2,3};
(4)         int[] y = x;
(5)         x[0] = 42;
(6)         System.out.println(y[0]);
(7)     }
(8) }
```

In line 3, an array named `x` is created with size 3, such that `x[0] = 1`, `x[1] = 2`, and `x[2] = 3`. In line 4, another array `y` is defined and initialized with `x`. In line 5, `x[0]` is changed to 42, and in line 6, `y[0]` is printed, which is now also 42, because of assignment by sharing.

Because of this aliasing problem in Java, Java has a mechanism for explicitly **cloning** any object, so that aliases are not created by assignment. For example, if we replace line 4 in the preceding code with:

```
int[] y = (int[]) x.clone();
```

then the value printed in line 6 is 1, as we might reasonably expect.

7.7.2 Dangling References

Dangling references are a second problem that can arise from the use of pointers. A dangling reference is a location that has been deallocated from the environment but that can still be accessed by a program. In other words, a dangling reference occurs if an object can be accessed beyond its lifetime in the environment.

A simple example of a dangling reference is a pointer that points to a deallocated object. In C, the use of the `free` procedure can cause a dangling reference, as follows:

```
int *x , *y;
...
x = (int *) malloc(sizeof(int));
...
*x = 2;
...
y = x; /* *y and *x now aliases */
free(x); /* *y now a dangling reference */
...
printf("%d\n", *y); /* illegal reference */
```


In C, it is also possible for dangling references to result from the automatic deallocation of local variables when the block of the local declaration is exited. This is because, as noted previously, C has the address of operator `&` that allows the location of any variable to be assigned to a pointer variable. Consider the following C code:

```
(1) { int * x;
(2)   { int y;
(3)     y = 2;
(4)     x = &y;
(5)   }
(6)   /* *x is now a dangling reference */
(7) }
```

At line 5, when we exit the block in which `y` is declared, the variable `x` contains the location of `y` and the variable `*x` is an alias of `y`. However, in the standard stack-based environment we described in Section 7.5, `y` was deallocated on exiting from the block. A similar example is the following C code:

```
int * dangle(){
    int x;
    return &x;
}
```

Whenever function `dangle` is called, it returns the location of its local automatic variable `x`, which has just been deallocated. Thus, after any assignment such as `y = dangle()` the variable `*y` will be a dangling reference.

Ada does not permit this kind of dangling reference, since it has no equivalent to the `&` operator of C. Ada *does* allow the first kind of dangling reference, but only if the program uses the `Unchecked_Deallocation` package (see Section 7.5, Figure 7.33).

Java does not allow dangling references at all, since there are no explicit pointers in the language, no address of operator, and no memory deallocation operators such as `free` or `delete`.

7.7.3 Garbage

One easy way to eliminate the dangling reference problem is to prevent any deallocation from the environment. This causes the third problem that we discuss in this section, namely, garbage. Garbage is memory that has been allocated in the environment but that has become inaccessible to the program.

A typical way for garbage to occur in C is to fail to call `free` before reassigning a pointer variable:

```
int *x;
...
x = (int *) malloc(sizeof(int));
x = 0;
```

At the end of this code, the location allocated to `*x` by the call to `malloc` is now garbage, since `x` now contains the null pointer, and there is no way to access the previously allocated object. A similar situation occurs when execution leaves the region of the program in which `x` itself is allocated, as in:

```
void p(){
    int *x;
    x = (int *) malloc(sizeof(int));
    *x = 2;
}
```

When procedure `p` is exited, the variable `x` is deallocated and `*x` is no longer accessible by the program. A similar situation occurs in nested blocks.

Garbage is a problem in program execution because it is wasted memory. However, an argument can be made that programs that produce garbage are less seriously flawed than programs that contain dangling references. A program that is internally correct but that produces garbage may fail to run because it runs out of memory. That is, if it does not exceed available memory, it will produce correct results (or at least not be incorrect because of the failure to deallocate inaccessible memory). A program that accesses dangling references, on the other hand, may run but produce incorrect results, may corrupt other programs in memory, or may cause runtime errors that are hard to locate.

For this reason it is useful to remove the need to deallocate memory explicitly from the programmer (which, if done incorrectly, can cause dangling references), while at the same time automatically reclaiming garbage for further use. Language systems that automatically reclaim garbage are said to perform **garbage collection**.

We should note that the stack-based management of memory in the environment of a block-structured language can already be called a kind of simple garbage collection: When the scope of an automatic variable declaration is exited, the environment reclaims the location allocated to that variable by popping from the stack the memory allocated for the variable.

Historically, functional language systems, particularly LISP systems, pioneered garbage collection as a method for managing the runtime deallocation of memory. Indeed, in LISP, all allocation as well as deallocation is performed automatically. As a result, as we have previously noted, functional languages have no explicit pointers or operations like `malloc` and `free`. Object-oriented language systems also often rely on garbage collectors for the reclamation of memory during program execution. This is the case for Smalltalk and Java. C++ is the notable exception.²²

Language design is a key factor in what kind of runtime environment is necessary for the correct execution of programs. Nevertheless, the language design itself may not explicitly state what kind of memory allocation is required. For example, the definition of Algol60 introduced block structure, and so implicitly advocated the use of a stack-based environment, without explicitly describing or requiring it. Definitions of LISP have also not mentioned garbage collection, even though a typical LISP system cannot execute correctly without it.

²² The C++ Standard does allow for nonstandard heap managers, however, and the `new` and `delete` operators can be overloaded (for example, `delete` may be redefined to do nothing at all). However, all such redefinitions are nonstandard and currently not well supported by typical C++ implementations.

One way for the designer of an Algol-like language to indicate the need for automatic garbage collection would be to include in the language definition a new procedure for pointers but fail to include a corresponding `free` or `delete` procedure. Java takes this approach, as we have noted, and so does Ada, except that Ada allows the manual introduction of such a procedure using `Unchecked_Deallocation`. A further quirk of Ada is that, even with the use of `Unchecked_Deallocation`, a garbage collector may be called if a pointer is not correctly deallocated. Since one of Ada's key design goals was to be used in real-time situations where control of the speed of execution of a program is critical, Ada provides a further option to turn off any existing garbage collectors for variables of given data types:

```
-- Intptr defined as in Figure 7.33
Pragma CONTROLLED(Intptr);
```

7.8 Case Study: Initial Static Semantic Analysis of TinyAda

Section 6.8 in the previous chapter introduced a syntax analyzer for the little language TinyAda. That program took the form of a simple parsing shell that pulled tokens from a scanner until a syntax error was detected or the source program was recognized as syntactically correct. In this section, we discuss extending the parsing shell to perform some semantic analysis. In particular, we focus on designing and using tools for scope analysis and for restricting the use of identifiers in a program. This will involve a focus on two of the attributes of an identifier discussed earlier in this chapter: its name and the role it plays in a program (constant, variable, type, or procedure).

7.8.1 Scope Analysis

TinyAda is lexically or statically scoped. Here are the scope rules of the language:

1. All identifiers must be declared before they are used or referenced.
2. There may be at most one declaration of a given identifier in a single block.
3. In TinyAda, a new block begins with the formal parameter specifications of a procedure and extends to the reserved word `end` belonging to that procedure.
4. The visibility of a declared identifier extends into nested blocks, unless that identifier is redeclared in a nested block.
5. Identifiers are not case sensitive.

TinyAda has five built-in, or predefined, identifiers. They are the data type names `integer`, `char`, and `boolean`, and the Boolean constants `true` and `false`. Obviously, these identifiers must already be visible in a top-level scope before a source program is parsed. The static nesting level of this scope is, therefore, 0. Also entered into the level 0 scope is the name of the single top-level procedure. Nested within the level 0 scope, at level 1, is a new scope that contains the procedure's formal parameters, if any, and any identifiers introduced in the procedure's basic declarations. The introduction of new scopes for the names declared in nested procedures follows the same pattern.

As mentioned earlier in this chapter, scope analysis involves entering information into a symbol table when identifier declarations are encountered, and looking up this information in the symbol table

when identifier references are encountered. To handle the nested scopes in a TinyAda source program, the parser can use a stack of symbol tables. This stack is initialized to empty at program startup. Before parsing commences, a new table for scope level 0 is pushed onto the stack, and the information for the built-in identifiers is entered into it. This process is repeated for the programmer-defined identifiers when each new scope is entered. When a scope is exited, at the end of a procedure declaration, the table at the top of the stack is popped. When an identifier reference is encountered, the parser searches for that identifier’s information in the topmost table in the stack, proceeding down through its tables until the information is found or the bottom of the stack is reached. The parser can now detect two scope errors. First, if a newly declared identifier is already present in the top-level table, the error is an identifier already declared in the same block. Second, if an identifier is referenced in the source program and cannot be found anywhere in the stack of tables, the error is an undeclared identifier.

We define two new classes to support scope analysis. The first class, called `SymbolEntry`, holds information about an identifier, such as its name and other attributes to be considered later in semantic analysis. The second class, called `SymbolTable`, manages the stack of scopes just discussed. This class handles the entry into and exit from a scope, the entry and lookup of identifiers, and the error detection process. The interface for the `SymbolTable` class is shown in Table 7.1. Its implementation is available on this book’s Web site.

Table 7.1 The interface for the <code>SymbolTable</code> class	
SymbolTable Method	What It Does
<code>SymbolTable(Chario c)</code>	Creates an empty stack of tables, with a reference to a <code>Chario</code> object for the output of error messages.
<code>void enterScope()</code>	Pushes a new table onto the stack.
<code>void exitScope();</code>	Pops a table from the stack and prints its contents.
<code>SymbolEntry enterSymbol(String name);</code>	If <code>name</code> is not already present, inserts an entry for it into the table and returns that entry; otherwise, prints an error message and returns an empty entry.
<code>SymbolEntry findSymbol(String name);</code>	If <code>name</code> is already present, returns its entry; otherwise, prints an error message and returns an empty entry.

Note that the parser does not halt when a scope error occurs. It is generally much easier to recover from semantic errors than from syntax errors.

The parser defines a new method for creating the symbol table and loading it with TinyAda’s built-in identifiers, as follows:

```
private void initTable(){
    table = new SymbolTable(chario);    // table is a new instance variable
    table.enterScope();
    table.enterSymbol("BOOLEAN");
    table.enterSymbol("CHAR");
    table.enterSymbol("INTEGER");
    table.enterSymbol("TRUE");
    table.enterSymbol("FALSE");
}
```

To facilitate the processing of identifiers, the `Parser` class defines two other new methods, `enterId` and `findId`. These methods are now called in all of the places in the grammar rules where the parser expects to consume identifiers. Each method first checks the token's code to determine that it is in fact an identifier, and, if it is not, registers a fatal error (this is no change from the raw syntax analyzer of Chapter 6). Otherwise, the method enters or looks up the identifier in the symbol table and then consumes the token. Here is the code for these two `Parser` methods:

```
private SymbolEntry enterId(){
    SymbolEntry entry = null;
    if (token.code == Token.ID)
        entry = table.enterSymbol(token.string);
    else
        fatalError("identifier expected");
    token = scanner.nextToken();
    return entry;
}

private SymbolEntry findId(){
    SymbolEntry entry = null;;
    if (token.code == Token.ID)
        entry = table.findSymbol(token.string);
    else
        fatalError("identifier expected");
    token = scanner.nextToken();
    return entry;
}
```

We conclude this subsection with some examples of the use of each new scope analysis method. The `Parser` methods `subprogramBody` and `subprogramSpecification` collaborate to process a TinyAda procedure declaration. Toward the end of `subprogramBody`, the parser must exit the local scope of the TinyAda procedure. The parser might also encounter an optional procedure identifier at the end of this phrase, so the identifier must be found in the outer scope just restored. Here is the revised code for `subprogramBody`:

```
/*
subprogramBody =
    subprogramSpecification "is"
    declarativePart
    "begin" sequenceOfStatements
    "end" [ <procedure>identifier ] ";"
*/
private void subprogramBody(){
    subprogramSpecification();
```

(continues)

(continued)

```

    accept(Token.IS, "'is' expected");
    declarativePart();
    accept(Token.BEGIN, "'begin' expected");
    sequenceOfStatements();
    accept(Token.END, "'end' expected");
    table.exitScope();                // The nested scope ends here
    if (token.code == Token.ID)       // Look up procedure's ID
        findId();
    accept(Token.SEMI, "semicolon expected");
}

```

Our last example method, `subprogramSpecification`, processes a TinyAda procedure header. This method should see the declaration of the procedure name, which is entered into the current scope. The method might also see formal parameter declarations, so it enters a new scope, as shown in the next definition.

```

/*
subprogramSpecification = "procedure" identifier [ formalPart ]
*/
private void subprogramSpecification(){
    accept(Token.PROC, "'procedure' expected");
    enterId();
    table.enterScope();                // Nested scope begins here
    if (token.code == Token.L_PAR)
        formalPart();
}

```

The scope entered just before the call of `formalPart` remains in effect for the other local declarations further down in the procedure declaration. The remaining modifications to the parsing methods are left as an exercise for the reader.

Note that the uses of the methods `enterId` and `findId` in the example throw away the `SymbolEntry` object returned by these methods. The parser does not really need this object for scope analysis. However, we will show how to use it to perform further semantic checks in the following section and in later chapters.

7.8.2 Identifier Role Analysis

In most high-level languages, an identifier names some entity, such as a variable, a constant, or an entire data type. We call this attribute of an identifier its **role**. You should not confuse an identifier's role with its value or its data type, both of which are additional attributes. Indeed, the roles of TinyAda identifiers appear among the semantic qualifiers (in angle brackets) in the EBNF grammar of Section 6.8. The role of an identifier imposes certain restrictions on its use. For example, only a variable or a parameter

identifier can appear on the left side of an assignment statement, and only a type identifier can appear as the element type of an array. If identifiers with any other roles are found in these positions, they should be flagged as semantic errors.

How does an identifier acquire a role to begin with? It does so in its declaration, where the parser has enough contextual information to determine it. But then how can the parser check for this role much later, when the identifier is used? The identifier's role is saved with it in the symbol table, where it can be accessed when needed. Just as in scope analysis, role analysis uses the symbol table to share contextual information about identifiers among otherwise independent parsing methods.²³

Role analysis requires modifications to the `SymbolEntry` class and the `Parser` class. A `SymbolEntry` object now includes a role field as well as a name field. The possible values for a role are defined as constants in this class. They include `CONST`, `VAR`, `TYPE`, `PROC`, and `PARAM`, with the obvious meanings. In addition to this new attribute and its possible values, the `SymbolEntry` class defines two new methods, `setRole` and `append`. We will see how these are used in a moment.

The parser sets the appropriate role of a `SymbolEntry` object when its identifier is declared. For example, the method `typeDeclaration` introduces a new type name in a TinyAda program. Thus, after the name is entered into the symbol table, the role `TYPE` is entered into the name's `SymbolEntry`, as follows:

```
/*
typeDeclaration = "type" identifier "is" typeDefinition ";"
*/
private void typeDeclaration(){
    accept(Token.TYPE, "'type' expected");
    SymbolEntry entry = enterId();
    entry.setRole(SymbolEntry.TYPE);      // Establish the ID's role as TYPE
    accept(Token.IS, "'is' expected");
    typeDefinition();
    accept(Token.SEMI, "semicolon expected");
}
```

Other type names in a TinyAda program include the built-in data types, which are entered into the symbol table when the table is created.

To access and check the role of an identifier, the parser just looks in its `SymbolEntry` after looking up the identifier in the symbol table. If the found role does not match the expected role, the parser prints an error message and continues processing. Occasionally, a phrase can allow identifiers with different roles. For example, variables, parameters, and constants can appear in expressions, but not type names or procedure names.

The utility method `acceptRole`, which takes a `SymbolEntry`, an error message, and either a role or a set of roles as arguments, performs the required role checking. Here is the code for this method, and

²³ Note that parsers for real-world languages such as Ada typically do not rely on the symbol table to perform static semantic analysis beyond scope analysis. Instead, they generate a parse tree during syntax analysis and then perform traversals of that tree to install and access semantic information for further processing. This type of strategy is the subject of courses in compiler construction.

its use in the parsing method `index`. Note that the `acceptRole` method is overloaded for a single role and for a set of roles.

```
private void acceptRole(SymbolEntry s, int expected, String errorMessage){
    if (s.role != SymbolEntry.NONE && s.role != expected)
        chario.putError(errorMessage);
}

private void acceptRole(SymbolEntry s, Set<Integer> expected,
                        String errorMessage){
    if (s.role != SymbolEntry.NONE && ! (expected.contains(s.role)))
        chario.putError(errorMessage);
}

/*
index = range | <type>name
*/
private void index(){
    if (token.code == Token.RANGE)
        range();
    else if (token.code == Token.ID){
        SymbolEntry entry = findId();
        acceptRole(entry, SymbolEntry.TYPE, "must be a type name");
    }
    else
        fatalError("error in index type");
}
```

The `acceptRole` methods test the entry's role for the value `SymbolEntry.NONE`. This will be the role if the identifier was not even found in the symbol table. Thus, no role error should be reported in this case.

Some TinyAda declarations, such as `index`, introduce a single identifier, whereas others, such as `parameterSpecification` and `enumerationTypeDefinition`, introduce a list of identifiers. To ease the entry of attributes, all of which are the same for a list of identifiers, the `SymbolEntry` class now includes a `next` field, which can serve as a link to the entry for the next identifier in a list. To build this list, the parser obtains the entry for the first identifier in the list and runs the `SymbolEntry` method `append` to append the entry of each remaining identifier to the list. The `SymbolEntry` method `setRole` can then set the same role in the entire list of entries. The parsing methods `enumerationTypeDefinition` and `identifierList` show how this mechanism works. The `identifierList` method now returns a `SymbolEntry` object, which is actually the head of a list of `SymbolEntry` objects. The `enumerationTypeDefinition` method receives this object and sets its role, just as if it represented a single identifier. Here is the code for these two parsing methods:


```

/*
enumerationTypeDefinition = "(" identifierList ")"
*/
private void enumerationTypeDefinition(){
    accept(Token.L_PAR, '(' expected);
    SymbolEntry list = identifierList(); // Obtain the list of IDs
    list.setRole(SymbolRef.CONST);      // All of the IDs are now constants
    accept(Token.R_PAR, ')' expected);
}

/*
identifierList = identifier { "," identifier }
*/
private SymbolEntry identifierList(){
    SymbolEntry list = enterId();
    while (token.code == Token.COMMA){
        token = scanner.nextToken();
        list.append(enterId());          // Grow the list of entries
    }
    return list;
}

```

The remaining modifications to the parsing methods for role analysis are left as an exercise for the reader. Still other attributes of identifiers will be used to enforce further semantic restrictions in later chapters.

Exercises

- 7.1 For the languages C and Java, give as precise binding times as you can for the following attributes. Explain your answers.
 - (a) The maximum number of significant digits in a real number
 - (b) The meaning of *char*
 - (c) The size of an array variable
 - (d) The size of an array parameter
 - (e) The location of a local variable
 - (f) The value of a constant
 - (g) The location of a function
- 7.2 Discuss the relative advantages of early binding and late binding.

- 7.3 In FORTRAN, global variables are created using a `COMMON` statement, but it is not required that the global variables have the same name in each subroutine or function. Thus, in the code

```
FUNCTION F
COMMON N
...
END

SUBROUTINE S
COMMON M
...
END
```

variable `N` in function `F` and variable `M` in subroutine `S` share the same memory, so they behave as different names for the same global variable. Compare this method of creating global variables to that of `C`. How is it better? How is it worse?

- 7.4 Discuss the advantages and disadvantages of `C`'s declaration before use rule with `C++`'s relaxation of that rule to extend the scope of a declaration inside a class declaration to the entire class, regardless of its position in the declaration.
- 7.5 `C++` relaxes `C`'s rule that all declarations in a block must come at the beginning of the block. In `C++`, declarations can appear anywhere a statement can appear. Discuss this feature of `C++` from the point of view of (a) the principle of locality; (b) language complexity; (c) implementation complexity.
- 7.6 Describe the bindings performed by a `C` extern declaration. Why is such a declaration necessary?
- 7.7 As noted in this chapter, `C` and `C++` make a distinction between a **declaration** and a **definition**. Which of the following are declarations only and which are definitions?
- (a) `char * name;`
 - (b) `typedef int * IntPtr;`
 - (c) `struct rec;`
 - (d) `int gcd(int,int);`
 - (e) `double y;`
 - (f) `extern int z;`
- 7.8 Using the first organization of the symbol table described in the text (a single simple table), show the symbol table for the following `C` program at the three points indicated by the comments (a) using lexical scope and (b) using dynamic scope. What does the program print using each kind of scope rule?

```
#include <stdio.h>

int a,b;

int p(){int a, p;
/* point 1 */
```

(continues)

(continued)

```

    a = 0; b = 1; p = 2;
    return p;
}

void print(){
printf("%d\n%d\n",a,b);
}

void q (){
int b;
    /* point 2 */
    a = 3; b = 4;
    print();
}

main(){
/* point 3 */
    a = p();
    q();
}

```

- 7.9 Using the second organization of the symbol table described in the text (a stack of tables), show the symbol table for the following Ada program at the three points indicated by the comments **(a)** using lexical scope and **(b)** using dynamic scope. What does the program print using each kind of scope rule?

```

procedure scope2 is
a, b: integer;
function p return integer is
a: integer;
begin -- point 1
    a := 0; b := 1; return 2;
end p;

procedure print is
begin -- point 2
    put(a); new_line; put(b); new_line; put(p);
    new_line;
end print;

procedure q is
b, p: integer;
begin -- point 3
    a := 3; b := 4; p := 5; print;
end q;

```

(continues)

(continued)

```
begin
  a := p;
  q;
end scope2;
```

- 7.10 Describe the problem with dynamic scoping.
- 7.11 Sometimes the symbol table is called the “static environment.” Does that seem right to you? Why or why not?
- 7.12 Is extent the same as dynamic scope? Why or why not?
- 7.13 An alternative organization for a simple symbol table in a block-structured language to that described in the text is to have a stack of names, rather than a stack of symbol tables. All declarations of all names are pushed onto this stack as they are encountered and popped when their scopes are exited. Then, given a name, its currently valid declaration is the first one found for that name in a top-down search of the stack.
- (a) Redo Exercise 7.8 with this organization for the symbol table.
- (b) Sometimes this organization of the symbol table is called “deep binding,” while the organization described in the text is called “shallow binding.” Why are these terms appropriate? Should one of these organizations be required by the semantics of a programming language? Explain.
- 7.14 The following program prints two integer values; the first value typically is garbage (or possibly 0, if you are executing inside a debugger or other controlled environment), but the second value might be 2. Explain why.

```
#include <stdio.h>

void p(void){
  int y;
  printf("%d\n", y);
  y = 2;
}

main(){
  p(); p();
  return 0;
}
```

- 7.15 The following program prints two integer values; the first value typically is garbage (or possibly 0, if you are executing inside a debugger or other controlled environment), but the second value might be 2. Explain why.

```
#include <stdio.h>
main()
{
  { int x;
```

(continues)

(continued)

```
    printf("%d\n",x);
    x = 2;
}
{ int y;
  printf("%d\n",y);
}
return 0;
}
```

7.16 Explain the difference between aliasing and side effects.

7.17 Suppose that the following function declarations (in C++ syntax) are available in a program:

```
(1) int pow(int, int);
(2) double pow(double, double);
(3) double pow(int, double);
```

and suppose that the following code calls the `pow` function:

```
int x;
double y;
x = pow(2,3);
y = pow(2,3);
x = pow(2,3.2);
y = pow(2,3.2);
x = pow(2.1,3);
y = pow(2.1,3);
x = pow(2.1,3.2);
y = pow(2.1,3.2);
```

Given the language **(a)** C++; **(b)** Java; or **(c)** Ada, write down the number of the `pow` function called in each of the eight calls, or write “illegal” if a call cannot be resolved in the language chosen, or if a data type conversion cannot be made.

7.18 Pointers present some special problems for overload resolution. Consider the following C++ code:

```
void f( int* x) { ... }

void f( char* x) { ... }

int main()
{ ...
  f(0);
  ...
}
```

What is wrong with this code? How might you fix it? Can this problem occur in Java? In Ada?

- 7.19 Default arguments in function definitions, such as:

```
void print( int x, int base = 10);
```

also present a problem for overload resolution. Describe the problem, and give any rules you can think of that might help resolve ambiguities. Are default arguments reasonable in the presence of overloading? Explain.

- 7.20 Figure 7.27 of the text illustrates Java namespace overloading, which allows the programmer to use the same name for different kinds of definitions, such as classes, methods (functions), parameters, and labels. In the code of Figure 7.27, there are 12 appearances of the name A. Describe which of these appearances represent definitions, and specify the kind of definition for each. Then list the other appearances that represent uses of each definition.

- 7.21 The text mentions that the lookup operation in the symbol table must be enhanced to allow for overloading. The insert operation must also be enhanced.

- (a) Describe in detail how both operations should behave using a standard interface for a dictionary data structure.
- (b) Should the symbol table itself attempt to resolve overloaded names, or should it leave that job to other utilities in a translator? Discuss.

- 7.22 Many programming languages (including C, C++, Java, and Ada) prohibit the redeclaration of variable names in the same scope. Discuss the reasons for this rule. Could not variables be overloaded the same way functions are in a language like C++ or Ada?

- 7.23 A common definition of “side effect” is a change to a nonlocal variable or to the input or output made by a function or procedure. Compare this definition of “side effect” to the one in the text.

- 7.24 (a) Which of the following C expressions are l-values? Which are not? Why? (Assume x is an int variable and y is an int* variable.)

- (1) x + 2
- (2) &x
- (3) *&x
- (4) &x + 2
- (5) *(&x + 2)
- (6) &*y

- (b) Is it possible for a C expression to be an l-value but not an r-value? Explain.

- (c) Is &(&z) ever legal in C? Explain.

- 7.25 Show how to use an address-of operator such as & in C to discover whether a translator is implementing assignment by sharing or assignment by cloning, as described in Section 7.6.1.

- 7.26 Given the following C program, draw box-and-circle diagrams of the variables after each of the two assignments to **x (lines 11 and 15). Which variables are aliases of each other at each of those points? What does the program print?

```
(1) #include <stdio.h>

(2) main(){
(3)     int **x;
```

(continues)

(continued)

```
(4)  int *y;
(5)  int z;
(6)  x = (int**) malloc(sizeof(int*));
(7)  y = (int*) malloc(sizeof(int));
(8)  z = 1;
(9)  *y = 2;
(10) *x = y;
(11) **x = z;
(12) printf("%d\n", *y);
(13) z = 3;
(14) printf("%d\n", *y);
(15) **x = 4;
(16) printf("%d\n", z);
(17) return 0;
(18) }
```

- 7.27 Explain the reason for the two calls to `malloc` (lines 6 and 7) in the previous exercise. What would happen if line 6 were left out? Line 7?
- 7.28 Repeat Exercise 7.26 for the following code:

```
(1) #include <stdio.h>

(2) main()
(3) { int **x;
(4)  int *y;
(5)  int z;
(6)  x = &y;

(7)  y = &z;
(8)  z = 1;
(9)  *y = 2;
(10) *x = y;
(11) **x = z;
(12) printf("%d\n", *y);
(13) z = 3;
(14) printf("%d\n", *y);
(15) **x = 4;
(16) printf("%d\n", z);
(17) return 0;
(18) }
```

- 7.29 A **single-assignment variable** is a variable whose value can be computed at any point but that can be assigned to only once, so that it must remain constant once it is computed. Discuss the usefulness of this generalization of the idea of constant compared to constant declarations in C, C++, Java, or Ada. How does this concept relate to that of a dynamic constant, as discussed in Section 7.6?

7.30 In Ada, an object is defined as follows: “An object is an entity that contains a value of a given type.” This means that there are two kinds of objects: constants and variables. Compare this notion of an object with the one discussed in Section 7.6.

7.31 Why is the following C code illegal?

```
{ int x;
  &x = (int *) malloc(sizeof(int));
  ...
}
```

7.32 Why is the following C code illegal?

```
{ int x[3];
  x = (int *) malloc(3*sizeof(int));
  ...
}
```

7.33 Here is a legal C program with a function variable:

```
(1) #include <stdio.h>

(2) int gcd( int u, int v){
(3)     if (v == 0) return u;
(4)     else return gcd(v, u % v);
(5) }
(6) int (*fun_var)(int,int) = &gcd;

(7) main(){
(8)     printf("%d\n", (*fun_var)(15,10));
(9)     return 0;
(10) }
```

Compare lines 6 and 8 of this code to the equivalent code of Figure 7.45 (lines 1 and 3). Why is this version of the code permitted? Why is it not required?

7.34 Make a list of the rules in the EBNF of TinyAda (Section 6.8) where identifiers are declared, and modify the code for your parser so that their names are entered into the symbol table when these declarations are encountered.

7.35 Make a list of rules in the EBNF of TinyAda where identifiers are referenced, and modify the relevant parsing methods to look up these identifiers in the symbol table.

7.36 Make a list of the rules in the EBNF of TinyAda (Section 6.8) where identifier roles are introduced in declarations, and modify the code for your parser so that this information is recorded in symbol entries when these declarations are encountered.

7.37 Make a list of rules in the EBNF of TinyAda where identifiers are referenced, and modify the code to accept the roles of these identifiers, using appropriate error messages.

Notes and References

Most of the concepts in this chapter were pioneered in the design of Algol60 (Naur [1963a]), except for pointer allocation. Pointer allocation was, however, a part of the design of Algol68 (Tanenbaum [1976]). Some of the consequences of the design decisions of Algol60, including the introduction of recursion, may not have been fully understood at the time they were made (Naur [1981]), but certainly by 1964 the full implementations of Algol60 used most of the techniques of today's translators (Randell and Russell [1964]). For more detail on these techniques, consult Chapter 10 and a compiler text such as Louden [1997] or Aho, Sethi, and Ullman [1986].

Structured programming, which makes use of the block structure in an Algol-like language, became popular some time later than the appearance of the concept in language design (Dahl, Dijkstra, and Hoare [1972]). For an interesting perspective on block structure, see Hanson [1981].

The distinction that C and C++ make between definition and declaration is explained in more detail in Stroustrup [1997] and Kernighan and Ritchie [1988]. The effect of a declaration during execution, including its use in allocation, is called **elaboration** in Ada and is explained in Cohen [1996].

The term “object” has almost as many definitions as there are programming languages. For definitions that differ from the one used in this chapter, see Exercise 7.30 and Chapter 6. The notions of symbol table, environment, and memory also change somewhat from language to language. For a more formal, albeit simple, example of an environment function, see Chapter 12. For a more detailed discussion of the theoretical representation of environments, see Meyer [1990]. For a more detailed description of environments in the presence of procedure calls, see Chapter 10.

Overloading in C++ is discussed in detail in Stroustrup [1997]; overloading in Java in Arnold, Gosling, and Holmes [2000] and Gosling, Joy, Steele, and Bracha [2000]; overloading in Ada in Cohen [1996]. As an aside, overloading rules and ambiguity resolution occupy a significant portion of the C++ language standard (more than 50 pages, including one entire chapter).

Aliases and the design of the programming language Euclid, which attempts to remove all aliasing, are discussed in Popek et al. [1977] (see also Lampson et al. [1981]). Techniques for garbage collection and automatic storage management have historically been so inefficient that their use in imperative languages has been resisted. With modern advances that has changed, and many modern object-oriented languages require it (Java, Python, Ruby) and do not suffer greatly from execution inefficiency. Functional languages, too, have improved dramatically in efficiency while maintaining fully automatic dynamic allocation. For an amusing anecdote on the early use of garbage collection, see McCarthy [1981, p. 183]. Garbage collection techniques are studied in Chapter 10.

For a perspective on the problem of dynamic scoping and static typing, see Lewis et al. [2000].